SecureDBAgent: Shielding Databases from Agentic AI

RAJANI NAGARAJU

Final Thesis Report

DECEMBER 2025

DEDICATION

I dedicate this thesis to my family, particularly my daughter and my husband, in appreciation of their unwavering support and encouragement throughout my Master's studies.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my professor and thesis guide, Mr. Randheer Bhagi, for his invaluable guidance, continuous encouragement and constructive feedback throughout the course of this work. His academic expertise and thoughtful supervision were instrumental in the successful completion of this thesis.

I would also like to extend my heartful thanks to UpGrad and Liverpool John Moores University for providing the academic platform, resources and learning environment that enabled me to pursue and complete my Master's studies.

Finally, I would like to express my heartfelt appreciation to my family, especially my daughter and my husband, for their patience, understanding, and unwavering support throughout this academic journey. Their encouragement and belief in me were a constant source of motivation. A special note of gratitude goes to my husband, whose persistent encouragement was the reason I dared to pursue this degree at this stage of my life. Without his motivation, I would never have considered taking this leap. His unwavering support in every aspect - emotional, practical and beyond is something I will remain deeply grateful for, always.

ABSTRACT

Large Language Models (LLMs) are increasingly deployed as autonomous agents capable of generating and executing SQL queries directly against enterprise databases. While this enables flexible, natural-language-driven data access, it also introduces security risks that are not adequately addressed by traditional database controls or existing ML based SQL injection detectors. In agentic workflows, unsafe queries often arise from contextual misuse such as role violations, unauthorized access to sensitive schemas, abnormal session behavior or provenance anomalies, rather than purely malicious syntax.

This research presents SecureDBAgent, a multi-layered framework for securing database access in LLM-driven systems through the combined use of deterministic policy enforcement and learned threat detection. The framework incorporates a context policy engine that evaluates generated SQL queries against role-based access rules, schema sensitivity tags, permitted operations, session dynamics and historical provenance, enabling early blocking or escalation of unsafe queries. Queries that pass contextual checks are further analysed using a hybrid threat classification pipeline that fuses statistical structural features with semantic representations learned via a token-level convolutional neural network. A multilayer perceptron processes the fused representation to estimate maliciousness probabilities across diverse attack patterns.

To support operational transparency, SecureDBAgent integrates an explainability module combining rule level policy justifications, SHAP based static feature attribution and token level saliency analysis. Evaluation is performed using multiple open SQL datasets containing benign, adversarial and real-world workloads, augmented with policy-aware and security-aware ground truth labels. Results demonstrate high detection accuracy, low false positive rates, substantial coverage through contextual enforcement and minimal per-query latency overhead.

The proposed framework demonstrates that combining governance aware policy reasoning with statistical and semantic threat modelling provides a practical and scalable approach to securing database access in agentic AI systems.

TABLE OF CONTENTS

Table of Contents

DEDICATION ii

ACKNOWLEDGEMENTS iii

ABSTRACT iv

LIST OF TABLES xii

LIST OF FIGURES xiii

LIST OF ABBREVIATIONS xiv

1. INTRODUCTION 1

1.1 Background of the study 1

1.2 Problem Statement 3

1.3 Aim and Objectives 4

1.4 Research questions 5

1.5 Scope of the study 6

1.6 Significance of the study 7

1.7 Structure of the study 8

2. LITERATURE REVIEW 11

2.1 Introduction 11

2.2 Evolution of Database systems and expanding attack surfaces 12

2.3 Classical SQL injection attacks and early defensive mechanisms 13

2.4 Rule-based, Policy-driven and Static defensive mechanisms 14

2.5 Machine learning approaches for SQL Injection detection 16

2.6 Deep Learning and Semantic Modelling of SQL queries 17

2.7 Identified research gaps and Theoretical framework 18

2.7.1 Identified Research Gaps 18

2.7.2 Theoretical framework underpinning this research 20

2.7.3 Positioning of the present research 21

3. RESEARCH METHODOLODY 22

3.1 Chapter overview 22

3.2 Data gathering 23

3.3 Research Design and Methodological Approach 24

3.3.1 Rationale for a Quantitative experimental approach 24

3.3.2 Supervised learning Paradigm 24

3.3.3 Multi-stage hybrid system design 25

3.3.4 Experimental evaluation strategy 25
3.3.5 Alignment with research objectives 26
3.4 Overall System Architecture and End-to-End flow 26
3.4.1 Architectural Design Principles 26
3.4.2 High-Level Architectural Components 27
3.4.3 System architecture 27
3.4.4 End-to-End Query Processing Workflow 28
3.4.5 Architectural Novelty and Contribution 29
3.5 Context Policy engine methodology 29
3.5.1 Motivation for a Context-aware policy layer 29
3.5.2 Design principles and Threat model assumptions 30
3.5.3 Context Policy Configuration Model and Administrative Control Plane 31
3.5.4 Query Parsing and Context Extraction 36
3.5.5 Hard Deterministic RBAC and Governance Checks 36
3.5.6 Detector Suite Overview and Rationale 37
3.5.7 Plausibility Detector Methodology (Three-Factor Scoring) 37
3.5.8 Provenance Novelty Detector 38
3.5.9 Session anomaly detector 39
3.5.10 Schema Integrity Detector 39
3.5.11 Decision Fusion logic and Context action output 39
3.5.12 Contextual metadata augmentation for Evaluation 41
3.5.13 Policy and Security Ground-Truth Augmentation 41
3.5.14 Novelty contribution of Context policy engine 43
3.6 Statistical Feature Extractor 43
3.6.1 Motivation for Statistical Feature Extraction 43
3.6.2 Design Principles 44
3.6.3 Feature categories 44
3.6.4 Statistical Feature set 45
3.6.5 Integration with hybrid detection pipeline 46
3.6.6 Contribution to Explainability and Policy enforcement 46
3.6.7 Summary 47
3.7 CNN based Token Encoder 47
3.7.1 Motivation for semantic encoding of SQL queries 47
3.7.2 Why CNN for SQL token encoding? 47
3.7.3 Comparative Justification of Encoder choices 48
3.7.4 Tokenization and input representation 49

3.7.5 CNN Encoder architecture 50
3.7.6 Regularization and Generalization strategy 50
3.7.7 Role of CNN embedding in overall framework 51
3.7.8 Summary 51
3.8 Feature Fusion Methodology 51
3.8.1 Motivation for feature fusion 51
3.8.2 Feature spaces to be fused 52
3.8.3 Fusion strategy and mathematical formulation 52
3.8.4 Normalization and Feature scaling 53
3.8.5 Comparative analysis against late fusion and attention-based fusion 53
3.8.6 Contribution to Novelty 53
3.9 Multi Layer Perceptron - Threat Classifier architecture 53
3.9.1 Rationale for Selecting an MLP Classifier 53
3.9.2 Architecture of the MLP Classifier 54
3.9.3 Regularization and Stability Considerations 55
3.9.4 Why MLP and Not Other Classification Models? 55
3.9.5 Contribution to Methodological Novelty 55
3.10 Risk Tier Decision 56
3.10.1 Motivation for a Risk-Tiered Decision Framework 56
3.10.2 Inputs to the Risk Tier Decision Layer 56
3.10.3 Definition of Risk Tiers 57
3.10.4 Threshold-Based Risk Assignment 57
3.10.5 Threshold Optimisation Strategy 57
3.10.6 Justification for Using Both ROC and PR Curves 58
3.10.7 Integration with Context Policy Decisions 58
3.10.8 Final Decision Logic 59
3.10.9 Contribution to Novelty 59
3.10.10 Summary 59
3.11 Explainability and Decision transparency 59
3.11.1 Motivation for explainability in database security 59
3.11.2 Design principles of Explainability module 60
3.11.3 Static feature attribution using SHAP 60
3.11.4 Token level saliency for semantic interpretation 60
3.11.5 Contextual rule traceability 61
3.11.6 Unified explainability output 61
3.11.7 Summary 61

3.12 Experimental Protocol, Reproducibility and Validation strategy 62
3.12.1 Motivation for a Rigorous Experimental Protocol 62
3.12.2 Dataset Partitioning Strategy 62
3.12.3 Reproducibility Controls and Determinism 62
3.12.4 Hyperparameter Optimisation Protocol 63
3.12.5 Threshold Validation Protocol 64
3.12.6 Evaluation Under Known and Unknown Conditions 64
3.12.7 Validation of Context Policy Coverage 64
3.12.8 Ethical and Privacy Considerations 65
3.12.9 Summary 65
3.13 Requirements 65
3.13.1 Programming Language and Execution Platform 66
3.13.2 Core Software Libraries and Frameworks 66
3.13.3 Explainability and Auditability Tooling 67
3.13.4 Directory Structure and Modular Organization 68
3.13.5 Reproducibility and Experimental Control 68
3.13.6 Alignment with Real-World Deployment Constraints 68
4. ANALYSIS 70
4.1 Overview of Analytical Objectives 70
4.1.1 Purpose of this chapter 70
4.1.2 Analytical methodology adopted 71
4.2 Dataset composition and label distribution 71
4.2.1 Overview of Data Sources 72
4.2.2 SQLiV5 Dataset Characteristics 72
4.2.3 Kaggle SQL Injection Dataset Characteristics 73
4.2.4 Spider Dataset as a Benign Stress Test 73
4.2.5 Data Pre-processing 73
4.2.6 Exploratory Data analysis 75
4.2.7 Distribution of Policy and Security Violations 78
4.2.8 Summary of Analytical Observations 79
4.3 SQL Tokenization and Sequence length Analysis 79
4.3.1 Need for SQL-Aware Tokenisation 79
4.3.2 Token Categories and Normalisation Strategy 80
4.3.3 Vocabulary Construction and Minimum Frequency Threshold 80
4.3.4 Empirical Analysis of Sequence Length Distribution 81
4.3.5 Impact on Downstream Representation Learning 83

4.4 CNN Encoder Input-Output Geometry and Feature representation 83
4.4.1 From SQL Query to Token Sequence 83
4.4.2 Embedding Layer: Dense Token Representation 84
4.4.3 Convolutional Feature Extraction with Multi-Scale Kernels 84
4.4.4 Global Max Pooling: Temporal Compression 85
4.4.5 Concatenation and Semantic Vector Formation 85
4.4.6 CNN Hyperparameter Optimisation and Empirical Selection 86
4.4.7 End-to-End Dimensional Flow Summary 87
4.5 Feature Fusion Dynamics and Representation Complementarity 88
4.5.1 Motivation for Feature Fusion 88
4.5.2 Constituent Feature Spaces 88
4.5.3 Fusion Strategy and Dimensional Integration 89
4.5.4 Normalisation and Scale Alignment 89
4.5.5 Implications for Downstream Explainability 90
4.5.6 Summary 90
4.6 Threat Classification Behaviour and Decision Boundary Analysis 90
4.6.1 Classification Architecture Recap and Input Space 90
4.6.2 Hyperparameter optimization of MLP Threat classifier 91
4.6.3 Threshold Sensitivity Analysis using Youden's Index 93
4.6.4 Risk-Aware Threshold Banding and Action Mapping 94
4.6.5 Comparative Ablation Analysis of Model Architectures 95
4.6.6 Decision Boundary Robustness and Error Profile 96
4.6.7 Implications for SecureDB Agent Decision Logic 96
4.7 Impact of Context Policy Enforcement on Threat Detection Outcomes 96
4.7.1 Role of Context Policy in the SecureDB Agent Pipeline 96
4.7.2 Context Policy Coverage on Malicious Queries 97
4.7.3 Reduction in ML Decision Surface and Error Exposure 98
4.7.4 Complementarity Between Contextual Rules and Learned Representations 98
4.7.5 Impact on System Latency and Operational Viability 99
4.7.6 Analytical Implications for Agentic AI Security 99
4.8 Explainability Analysis and Model Interpretability 99
4.8.1 Motivation for Explainability in SecureDBAgent 99
4.8.2 Case-Level Explainability: End-to-End Decision Trace 100
4.8.3 Local Static Feature Attribution using SHAP 101
4.8.4 Token-Level Saliency Analysis of CNN Encoder 102
4.8.5 Global Static Feature Importance 102

4.8.6 Feature Behaviour Distribution and Stability (SHAP Beeswarm) 103
4.8.7 Explainability Across Decision Sources 104
4.8.8 Summary of Explainability Findings 104
5. RESULTS AND DISCUSSIONS 105
5.1 Chapter Overview and Reporting Logic 105
5.2 Model Ablation Results - Static-only vs. CNN-only vs. Fusion 105
5.3 Full Pipeline Performance - Context Policy + Fusion 107
5.3.1 Impact of Policy-Aware Ground Truth on Detection Performance 107
5.3.2 Full pipeline evaluation results 108
5.3.3 Reason for not including Full pipeline model in Ablation Study 109
5.4 Risk Tier Thresholding 110
5.5 Context Policy Coverage on Malicious Queries 111
5.6 Operational Latency Across Models 113
5.7 Explainability Results and Security Interpretation 114
5.7.1 Case-based XAI dashboard: end-to-end decision trace 114
5.7.2 Local SHAP: structural reasons for a single decision 114
5.7.3 Token-level saliency: semantic triggers within the SQL sequence 115
5.7.4 Global SHAP and beeswarm: stable drivers of risk across the test set 115
5.8 Summary of Key Findings 115
6. CONCLUSIONS AND RECOMMENDATIONS 117
6.1 Summary of Research objectives and achievements 117
6.2 Key Findings and their Implications 118
6.2.1 Effectiveness of Context-Aware Security Mediation 118
6.2.2 Complementarity of Static and Semantic Representations 118
6.2.3 Importance of Risk-Tiered Decisioning 118
6.2.4 Explainability as an Operational Requirement, Not an Add-on 119
6.2.5 Performance and Deployability Considerations 119
6.3 Contributions to Knowledge and Practice 119
6.4 Limitations 120
6.5 Recommendations for Future Work 120
6.5.1 Adaptive and Self-Learning Context Policies 120
6.5.2 Cross-Query and Session-Level Reasoning 121
6.5.3 Deeper Integration with LLM Prompt Context 121
6.5.4 Real-Time Production Integration 121
6.5.5 Expanded Explainability for Non-Technical Stakeholders 121
6.6 Final Conclusion 121

REFERENCES 122
APPENDIX A: RESEARCH PROPOSAL 124

LIST OF TABLES

Table 3.1 Role Policy Configuration Model 32
Table 3.2 Knowledge Base (KB) Artefacts Catalogue 34
Table 3.3 Prefilter and Detector Threshold Configuration.....40
Table 3.4 Statistical Features Used for SQL Query Characterization……...45
Table 3.5 Comparison of Token Encoding Architectures for SQL Threat Detection……48
Table 3.6 Model Comparison for Threat classifier……55
Table 4.1 Policy action distribution……...78
Table 4.2 Security Violation distribution……...78
Table 4.3 Performance metrics across different thresholds……93
Table 4.4 Threshold Band to Action Mapping……94
Table 4.5 Model wise Performance comparison……. 95
Table 5.1 Threshold -> Action mapping table……111

LIST OF FIGURES

Figure 3.1: SecureDB Agent: System Architecture…….27
Figure 3.2: Context-Policy Engine HLD…….30
Figure 4.1: Data Cleaning summary……74
Figure 4.2: Structural features density graph……...75
Figure 4.3: Presence of Obfuscation markers……..76
Figure 4.4: Correlation heatmap of Numeric features…….77
Figure 4.5: Vocab size and coverage across multiple thresholds for defining rare tokens…..80
Figure 4.6: Distribution of Token count by label…….81
Figure 4.7: Overall Token count distribution……...82
Figure 4.8: Emperical CDF of token count……..82
Figure 4.9: CNN Hyperparameter Tuning results……86
Figure 4.10: MLP Hyperparameter Tuning results……..92
Figure 4.11: Context policy coverage on Malicious queries……99
Figure 4.12: XAI Response - Example Query 1…….100
Figure 4.13: XAI Response - Example Query 2…….100
Figure 4.14: Global SHAP Importance……...102
Figure 4.15: SHAP Beeswarm plot……103
Figure 5.1: Model Ablation Results……106
Figure 5.2: Full Pipeline Result (Context Policy+Fusion model)……...108
Figure 5.3: Context Policy Coverage on Malicious queries……111
Figure 5.4: Operational Latency across Models……..113

LIST OF ABBREVIATIONS

LLM.....	Large Language Model
ML.....	Machine learning
SQL.....	Structured Query Language
SQLi.....	SQL Injection
CNN.....	Convolution neural network
RBAC.....	Rule based access control
HLD.....	High level design
ROC.....	Receiver Operating Characteristic
TPR.....	True positive rate
FPR.....	False positive rate
NL2SQL.....	Natural Language to SQL
SHAP.....	SHapley Additive exPlanations
GPT.....	Generative Pre-trained Transformer
PaLM.....	Pathways Language Model
LLaMA.....	Large Language Model Meta AI
GDPR.....	General Data Protection Regulation
NIST.....	National Institute of Standards and Technology
EU AI.....	European Union Artificial Intelligence Act
XAI.....	Explainable Artificial Intelligence
IDS.....	Intrusion Detection system
RDBMS.....	Relational Database Management System
HTTP.....	Hypertext Transfer Protocol
OWASP.....	Open Worldwide Application Security Project
SOA.....	Service oriented architecture
WAF.....	Web application Firewall
SVM.....	Support Vector Machine
RNN.....	Recurrent Neural Network
LSTM.....	Long Short-Term Memory
BERT.....	Bidirectional Encoder Representations from Transformers
AUC.....	Area Under the Curve

CHAPTER 1: INTRODUCTION

This chapter sets the context, defines the research problem, and explain the study's significance.

1.1 Background of the study

Over the last two decades, database systems have undergone a structural transformation driven by the rapid expansion of data centric applications, automation pipelines, and intelligent computational agents. Traditionally, the interaction between users and databases followed a deterministic pattern - SQL queries were manually authored by developers or generated by fixed application logic. Security controls including role-based access control (RBAC), parameterised queries, and network level filtering were designed for a world in which SQL was predictable, human-crafted, and constrained by application interfaces. This paradigm began shifting as LLM systems became capable of generating structured commands autonomously, fundamentally altering the assumptions underpinning database security.

The introduction of Natural-language-to-SQL (NL2SQL) models signalled a major advancement in this direction. Benchmarks such as Spider (Yu et al., 2018) demonstrated that neural models could infer schema relationships, compose joins, and generate executable SQL from ambiguous natural language prompts. More recently, large language models (LLMs) such as GPT-4, PaLM, and LLaMA have shown even higher proficiency in structured reasoning (OpenAI, 2023; Chowdhery et al., 2022; Touvron et al., 2023). While these models have enabled sophisticated analytics assistants and conversational BI systems, they also introduce risks that traditional database security mechanisms are not equipped to manage. LLMs may hallucinate non-existent tables, produce privilege escalating SELECT clauses or embed implicit tautologies, resulting in queries that violate enterprise security policies without malicious intent (Zhao et al., 2023).

The broader security research landscape has recognised the challenges associated with model hallucination, semantic drift and unsafe autonomous behaviour. Studies such as "LLMs Can Generate Vulnerable Code" (Pearce et al., 2023) and "Prompt Injection Attacks Against LLMs" (Greshake et al., 2023) highlight how LLM mediated execution pathways can expose new attack surfaces. While these works primarily focus on code generation or prompt-level manipulation, they underscore a common problem: LLMs often produce syntactically valid but semantically dangerous outputs. When projected into the context of enterprise databases, this

means that AI generated SQL can be incorrect, incomplete or harmful even in the absence of adversarial intent.

In parallel with developments in AI, security models have gradually shifted toward behaviour aware and context aware frameworks. Contemporary research in adaptive access control argues that static RBAC rules are insufficient for detecting sophisticated misuse patterns (Hu et al., 2021; Colantonio et al., 2019). Similarly, anomaly-based intrusion detection systems (IDS) have become more prominent in handling insider threats and operational misuse, especially in environments where traditional perimeter defences are insufficient (Sharma & Jose, 2022). Database specific anomaly detection research involving query entropy, lexical features and behavioural profiling demonstrate the feasibility of modelling normal query characteristics to identify deviations (Zimmermann et al., 2019). However, these approaches are generally designed for human authored SQL and do not account for the unique failure modes introduced by LLM generated commands.

Another important development is the increasing regulatory emphasis on transparency and accountability in automated systems. Frameworks such as GDPR, NIST AI Risk Management Framework and the EU AI Act emphasise that decision making systems must be explainable, auditable and aligned with human oversight principles. This requirement extends naturally to database security. Organisations must not only block suspicious queries, but also articulate why a query was judged unsafe. Several recent studies underline the limitations of black-box deep learning models in security contexts and advocate for Explainable AI (XAI) mechanisms to support operational trust (Linardatos et al., 2021; Shrikumar et al., 2017). Despite these insights, explainability is seldom integrated into database focused threat classifiers, representing another gap in existing research.

The combined effect of these developments is a widening mismatch between modern AI-mediated query generation and legacy database security frameworks. Traditional SQL injection defences target adversarial payloads such as tautologies, UNION-based exfiltration patterns or timing attacks (Halfond et al., 2006). Yet these methods are ineffective against hallucinated table names, unintended elevated privilege projections, or context inappropriate data access issues introduced by probabilistic LLM reasoning rather than malicious user behaviour. Recent papers acknowledge the existence of "AI-induced security faults" (Hendrycks et al., 2023), but concrete defensive architectures designed for enterprise database governance are still limited.

These gaps justify the need for a new class of database side verification layers that are:

- Context-aware, incorporating behavioural and role specific information.

- Semantically grounded, able to interpret the meaning of SQL structures.
- Explainable, enabling transparent justification of decisions.
- LLM-aligned, capable of mitigating hallucination induced query risks.

The SecureDBAgent proposed in this research addresses this unmet need by integrating deterministic policy rules with statistical anomaly features, semantic embeddings through a CNN encoder, a fusion architecture for threat scoring, and a multi tier risk decision engine. This research is not a reiteration of SQL injection detection nor a replication of anomaly detection frameworks. Rather, it fills a novel gap at the intersection of Agentic AI behaviour, database governance and explainable security enforcement.

In summary, the background of this study is shaped by three converging trends:

1) AI-mediated SQL generation, which challenges assumptions underlying traditional security controls.
2) The rise of context-aware security paradigms, emphasising behaviour, provenance, and risk conditions.
3) The demand for explainable, auditable decision systems, driven both by regulatory requirements and operational necessity.

The research therefore responds to an emerging problem that current academic and industrial solutions have not adequately addressed - ensuring safe, interpretable, and policy-compliant SQL execution in environments where queries are increasingly authored by autonomous AI systems rather than humans.

1.2 Problem Statement

While LLMs increasingly generate SQL on behalf of users and internal systems, organisations currently lack mechanisms to ensure that these outputs comply with security policies and do not inadvertently expose sensitive data. Existing security controls address only parts of the problem:

- RBAC and access-control models presuppose human intent and provide no guarantee that an LLM generated query respects organisational norms.
- Conventional SQL injection detectors focus on attack signatures rather than model driven semantic errors or privilege violating access.
- Neural anomaly detection systems operate without an understanding of enterprise schema, business semantics or user context.

- Text-to-SQL models often hallucinate tables or fields that do not exist (Neeraja et al., 2023) or generate overly broad queries.

Despite these limitations, existing research rarely integrates these viewpoints. Most studies examine either syntactic security (e.g., injection detection) or semantic correctness (e.g., NL2SQL evaluation), but not their combined impact on operational security in autonomous systems.

This gap is becoming increasingly consequential. As organisations begin to delegate database operations to agentic systems, the risk stems not only from attackers but from the model itself, which may unintentionally generate unsafe or inappropriate SQL. The absence of contextual, multi-layered validation means that enterprises are exposed to risks that traditional database security architectures were never designed to handle. These include:

- Semantic but unsafe queries generated by LLMs that syntactically appear legitimate.
- Role and privilege violations due to hallucinated table access.
- Behavioural anomalies indicative of misuse, compromised credentials or agentic drift.
- Ambiguous cases where query intent is unclear and deterministic policies alone cannot offer conclusive decisions.

Therefore, the core problem addressed in this thesis is "How can enterprises enforce safe, context-aware and verifiable SQL execution when queries are generated by agentic AI systems rather than human users?" The study proposes that solving this problem requires a framework that unifies deterministic policy checks with behavioural analytics and machine learning based threat detection, operating as a mediator between LLM-generated actions and the underlying data systems with complete transparency.

1.3 Aim and Objectives

Aim of this research is to design and evaluate a context-aware, multi-layered SecureDBAgent capable of detecting, blocking or flagging unsafe SQL queries generated by LLM-powered systems by integrating deterministic policy enforcement, behavioural modelling, and neural threat classification.

To achieve the above aim, following objectives have been articulated.

1) Design and implement a context-policy engine that evaluates AI-generated SQL queries against role-based access control rules, schema sensitivity, operation intent, session

behaviour and provenance constraints, and to measure its effectiveness in identifying policy violations independently of query maliciousness.

2) Develop a hybrid threat classification model that fuses engineered statistical features with semantic embeddings learned from a CNN encoder, and to train and validate this model on multiple open datasets of benign and malicious SQL.
3) Construct a risk-tier decision layer that categorises SQL queries into Allow, Flag, or Block actions using probabilistic thresholds derived from ROC and precision–recall curve optimisation on validation data.
4) Integrate an explainability module combining SHAP-based static analysis, gradient-based token saliency, and rule-level context justification and to demonstrate its effectiveness in supporting transparent security auditing and human-in-the-loop decision making.
5) Quantitatively evaluate the proposed framework using standard classification metrics (ROC-AUC, F1, FPR), context-policy coverage analysis and zero-day generalisation tests on previously unseen benign workloads from Spider dataset.
6) Deliver a reproducible implementation of the SecureDBAgent and to empirically measure its operational latency across different configurations, demonstrating suitability for real-time enterprise deployment.

The objectives of this study are formulated in alignment with the SMART framework to ensure clarity, feasibility and evaluability. Each objective specifies a concrete system component or evaluation outcome, incorporates measurable criteria, and remains achievable within the scope and duration of this research project.

1.4 Research questions

Based on the problem and objectives, the research investigates the following questions:

1) How can contextual signals such as user roles, table sensitivity, behavioural profiles and provenance be systematically integrated into SQL decision-making?
2) Does combining semantic embeddings with handcrafted statistical features improve malicious SQL detection compared to existing single-modality approaches?
3) Can a unified risk-tier framework effectively reduce false positives while preserving high detection accuracy, particularly for ambiguous LLM-generated queries?
4) How can explainability be provided for machine learning and policy layer decisions in a manner suitable for enterprise audit and governance?

5) What are the measurable performance characteristics of SecureDBAgent, and can it operate with minimal latency in production workflows?

1.5 Scope of the study

This research improves database security with environments where autonomous AI agents generate SQL queries using Large Language Models (LLMs). More specifically, this research aims to detect and block malicious or unauthorized SQL queries by employing a hybrid model involving machine learning classification, role and schema-based policy enforcement and explainable AI.

This research includes the imposition of a supervised classification model trained on datasets approved for use - SQLiV5, Kaggle SQL Injection Dataset along with the Spider dataset as a basis for evaluating the false positives. The model will be utilized to detect a wide variety of SQL-based threats including tautology attacks, time-based blind SQL injection, piggy-backed queries, UNION-based injection, hallucinated table or column access, privilege violations at the schema level, etc., While it will not be able to truly simulate context-awareness, it will have a role-policy engine that applies access permissions to a query based on the definition of role-to-table-tag mappings.

It also provides model-level explainability to explain the key attributes driving query classification decisions thus adding transparency and to aid in supporting security audits as well without exposing internal model complexity.

However, a few areas are explicitly left out of scope to ensure this research is focused, manageable and able to be completed during the given timeframe.

- The agent operates at the SQL-query layer and assumes that authentication and network-layer protections already exist.
- This research will not include training or fine-tuning the LLMs that generate SQL queries. It will not explore the internal workings of LLM prompt engineering. It treats the SQL query as an input and does not include the generation logic or feedback driven prompt regeneration. This ensures a modular design that is model-agnostic and focuses solely on post-generation threat detection and mitigation.
- This research will not address additional areas of threat outside of SQL like NoSQL injection threats, file-based vulnerabilities and front-end security threats like cross-site scripting (XSS).

- This research purposely scopes context-awareness in a simulated, but meaningful manner. Because the test datasets (SQLiV5 and Kaggle SQL Injection Dataset) contain no role annotations or agent-specific metadata, roles are heuristically assigned based on table names. This allows for a reasonable evaluation of policy enforcement, without manipulating the data set. Also, the schema policy engine is implemented using a static model for role-to-table-tag mapping, allowing for real time enforcement. While the implementation of role-based context-awareness is static and predetermined, this work is a solid stepping stone towards future work. In a production scenario, this context could be supplemented even more with dynamic session-based role recognition, agent-specific behavioural profiles or even real access control metadata from database logging. In a similar vein, context-appropriate significance could be investigated with, for example, adaptive policy enforcement depending on the agent's query history, anomaly patterns, or on feedback provided by human reviewers, all of which are outside the current scope but would be interesting extensions of this research.

1.6 Significance of the study

The increasing reliance on Large Language Models (LLMs) in enterprise applications has created a paradigm shift in how systems interact with databases. Autonomous, Agentic AI systems use LLMs for automated code generation including SQL queries, without human review. While this functionality creates new opportunities for automation and user access, it also introduces new security concerns, most of which have not been fully addressed.

We observe that LLMs have no inherent understanding of data sensitivity, role-based access constraints, or schema rules of enterprise databases. LLM-generated SQL queries may inadvertently leak sensitive information, exploit unintended access paths, or escalate privileges. This research is significant because, it introduces a technically sound and viable mechanism to monitor, assess, and block LLM generated harmful queries in a pro-active and real-time manner, thus safeguarding the important data infrastructure from inadvertent or malicious attacks.

A key contribution of this research is its hybrid architecture that combines SQL threat classification based on machine learning, with a contextual policy enforcement layer. Through post-query-generation enforcement of role-based access and schema-aware controls, the system adds safeguards beyond traditional static rules or firewalls. Using Explainable AI, also brings new level of transparency and accountability, enabling system administrators and auditors to understand why a query was blocked or rejected, which enhances the trust not only in the

choices made by AI but also support the compliance in the sectors like finance, healthcare, and governmental services that require accountability.

This research has important implications. It opens an avenue for academic researchers and security practitioners to explore the nexus between natural language generation and database security. It contributes to the growing body of knowledge on securing AI agent's interactions with critical back-end systems, which remains an under explored area despite its growing importance. For software developers in enterprise environments or DevSecOps requirements, the framework contemplated in this research could serve as a reference model for developing safer interfaces between generative AI and structured data stores. This research provides an actionable approach for organizations interested in leveraging AI agents in a context before or while complying with data governance, using a proactive, explainable, and adaptable protection layer.

From the perspective of society, the importance of this research is in its ability to potentially diminish AI-enabled data breaches, unauthorized disclosures and exploitation of systems. As enterprises and public service organizations implement generative AI in customer support, formalized workflows, data analytics approaches, and administration systems, there is a clear necessity for mechanisms that protect users' information, while promoting the responsible innovation of generative AI. This research expands upon that necessity just in time, in pursuit of responsible implementation of AI, while addressing the user's information security.

In summary, this research is significant given its timing and detailed investigation of critical blind spot with respect to current AI-augmented systems. It provides a plausible, explainable and enforceable solution for validating LLM-generated SQL queries with a deliverable that has conceptual and practical employability in a scenario to allow autonomous AI agents to operate safely within structured data repositories. The implications of these works are widespread in academia, industry, and society - all standing to benefit from a safer and more accountable AI-data interface.

1.7 Structure of the study

This dissertation is organised into six chapters, each of which contributes systematically to the development, implementation, evaluation, and validation of the proposed SecureDBAgent framework.

Chapter 1: Introduction

Chapter 1 establishes the foundational context for the research. It introduces the background of the study by situating the work within the broader evolution of database security and the emerging risks introduced by LLM driven automation. The chapter clearly articulates the problem statement, highlighting the limitations of existing database security mechanisms when confronted with AI-generated SQL queries. The research aim, objectives and questions are then defined to provide a focused direction for the study. Finally, the scope and significance of the research are discussed, outlining the academic and practical relevance of the proposed framework, followed by an overview of the dissertation structure.

Chapter 2: Literature Survey

Chapter 2 presents a comprehensive review of existing literature related to database security, SQL injection detection, role-based access control, AI-driven security systems and explainable artificial intelligence. The chapter critically examines traditional rule-based approaches, ML based detection methods and recent research on securing LLM generated code. Emphasis is placed on identifying gaps in current solutions, particularly the lack of context-aware enforcement, limited handling of behavioural anomalies and insufficient explainability in automated decision-making systems. This chapter concludes by justifying the need for the proposed research and positioning the SecureDBAgent as a novel contribution to the field.

Chapter 3: Research Methodology

Chapter 3 describes the overall research methodology adopted in this study. It outlines the design rationale of the proposed SecureDBAgent, detailing the architectural components including the context policy engine, statistical feature extraction, semantic encoding using convolutional neural networks, feature fusion strategy, threat classification and risk-tier decision logic. The chapter also explains the data sources used, preprocessing steps, training strategy, and threshold calibration methods. Additionally, the evaluation methodology, including performance metrics and validation datasets, is presented to ensure reproducibility and methodological rigour.

Chapter 4: Analysis

Chapter 4 focuses on the analytical aspects of the research. It examines the behaviour of individual system components through ablation studies and component-level analysis. The chapter analyses the contribution of static features, semantic embeddings, and fusion strategies to overall detection performance. Special attention is given to analysing the role of the context

policy engine in intercepting policy violations that are not captured by content-based classifiers. This chapter provides quantitative and qualitative insights into model behaviour, decision boundaries and error characteristics.

Chapter 5: Results and Discussion

Chapter 5 presents the experimental results obtained from extensive evaluation of the proposed framework. Performance metrics such as ROC-AUC, F1-score, false positive rate, and operational latency are reported and discussed in detail. The results are compared across multiple configurations, including standalone classifiers and the full context-aware pipeline. The chapter further discusses the results of testing on unseen benign workloads and evaluates the effectiveness of the explainability module in supporting transparent and auditable decisions. The implications of the findings are interpreted in relation to the research objectives and existing literature.

Chapter 6: Conclusions and Recommendations

Chapter 6 concludes the dissertation by summarising the key findings and contributions of the research. It reflects on how the proposed SecureDBAgent addresses the identified research gaps and advances the state of the art in secure AI-driven database access. The chapter also discusses limitations of the current study and provides recommendations for future work, including potential enhancements to context modelling, scalability and integration with enterprise governance frameworks. The dissertation concludes by highlighting the broader academic and industrial relevance of the research outcomes.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The increasing reliance on data-driven decision-making has positioned relational database systems as a foundational component of modern enterprise infrastructure. Over the past three decades, databases have evolved from isolated transactional systems into highly interconnected platforms accessed through web applications, microservices, cloud-native architectures, and more recently, intelligent agents powered by large language models (LLMs). While this evolution has enabled unprecedented flexibility and automation, it has simultaneously expanded the attack surface of database systems, introducing new categories of security vulnerabilities that are not adequately addressed by traditional defensive mechanisms.

This chapter presents a comprehensive review of the literature relevant to database security, SQL injection (SQLi) attacks, and machine learning based detection techniques, with particular emphasis on the emerging risks introduced by LLM-generated SQL queries. The review synthesises both classical and contemporary research to establish the theoretical and empirical foundations upon which this research is built. Unlike conventional literature surveys that focus solely on algorithmic performance, this chapter critically examines how existing approaches conceptualise context, intent and behaviour in database access, and where they fall short in the presence of autonomous or semi-autonomous AI systems.

The scope of this literature review spans four major thematic areas. First, it traces the historical evolution of database systems and the corresponding development of attack vectors, highlighting how architectural shifts have reshaped security assumptions. Second, it examines classical SQL injection techniques and the defensive strategies traditionally employed to mitigate them, including static analysis, input sanitisation, and rule-based access control. Third, it reviews machine learning and deep learning approaches proposed for SQL injection detection, analysing their strengths, limitations and underlying assumptions. Finally, it situates these findings within the context of modern AI-assisted database access, identifying clear gaps in the literature that motivate the need for a context-aware, multi-layered security framework.

By consolidating insights from foundational security research and recent publications, this chapter establishes a rigorous baseline for understanding why existing solutions are insufficient in contemporary enterprise environments. The review also provides the theoretical grounding necessary to justify the research questions, methodology and architectural choices presented in subsequent chapters.

2.2 Evolution of Database systems and expanding attack surfaces

Relational database management systems (RDBMS) emerged in the 1970s as structured repositories designed primarily for internal organisational use, operating within tightly controlled environments. Early systems such as IBM System R and Oracle's initial releases were accessed by trusted users through predefined applications, and security concerns were largely confined to authentication and basic authorisation mechanisms (Codd, 1970; Date, 2003). In these early architectures, the assumption of a trusted execution environment significantly limited exposure to external threats.

The proliferation of client-server architectures in the 1990s marked the first major shift in database accessibility. Databases were no longer isolated systems but became backend services supporting multiple client applications. This transition introduced new risks, particularly as SQL queries began to be constructed dynamically based on user input. Research during this period identified the earliest forms of SQL injection attacks, where improperly sanitised inputs could alter query logic and bypass intended access controls (Halfond, Viegas, & Orso, 2006).

The rise of web applications in the early 2000s further amplified these risks. Web-facing databases became common targets for attackers due to their exposure to untrusted inputs over HTTP. Empirical studies consistently ranked SQL injection among the most severe and prevalent web application vulnerabilities, a trend reflected in the OWASP Top 10 reports over multiple years (OWASP, 2017; OWASP, 2021). During this phase, attackers exploited weaknesses in application layer input handling to execute unauthorised queries, extract sensitive data, and escalate privileges.

Subsequent architectural transformations, including service-oriented architectures (SOA), microservices, and cloud native deployments, introduced additional complexity. Databases began serving multiple services with differing privilege requirements, often across distributed environments. While role-based access control (RBAC) and policy-driven security models were adopted to manage this complexity, several studies highlighted their limitations in enforcing fine-grained, context-sensitive access decisions (Sandhu et al., 1996; Hu et al., 2015). These models typically relied on static role definitions and predefined permissions, assuming predictable query patterns and well-defined user roles.

More recently, the integration of natural language interfaces and AI-driven automation has fundamentally altered the database threat landscape. Systems that translate natural language requests into SQL queries commonly referred to as NL2SQL systems, have been widely

adopted to improve usability for non-technical users (Zhong, Xiong, & Socher, 2017; Wang et al., 2020). While these systems reduce the need for direct SQL authoring, they introduce a critical abstraction layer in which query generation is delegated to machine learning models rather than deterministic application logic.

This shift has profound security implications. Unlike traditional applications where developers control query templates, LLM-powered systems generate SQL dynamically based on probabilistic inference. Recent studies have demonstrated that such models can hallucinate tables, misuse joins, violate access policies or generate syntactically valid yet semantically unsafe queries (Chen et al., 2023; Pearce et al., 2022). These behaviours challenge existing security paradigms, which are not designed to evaluate the intent or contextual appropriateness of machine generated queries.

Despite these developments, much of the existing database security literature continues to assume a human-in-the-loop model, where developers or users explicitly author queries. As a result, there is a growing disconnect between the assumptions underpinning traditional security mechanisms and the realities of AI-driven database access. This disconnect forms a critical backdrop for the present study and underscores the necessity of rethinking database security in the age of autonomous agents.

2.3 Classical SQL injection attacks and early defensive mechanisms

SQL injection has long been recognised as one of the most critical threats to database backed applications. At its core, SQL injection occurs when untrusted input is concatenated into a query without adequate validation, allowing attackers to manipulate query structure and execution semantics (Halfond et al., 2006). Early taxonomies of SQL injection attacks categorised them into several classes, including tautology-based attacks, union-based injections, piggy-backed queries, and inference-based techniques such as blind and time-based SQL injection.

Tautology-based attacks exploit logical conditions that always evaluate to true, enabling attackers to bypass authentication checks. Union-based attacks leverage the SQL UNION operator to append malicious queries and extract data from unintended tables. Piggy-backed queries introduce additional statements separated by delimiters, while blind SQL injection infers sensitive information through boolean responses or response timing (Su & Wassermann, 2006). These attack patterns were extensively studied in the early 2000s and formed the basis for many defensive strategies.

Initial counter-measures focused on input sanitisation and prepared statements. Parameterised queries were widely promoted as an effective means of preventing injection by separating query structure from user input (Clarke, 2009). Static analysis tools were also developed to identify vulnerable code paths during development, analysing string concatenation patterns and tainted data flows (Livshits & Lam, 2005). While effective in many scenarios, these approaches relied heavily on correct developer implementation and did not account for runtime context or evolving attack strategies.

Web application firewalls (WAFs) emerged as a complementary defence, using signature-based detection to block known attack patterns at the network or application perimeter. Studies evaluating WAF effectiveness reported mixed results, noting high false positive rates and susceptibility to evasion through obfuscation and encoding techniques (ModSecurity, 2018; Behl & Behl, 2017). Moreover, signature-based systems struggled to generalise beyond known attack patterns, limiting their effectiveness against novel or obfuscated SQL injection attempts.

Role-based access control mechanisms were also employed to mitigate the impact of successful injections by restricting database privileges. While RBAC reduces the potential damage of compromised queries, it does not prevent the execution of malicious logic within permitted operations. Research has shown that even low-privileged roles can be exploited to perform inference attacks or aggregate sensitive information over time (Bertino & Sandhu, 2005).

Collectively, classical defences addressed specific manifestations of SQL injection but treated security as a static property of queries or roles. They did not incorporate behavioural context, historical usage patterns, or semantic understanding of queries. As attacks grew more sophisticated and systems more dynamic, these limitations became increasingly apparent, prompting researchers to explore adaptive and learning based detection method - a shift that is examined in subsequent sections of this chapter.

2.4 Rule-based, Policy-driven and Static defensive mechanisms

As SQL injection attacks became increasingly prevalent, research attention shifted toward preventive mechanisms that could enforce security constraints independently of application code quality. One of the earliest and most widely adopted approaches in this category was role-based access control (RBAC), which sought to limit the impact of malicious queries by constraining database privileges according to predefined user roles (Sandhu et al., 1996). In theory, RBAC ensures that even if an attacker succeeds in injecting SQL, the resulting query cannot exceed the permissions granted to the compromised role.

However, subsequent empirical studies revealed that RBAC alone is insufficient as a primary defence mechanism. Bertino and Sandhu (2005) demonstrated that RBAC models typically operate at the level of coarse-grained privileges such as SELECT, INSERT, or UPDATE without reasoning about query intent, data sensitivity or contextual appropriateness. As a result, attackers can exploit legitimate privileges to perform inference attacks, aggregate sensitive information, or execute logic that violates organisational policies without explicitly breaching access control rules.

To address these limitations, researchers proposed policy-driven extensions that incorporated finer-grained constraints. Attribute-based access control (ABAC) emerged as a more expressive alternative, allowing access decisions to be based on attributes of users, resources, actions, and environments (Hu et al., 2015). While ABAC offers greater flexibility than RBAC, its effectiveness depends heavily on the completeness and correctness of policy definitions. In dynamic enterprise environments, maintaining exhaustive attribute sets and policies is both complex and error-prone, often resulting in overly permissive configurations.

Parallel to access control research, database firewalls and query whitelisting techniques were developed to monitor SQL statements at runtime. These systems compare incoming queries against predefined templates or statistical profiles of legitimate behaviour (Low et al., 2010). Although effective in controlled environments with stable workloads, whitelist-based approaches struggle to accommodate evolving schemas, ad hoc analytical queries, and legitimate variations in query structure. Studies evaluating commercial database firewalls have reported high false positive rates when deployed in real-world systems with heterogeneous access patterns (Behl & Behl, 2017).

Static analysis techniques represent another important class of defences. These methods analyse application source code to detect vulnerable query construction patterns, such as unsafe string concatenation or improper input handling (Livshits & Lam, 2005). While static analysis can identify vulnerabilities early in the development lifecycle, it offers no protection against runtime threats originating outside the analysed codebase, including dynamically generated queries produced by AI systems or third-party services.

Collectively, rule-based and static mechanisms conceptualise database security as a deterministic problem: if all possible unsafe patterns can be enumerated and constrained, attacks can be prevented. However, this assumption becomes increasingly untenable in modern environments characterised by dynamic workloads, evolving schemas and probabilistic query

generation. These limitations motivated the exploration of data-driven and learning-based detection approaches, which are discussed in the following section.

2.5 Machine learning approaches for SQL Injection detection

Machine learning techniques were introduced into SQL injection detection research to overcome the rigidity of rule-based systems and improve generalisation to unseen attack patterns. Early work in this area focused on classical supervised learning algorithms, including decision trees, support vector machines (SVMs), and Naive Bayes classifiers. These models typically relied on manually engineered features extracted from SQL queries, such as keyword frequencies, operator counts, query length, and character distributions (Valeur et al., 2005; Nguyen et al., 2011).

Several studies reported promising results using statistical features derived from lexical and syntactic properties of queries. For example, Valeur et al. (2005) demonstrated that anomaly detection based on query structure could identify deviations from normal behaviour without relying on explicit attack signatures. Similarly, Nguyen et al. (2011) employed n-gram models and SVMs to distinguish malicious queries from benign ones, achieving high detection rates on benchmark datasets.

Despite these successes, classical machine learning approaches exhibit notable limitations. Feature engineering is highly domain-specific and often fails to capture deeper semantic relationships within queries. Moreover, many studies evaluate models on synthetic or narrowly curated datasets, raising concerns about their robustness in real-world deployments. Comparative analyses have shown that models trained on limited attack distributions tend to overfit known patterns and degrade significantly when exposed to obfuscated or zero-day SQL injection techniques (Zhang et al., 2019).

Another critical limitation lies in the treatment of context. Most machine learning based detectors operate on individual queries in isolation, ignoring information about user roles, historical behaviour, session dynamics or schema semantics. As noted by Cova et al. (2010), attacks that unfold gradually over multiple queries or exploit legitimate privileges are particularly difficult to detect using per-query classifiers. This observation underscores a fundamental gap in the literature. Machine learning improves pattern recognition, but it does not inherently address contextual reasoning unless explicitly designed to do so.

Furthermore, the interpretability of classical machine learning models varies widely. While decision trees offer some degree of transparency, more powerful models such as SVMs and

ensemble methods often function as black boxes. This lack of explainability poses challenges for security auditing, regulatory compliance, and operational trust concerns that have become increasingly prominent in enterprise settings.

These shortcomings paved the way for deep learning-based approaches, which aim to learn richer representations of SQL queries and capture higher level semantic patterns.

2.6 Deep Learning and Semantic Modelling of SQL queries

Deep learning techniques have been increasingly adopted for modelling structured and semi-structured data, including source code and database queries. Inspired by advances in natural language processing, researchers began treating SQL queries as sequences of tokens and applying neural architectures to learn latent representations that encode semantic information. Early work explored recurrent neural networks (RNNs) and long short-term memory (LSTM) models to capture sequential dependencies in SQL statements (Zhang et al., 2019).

Convolutional neural networks (CNNs) have also been employed for SQL injection detection, leveraging their ability to identify local patterns and n-gram–like features within token sequences. Several studies report that CNN-based models outperform traditional machine learning approaches on benchmark datasets, particularly in detecting obfuscated attacks and variants not seen during training (Kim et al., 2020). CNNs offer computational efficiency and parallelism advantages over recurrent models, making them attractive for real-time security applications.

More recently, transformer-based architectures and pretrained language models have been applied to SQL analysis tasks. Models such as BERT and GPT variants have demonstrated strong performance in understanding query semantics and generating SQL from natural language (Devlin et al., 2019; Wang et al., 2020). However, research examining their use in security contexts has raised significant concerns. Pearce et al. (2022) and Chen et al. (2023) highlight that large language models can produce syntactically valid yet semantically unsafe code, often without explicit indicators of malicious intent.

A recurring limitation in deep learning-based SQLi detection research is the reliance on content-only analysis. While semantic embeddings capture rich information about query structure and intent, they do not account for whether a query is appropriate given the operational context in which it is executed. For example, a SELECT query accessing sensitive tables may be benign for an administrative role but anomalous for a reporting service. Deep models trained solely on query text lack the contextual signals required to make such distinctions.

Another important concern relates to explainability. Deep neural networks, particularly those involving multiple layers and nonlinear transformations, are often criticised for their opacity. In security-sensitive domains, the inability to justify why a query was flagged as malicious can hinder incident response and undermine trust in automated systems (Doshi-Velez & Kim, 2017). While post-hoc explanation techniques such as SHAP and gradient-based saliency have been proposed, their integration into database security workflows remains limited in existing literature.

In summary, deep learning approaches represent a significant advancement in modelling SQL query semantics, but they are not a panacea. The literature reveals a consistent pattern. That is models that focus exclusively on query content - whether statistical or semantic, fail to capture the broader contextual and behavioural dimensions of database access. This observation directly informs the research gap identified in the next section of this chapter, where the need for a context-aware, multi-layered security framework is formally articulated.

2.7 Identified research gaps and Theoretical framework

The preceding review of literature reveals substantial progress in database security mechanisms, ranging from deterministic policy enforcement to advanced deep learning-based SQL injection detection. However, it also exposes a set of persistent and structurally unresolved gaps that collectively motivate the need for a fundamentally different approach to securing modern database systems, particularly in environments where SQL queries are generated autonomously by AI-driven agents.

2.7.1 Identified Research Gaps

Gap 1: Absence of Context-aware security reasoning

A dominant limitation across existing studies is the treatment of SQL queries as isolated artefacts, evaluated solely on their syntactic or semantic properties. Traditional rule-based systems enforce access controls based on static role definitions, while machine learning and deep learning models operate almost exclusively on query content (Valeur et al., 2005; Zhang et al., 2019; Kim et al., 2020).

What is notably missing is contextual reasoning, where a query is evaluated relative to the role executing it, the sensitivity of the accessed schema, historical user behaviour,

session-level anomalies and provenance of query generation.

As highlighted by Bertino and Sandhu (2005), security violations often occur not because a query is syntactically malicious, but because it is contextually inappropriate. For example, a SELECT * query on a financial table may be valid for an auditor but anomalous for an application service account. Existing literature does not adequately model this distinction, resulting in high false positive rates or missed attacks when deployed in heterogeneous enterprise environments.

Gap 2: Over-Reliance on Content-only Machine Learning Models

Machine learning-based SQL injection detectors have demonstrated impressive detection accuracy on benchmark datasets. However, their reliance on content-only features introduces fundamental blind spots. Statistical feature models struggle with obfuscated attacks, while deep semantic models may misclassify benign but rare queries as malicious due to distributional shift (Nguyen et al., 2011; Pearce et al., 2022).

More critically, these models lack awareness of operational intent. They cannot distinguish between malicious intent and legitimate administrative activity, exploratory analytics and inference attacks, anomalous behaviour and schema evolution.

This limitation becomes especially problematic in the context of large language models (LLMs), which can generate syntactically correct SQL that violates enterprise policies without exhibiting classical injection patterns (Chen et al., 2023). The literature largely treats this as a data problem—solvable through more training rather than a reasoning problem requiring policy and behavioural context.

Gap 3: Lack of Multi-Layered decision architectures

Most existing approaches adopt a monolithic detection strategy with a single classifier or rule engine determines whether a query is malicious. This design does not reflect how security decisions are made in practice, where multiple signals are considered hierarchically and conservatively.

Cova et al. (2010) argue that effective security systems must separate policy violations from content-based threats, yet few SQL security frameworks implement this separation explicitly. As a result, classifiers are often burdened with decisions they are ill-equipped to make, such as enforcing compliance rules or reasoning about organisational norms.

The absence of layered architectures also prevents graceful degradation. When a model is uncertain, existing systems typically default to allow or block decisions without intermediate risk states, increasing either operational friction or security exposure.

Gap 4: Insufficient Explainability and Auditability

Explainability remains a peripheral concern in much of the SQL security literature. While recent work has introduced post-hoc explanation techniques such as SHAP and gradient-based saliency for machine learning models, these explanations are rarely integrated into end-to-end security workflows (Doshi-Velez & Kim, 2017).

Moreover, explanations are often limited to model behaviour and do not account for policy-level decisions or contextual constraints. From an enterprise perspective, security decisions must be auditable, justifiable and aligned with governance requirements. The literature lacks frameworks that unify model explanations with policy rationales in a coherent and operator-friendly manner.

Gap 5: Limited evaluation on realistic and unseen workloads

A final gap concerns evaluation methodology. Many studies report near-perfect accuracy on curated datasets but fail to assess robustness under distribution shift, evolving schemas, or unseen benign workloads. The use of datasets such as Spider, which contains complex, real-world benign SQL queries remains rare in SQL injection research.

Without evaluating false positive behaviour on such workloads, claims of effectiveness are difficult to generalise to production environments. This gap is particularly critical for AI-generated SQL, where benign queries may deviate significantly from historical norms.

2.7.2 Theoretical framework underpinning this research

In response to the identified gaps, this research is grounded in a multi-layered security reasoning framework that synthesises concepts from access control theory, behavioural anomaly detection, representation learning and explainable AI.

At the foundation lies policy-based security theory, which asserts that system behaviour must conform to explicitly defined organisational constraints (Sandhu et al., 1996). This is extended with contextual integrity theory, which emphasises that the appropriateness of an action depends on situational factors rather than static rules alone.

Building upon this, the framework incorporates behavioural anomaly detection, modelling deviations in session activity, access patterns, and operational frequency as indicators of potential misuse (Cova et al., 2010). These signals complement, rather than replace, content-based threat detection.

Semantic understanding is achieved through representation learning, where convolutional neural networks encode SQL queries into latent vectors capturing structural and semantic relationships (Kim et al., 2020). Unlike standalone deep learning models, these representations are interpreted within a broader decision context.

Finally, the framework integrates explainable AI principles, ensuring that each decision - whether policy-based or model-driven can be decomposed into human-interpretable rationales. This aligns with emerging best practices in trustworthy AI and security governance.

2.7.3 Positioning of the present research

By unifying deterministic policy enforcement, behavioural context modelling, semantic threat classification, and explainability into a single coherent architecture, this study addresses gaps that have remained largely unexamined in prior research. Rather than treating SQL injection detection as a purely classification problem, it reframes database security as a context-aware decision-making process, capable of operating safely alongside autonomous AI systems.

This theoretical positioning directly informs the research methodology presented in Chapter 3 and provides a rigorous foundation for evaluating the proposed Secure DB agent in realistic enterprise scenarios.

CHAPTER 3: RESEARCH METHODOLODY

3.1 Chapter overview

This chapter presents the research methodology adopted to design, implement and evaluate the proposed SecureDBAgent - a context-aware, multi-layered framework for detecting and controlling unsafe SQL queries generated by LLM powered systems. The methodological choices made in this study are guided by the nature of the research problem, the operational constraints of enterprise database environments and the need for both technical rigour and practical applicability.

The core objective of this methodology is not merely to achieve high predictive performance, but to construct a defensible, auditable and deployable security system capable of operating in real-world enterprise contexts. As such, the methodology integrates deterministic policy enforcement, statistical feature engineering, neural semantic modelling and explainable decision-making within a single cohesive framework. This hybrid design reflects the recognition that purely machine learning based approaches are insufficient when used in isolation for high-risk database security scenarios, particularly when dealing with dynamically generated SQL produced by LLMs.

The chapter follows a quantitative, experimental research approach, supported by supervised learning and controlled evaluation on multiple publicly available datasets. Emphasis is placed on reproducibility, objective measurement and systematic comparison across different architectural variants, including static-only models, semantic-only models, fused models, and the complete context-policy–augmented pipeline. This layered evaluation strategy enables a clear assessment of the contribution of each system component and supports evidence-based conclusions regarding the effectiveness of the proposed design.

From a structural perspective, this chapter begins by outlining the overall research design and methodological paradigm, followed by a detailed description of the end-to-end system workflow. Subsequent sections describe the data sources used, preprocessing steps and feature representations employed. Particular attention is given to the Context Policy Engine, which constitutes the primary methodological and conceptual novelty of this research. The chapter then details the construction of the static feature extractor, CNN-based semantic encoder, feature fusion mechanism, and threat classification model. Finally, the methodology for risk-tier decision-making and explainable AI integration is presented, along with implementation considerations and ethical safeguards.

Overall, this chapter establishes a clear and systematic foundation for the experimental analysis and evaluation presented in later chapters. By explicitly linking design decisions to research objectives and real-world constraints, the methodology ensures that the proposed SecureDBAgent is both technically sound and practically relevant, thereby supporting the broader aims of the study.

3.2 Data gathering

This research uses three publicly available datasets (Abu Syeed Sajid Ahmed and Mehjabeen Shachi, 2021; Henrique and Andrea Valenza, 2022; Spider: Yale Semantic Parsing and Text-to-SQL Challenge, 2024)to produce results that are consistent and reproducible.

- SQLiV5 dataset (Henrique and Andrea Valenza, 2022): This dataset is an enhanced version of SQLiV3 with 20,000-30,000 SQL queries, both benign and malicious. SQLiV5 categorizes these attack types: (1) tautology-based injection, (2) piggy-backed queries, (3) time-based blind SQLi, (4) union-based injection, and (5) hallucinated structures that LLMs could generate. All queries in SQLiV5 are labelled, have coherent structure, and can facilitate supervised learning.
- Kaggle SQL injection dataset (Abu Syeed Sajid Ahmed and Mehjabeen Shachi, 2021): Containing a total of 30,919 rows, the Kaggle SQL Injection dataset reports 19,529 benign queries, and 11,382 malicious queries, which can add to SQLiV5 with real world SQL injections examples including encoded, logic-based and semantic attacks. This dataset is available as a CSV file consisting of columns representing query, label, and payload_type.
- Spider dataset (Testing Only)(Spider: Yale Semantic Parsing and Text-to-SQL Challenge, 2024): The spider dataset consists 10,181 real world benign queries across multiple database schemas. This dataset will be used exclusively to measure the false positive rate (FPR) of the detection model applied within the research. The diverse forms of the benign queries enable the classifier's generalization of clean enterprise queries.

The datasets are downloaded from official source. No synthetic data or external datasets are used.

These datasets are divided into three subsets, 70% train, 15% validation (for tuning), and 15% testing. The training subsample will be withheld from the Spider dataset entirely and used only for final false positive evaluation.

3.3 Research Design and Methodological Approach

This study adopts a quantitative, experimental research design to investigate the effectiveness of a context-aware, multi-layered SecureDBAgent for mitigating unsafe SQL queries generated by LLM driven systems. The methodological approach is grounded in supervised machine learning, system level security design and empirical performance evaluation across diverse datasets. This combination is particularly suitable given the measurable nature of the research objectives and the requirement to objectively assess detection accuracy, false positive rates, explainability and operational feasibility.

3.3.1 Rationale for a Quantitative experimental approach

A quantitative methodology is appropriate for this study because the core research questions revolve around measurable system behaviour, such as:

- The probability of correctly identifying malicious SQL queries
- The reduction of false positives on benign, real-world workloads
- The impact of contextual constraints on detection performance
- The latency overhead introduced by security controls

These outcomes can be objectively quantified using established evaluation metrics such as ROC-AUC, F1-score, false positive rate (FPR) and latency measurements. Unlike qualitative approaches, which are better suited for exploratory or interpretive studies, a quantitative experimental design enables systematic comparison between competing model architectures and security strategies under controlled conditions.

Furthermore, the study involves repeated experimentation under varying configurations (e.g., static-only models, CNN-only models, fused models, and context-aware pipelines). Such comparative analysis necessitates a structured experimental framework where variables can be isolated and their effects empirically validated.

3.3.2 Supervised learning Paradigm

At the core of the threat detection component lies a supervised learning paradigm, wherein models are trained using labelled datasets containing benign and malicious SQL queries. Supervised learning is particularly well suited for this domain because historical attack patterns,

known injection techniques, and benign query distributions can be explicitly encoded through labelled examples.

However, unlike conventional SQL injection detection systems that rely solely on pattern matching or token-level classification, this research extends the supervised paradigm by embedding it within a broader policy-driven and context-aware framework. The learning model does not operate in isolation. Instead, its outputs are interpreted in conjunction with deterministic policy checks, behavioural detectors, and schema-aware constraints. This methodological choice reflects a shift from pure classification toward decision support within a governed security architecture.

3.3.3 Multi-stage hybrid system design

The research methodology follows a multi-stage hybrid system design, where different analytical techniques are applied at distinct stages of the query evaluation pipeline. This design is motivated by the recognition that SQL query safety cannot be reliably determined using a single analytical lens.

The overall methodological flow consists of the following conceptual stages:

- Context Policy Enforcement
- Statistical Feature Engineering
- Semantic Embedding via Neural Encoding
- Fusion-Based Threat Classification
- Risk-Tier Decision and Explainability

3.3.4 Experimental evaluation strategy

The experimental design incorporates multiple datasets, training-validation-test splits and ablation studies to ensure robust evaluation. Models are assessed not only on standard malicious query datasets but also on previously unseen workloads, such as the Spider dataset, which contains real-world, human-authored SQL queries. This choice enables a realistic assessment of generalisation and false positive behaviour, an aspect often overlooked in prior research.

Additionally, experiments are structured to isolate the contribution of individual components, such as static features, semantic embeddings and context policy checks. This systematic decomposition supports transparent analysis and strengthens the validity of conclusions drawn from the results.

3.3.5 Alignment with research objectives

The methodological approach outlined in this section directly supports the research aims and objectives defined in Chapter 1. By combining policy enforcement, machine learning and explainability within a single framework, the methodology enables:

- Rigorous evaluation of detection effectiveness
- Objective assessment of system robustness and generalisation
- Practical validation of deployment feasibility in enterprise environments

In summary, the chosen research design provides a balanced and defensible foundation for investigating the research problem. It ensures that the proposed SecureDBAgent is evaluated not only as a predictive model, but as a complete security system operating under realistic constraints.

3.4 Overall System Architecture and End-to-End flow

The SecureDBAgent proposed in this research is designed as a layered, modular architecture that integrates deterministic policy enforcement with data-driven threat intelligence. The architecture reflects a deliberate separation of concerns, where governance, behavioural reasoning, semantic understanding and probabilistic decision-making are orchestrated into a single, coherent pipeline. This design enables the system to operate reliably in enterprise environments while remaining adaptable to evolving threat landscapes, particularly those introduced by LLM-generated SQL queries.

3.4.1 Architectural Design Principles

The architecture is guided by four core design principles:

- Context First, Intelligence Second

  Unlike traditional SQL security systems that prioritise pattern-based detection, the proposed system enforces context-aware policy constraints before invoking machine learning models. This ensures that clearly unsafe queries are intercepted deterministically, reducing unnecessary reliance on probabilistic inference.
- Layered Defence Strategy

  The architecture adopts a defence-in-depth approach, where multiple analytical layers independently assess query safety. Each layer contributes complementary evidence rather than acting as a single point of failure.
- Explainability by Design

Every architectural component is designed to emit interpretable signal policy violations, statistical anomalies, semantic cues or feature attributions, ensuring auditability and regulatory compliance.

- Enterprise Deploy ability

  The system is designed to integrate seamlessly with existing database access workflows, introducing minimal latency and requiring no modifications to underlying database engines.

3.4.2 High-Level Architectural Components

At a high level, the Secure Database Agent consists of the following interconnected modules:

- Query Ingestion layer
- Context policy engine
- Statistical Feature Engineering layer
- Semantic Embedding layer (CNN Encoder)
- Fusion Based Threat Classifier
- Risk Tier decision layer
- Explainable AI (XAI) Module
- Decision Output Interface

3.4.3 System architecture

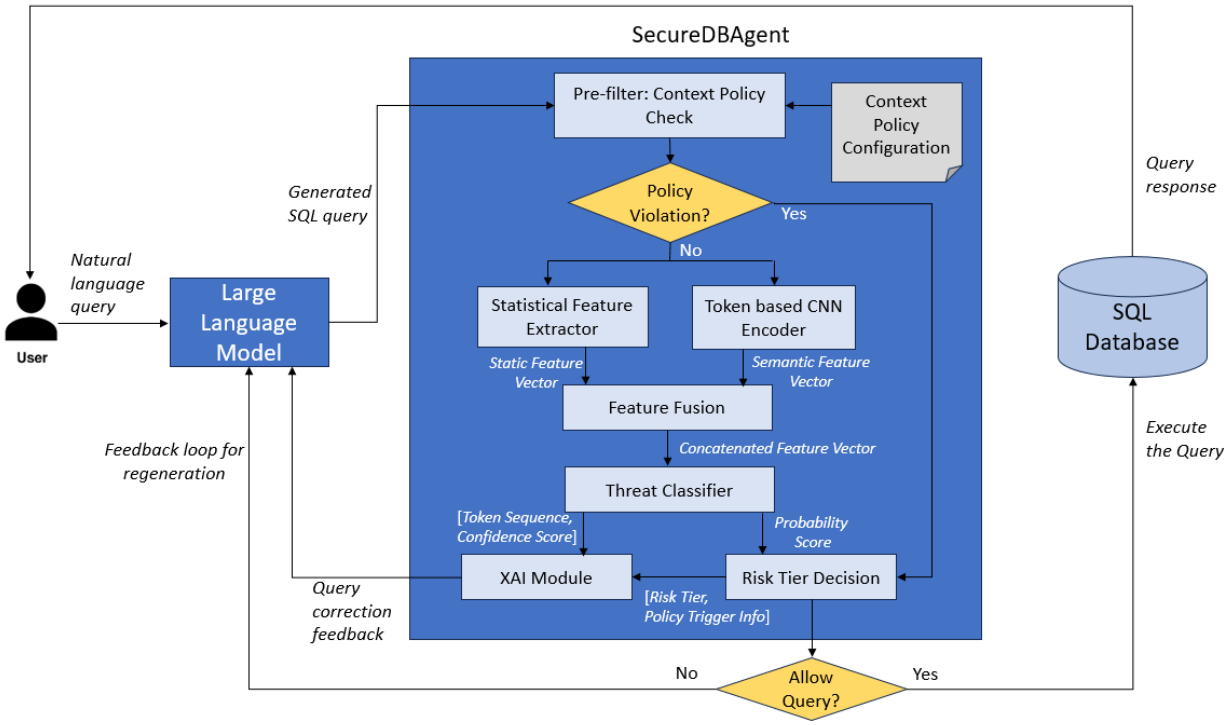


Figure 3.1: SecureDBAgent: System Architecture

3.4.4 End-to-End Query Processing Workflow

The end-to-end workflow begins when a SQL query is generated either by an LLM-powered application, an automated agent or a traditional software component and is intercepted by the SecureDBAgent before execution.

Step 1: Query Ingestion and Normalisation

The incoming query, along with its associated metadata (user role, user identifier, timestamp and session context), is first normalised. This includes canonicalization of SQL syntax, token standardisation, and extraction of preliminary structural elements such as operations, tables and clauses.

Step 2: Context Policy Evaluation

The normalised query is evaluated against enterprise defined context policies. These include role-based access control (RBAC) rules, schema sensitivity constraints, allowed operation mappings, and high-level behavioural checks. Queries that violate explicit policy constraints are immediately classified into high-risk categories and may be blocked or flagged without invoking machine learning models.

Step 3: Statistical Feature Extraction

Queries that pass the initial policy layer are transformed into structured numerical representations. This stage captures syntactic complexity, statistical irregularities, encoding artefacts, entropy measures, and behavioural indicators. These features provide a compact yet expressive summary of query characteristics that are difficult to capture using semantic embeddings alone.

Step 4: Semantic Embedding via CNN Encoder

In parallel, the SQL query is tokenised and passed through a CNN based encoder. This component learns dense semantic embeddings that capture contextual relationships between tokens, enabling the detection of obfuscated or novel attack patterns that may not be evident through handcrafted features.

Step 5: Feature Fusion and Threat Classification

The statistical feature vector and the semantic embedding are concatenated into a fused representation, which is then processed by a multi-layer perceptron (MLP) classifier. The classifier outputs a continuous threat probability representing the likelihood that the query is malicious.

Step 6: Risk Tier Decision Logic

The predicted threat probability is mapped to discrete risk tiers - Allow, Flag or Block using thresholds derived from ROC and precision-recall curve optimisation. This step ensures consistent decision making aligned with operational risk tolerance.

Step 7: Explainability and Evidence Generation

Simultaneously, the XAI module generates interpretable explanations corresponding to the decision. These include rule-level justifications from the context policy engine, SHAP-based attributions for statistical features and token-level saliency scores for semantic embeddings.

Step 8: Final Action and Output Formatting

The final decision, along with structured explanations and diagnostics, is returned to the calling application or enforcement layer. This output can be logged, audited or used to trigger downstream security workflows.

3.4.5 Architectural Novelty and Contribution

The novelty of this architecture lies not in any single component, but in the system-level integration of deterministic governance and probabilistic intelligence. Existing approaches in the literature typically focus on either rule-based SQL firewalls or standalone machine learning classifiers. In contrast, the proposed architecture:

- Embeds machine learning within an explicit policy framework rather than treating it as an autonomous decision-maker.
- Treats explainability as a first-class architectural concern rather than an afterthought.
- Explicitly accounts for the unique risks introduced by LLM-generated SQL, such as hallucinated tables, role misalignment and semantic ambiguity.
- By structuring the system as a layered pipeline with clear responsibility boundaries, the architecture supports extensibility, reproducibility, and real-world deployment - key requirements for enterprise-grade security systems.

3.5 Context Policy engine methodology

3.5.1 Motivation for a Context-aware policy layer

Traditional database security mechanisms such as role-based access control (RBAC), static permission matrices, and signature-based SQL injection detection were designed for environments in which queries are authored either by humans or by deterministic application logic. In such settings, the structure, intent and scope of database queries are relatively

predictable and bounded by predefined application workflows. However, the emergence of Large Language Model (LLM) driven agentic systems fundamentally alters this assumption.

In agentic AI pipelines, SQL queries are generated dynamically from natural language instructions, often without an explicit understanding of enterprise specific access policies, schema sensitivities or organisational norms. As a result, queries may be syntactically valid and semantically plausible while still being operationally unsafe or contextually inappropriate. Examples include excessive data access by low-privileged roles, novel cross-table joins involving sensitive entities or bursty query patterns that deviate significantly from historical user behaviour. These failure modes are not adequately addressed by classical SQL injection detectors or by machine-learning models that operate solely on query content.

The Context Policy Engine is therefore introduced as a deterministic yet adaptive pre-execution mediation layer that evaluates LLM-generated SQL queries against organisational context before any semantic or statistical threat classification is applied. Its role is not to replace machine learning based detection, but to provide a first line of defence that enforces hard governance constraints and captures behavioural anomalies that are orthogonal to token-level maliciousness.

3.5.2 Design principles and Threat model assumptions

Here is the high-level design of Context-policy engine

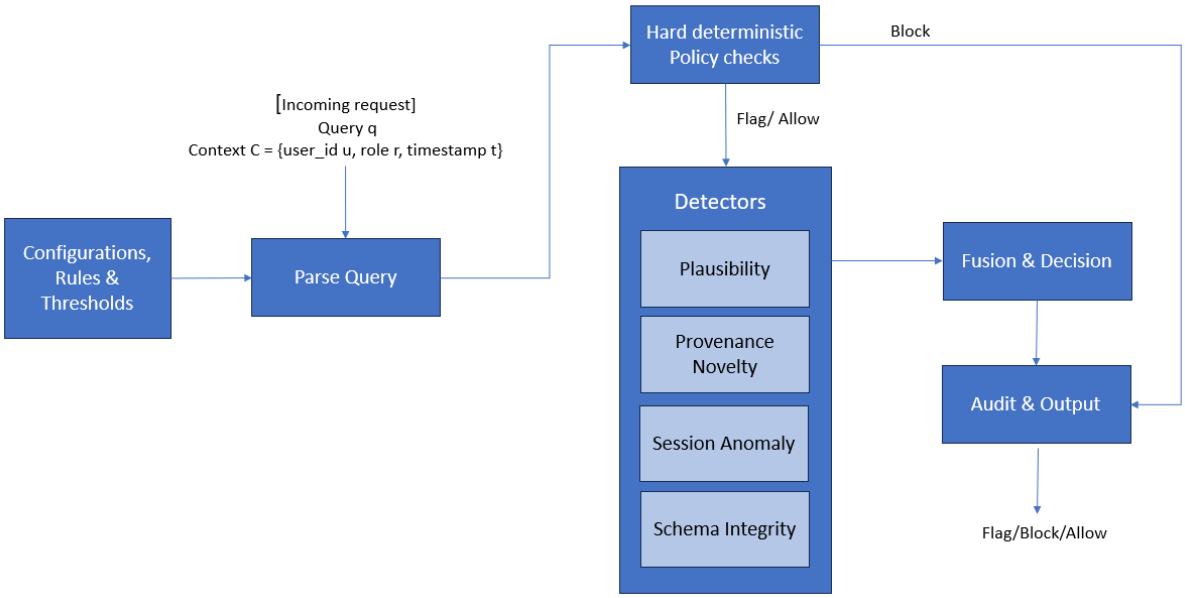


Figure 3.2: Context-Policy Engine HLD

The Context Policy Engine is grounded in four design principles.

Determinism where governance is required

Enterprise security policies are typically defined as explicit rules (role permissions, sensitive data constraints). These are best represented as deterministic checks rather than learned behaviour, because deterministic checks are auditable and stable under distribution shift.

Probabilistic reasoning where uncertainty is inherent

Some risks are behavioural rather than syntactic e.g., sudden access to unfamiliar tables or unusual join patterns. These are better modelled using detector scores derived from historical distributions, rather than as binary rules.

Explainability-by-construction

Each enforcement outcome must be explainable without relying on post-hoc explanations. Therefore the engine returns structured "reasons" and diagnostic details, including which policy rule triggered, which detector score exceeded a threshold, and which schema elements were involved.

Schema evolution tolerance

Enterprise schemas change. LLMs also introduce failure modes such as referencing outdated or non-existent columns. The engine must be robust to both realities: it should detect hallucination without generating false alarms during legitimate schema changes. This principle directly motivates the introduction of grace periods and smoothing strategies in plausibility scoring (discussed later).

The threat model assumed here is not limited to classical SQL injection. Instead, it includes role abuse, unauthorized sensitive access, schema-level hallucination, multi-statement execution patterns and behavioural anomalies consistent with compromised credentials or misaligned prompts.

3.5.3 Context Policy Configuration Model and Administrative Control Plane

A key methodological choice in the framework is that context policy must be admin-defined and externally configurable. This is necessary for two reasons:

1) Policies are organization-specific: "Allowed" behaviour depends on role design, table sensitivity and business process.
2) Policies must be auditable: External configuration artefacts (rather than embedded logic) enable reviews, approvals and change control.

Below are the administrative control policies that need to be configured:

Role-level policy files (one policy file per role)

For each enterprise role, the administrator maintains a dedicated JSON policy file (e.g., role_appuser.json, role_data_analyst.json). This one-to-one mapping is intentional. It avoids a single monolithic policy document becoming unmanageable and makes it explicit what privileges apply to a given role.

Each role policy typically encodes:

- Allowed operations: permitted SQL intents such as SELECT-only for analytics roles, or INSERT/UPDATE for operational roles.
- Denied operations: explicit "never allowed" operations such as DDL or administrative functions for non-admin roles.
- Sensitivity constraints: whether a role may access highly sensitive data categories.
- Tag permissions: the set of semantic domains (tags) that a role may read/write.
- Multi-statement policy: whether multiple statements, chained queries or embedded control structures must be blocked or flagged.
- Detector thresholds: configurable limits for plausibility, novelty and session anomaly that determine whether the output is allow/flag/block.
- Action mapping: what constitutes a hard block vs soft flag for a given violation type.

This approach converts security governance into a policy-as-data model.

Below table describes per-role policy JSON files, where each enterprise role is governed by an independent, versioned policy artefact. This design enables fine-grained RBAC enforcement, auditability, and policy evolution without retraining ML models.

Table 3.1: Role Policy Configuration Model

Field	**Type**	**Example Value**	**Purpose in Context Policy Engine**
role_id	String	admin	Unique identifier for the enterprise role. Used as the primary key for policy lookup during query evaluation.
description	String	Administrator	Human-readable role description for audit and governance documentation.
allowed_operations	List[String]	SELECT, INSERT, UPDATE, DELETE, DDL	Defines SQL operations permitted for this role. Enforced as hard deterministic constraints before ML invocation.

allowed_tags	List[String]	["*"]	Specifies which semantic table tags the role can access. Wildcard enables full access for privileged roles.
denied_tags	List[String]	[]	Explicitly prohibited data categories (e.g., PII, FINANCIAL). Enables deny-by-default semantics.
restricted_columns_by_tag	Dict	{}	Optional column-level restrictions tied to data sensitivity tags, supporting fine-grained least-privilege enforcement.
sensitivity_policy	Dict	{HIGH: flag, MEDIUM: flag, LOW: allow}	Maps data sensitivity levels to enforcement actions, enabling graduated risk handling rather than binary allow/block.
ddl_allowed	Boolean	TRUE	Controls whether schema-altering queries are permitted for the role.
multi_statement_allowed	Boolean	TRUE	Determines whether compound SQL statements are allowed, mitigating piggy-backed attack vectors.
temporal_constraints	Nullable	null	Time-bound access control (e.g., business hours, weekdays)
escalation_policy	Dict	{flag: require_human}	Defines how flagged queries are escalated, supporting human-in-the-loop governance.
notes	String	auto-generated	Metadata for policy provenance and lifecycle management.

Knowledge base artefacts required from administrators

To avoid table-specific brittle governance, the system relies on a small knowledge base (KB) that administrators populate. This KB contains schema and sensitivity context used by detectors and policy checks:

1) Table–tag mappings / tag vectors

Each table is associated with one or more semantic tags (e.g., financial, pii, public, admin). These tags are encoded into vectors that can be compared and aggregated across queries.

2) Tag ontology

A controlled vocabulary of tags ensures consistency across policies and evaluation.

3) Optional schema dictionary

For schema integrity and plausibility, the system can load known table/column relationships, supporting the detection of hallucinated schema references.

Below table documents static knowledge artefacts loaded by administrators to contextualise query evaluation. These artefacts represent enterprise ground truth, not learned behaviour.

Table 3.2: Knowledge Base (KB) Artefacts Catalogue

Artefact / File	**Type**	**Created / Maintained by**	**Used by (module)**	**Purpose in the Context Policy Engine**
table_catalog_kb.json	Admin KB (authoritative catalogue)	Admin configuration	Tag-based RBAC checks, sensitivity policy checks	Canonical mapping from tables to tags + sensitivity. Enables policy decisions at the data-category level (tags) rather than brittle table-name lists.
table_freq.csv	Ingested stats (from logs)	DB logs / ingest jobs	Plausibility, novelty gating, low-frequency checks	Captures how often tables appear in normal workloads. Helps detect suspicious references to rare/unused tables (or sudden targeting of sensitive tables).
table_columns.csv	Ingested stats (from logs)	DB logs / ingest jobs	Schema integrity checks, plausibility (column realism)	Supports plausibility of column usage: whether the query references columns that are historically present/used for that table.
table_cooccurrence.csv	Ingested relational stats	DB logs / ingest jobs	Join plausibility checks	Learns empirical join relationships from real workloads. Detects improbable joins (a common sign of hallucinated or exploratory malicious queries).
table_metadata.json	Ingested/curated metadata	DB logs / ingest jobs (or governance feed)	Grace-period logic, provenance/novelty logic	Enables controlled handling of newly introduced tables via “first seen” information, supporting a grace period to avoid false alarms immediately after schema evolution.

table_tag_vectors.json	Derived KB (table embedding in tag-space)	Derived from table_catalog_kb.json + tag_vocab.json	Tag similarity, policy alignment, plausibility/provenance	A machine-friendly representation of the table's semantic category membership, enabling fast similarity computations without repeated parsing of tag lists.
user_tag_vectors.json	Derived KB (user behavioural profile)	Derived from historical usage + tag mappings	Provenance novelty, session/contextual alignment	Captures what data categories a user typically accesses. Used to quantify behavioural novelty when a user suddenly targets new tag categories (especially sensitive ones).

Tag-based mapping v/s table-based permissions

Tag-based governance is chosen because it is more stable under schema change and more expressive than raw table lists.

- In large systems, tables change names, split, merge or are replaced. Hardcoding table lists yields policy drift and administrative burden.
- Tags allow policy rules like "data_analyst can read analytics + public, but cannot read pii" which generalizes naturally even when new tables are introduced.
- Tag mapping also aligns with enterprise data governance practices, where sensitivity is usually defined by category, not by table identifier.

Methodologically, this design supports the thesis goal that the context engine is not a static rule system. It is a governance-aware filter designed for evolving enterprise environments.

Policy Catalog hashing and integrity

A further design element is policy hashing, where each role policy file is hashed and recorded in a Catalog. The rationale is methodological rigor and operational safety.

- It guarantees that experiments are reproducible. The exact policy configuration used for evaluation can be verified later.
- It supports auditability. Policy changes are detectable and can be linked to changes in evaluation outcomes.
- It aligns with governance expectations in enterprise security, where configuration tampering is itself a threat.

This also strengthens our model by demonstrating that the proposed system is not only a model, but a controlled, auditable security mechanism.

3.5.4 Query Parsing and Context Extraction

Before any policy or detector logic executes, the engine parses the SQL query into a lightweight intermediate representation

- operations present (SELECT/INSERT/UPDATE/DELETE/DDL)
- tables mentioned
- columns mentioned (if extractable)
- presence of UNION, comments, encoded tokens, tautology patterns
- number of statements (single vs multi-statement)
- structural patterns (joins, nesting depth)

The engine also consumes contextual metadata – role, user_id and timestamp.

This step is critical because the policy engine does not operate over raw tokens alone. Its reasoning depends on the relationship between extracted intent and the governance configuration.

3.5.5 Hard Deterministic RBAC and Governance Checks

Hard checks are rules that represent non-negotiable enterprise constraints. They are executed first to achieve early enforcement and computational efficiency (a query that is clearly disallowed should not require behavioural scoring).

Common hard checks include:

- Operation not allowed for role: Example: an appuser role issuing DROP or ALTER is blocked immediately.
- Write operations on restricted data categories: Example: writing into pii or financial tags is blocked or flagged, depending on policy strictness.
- Explicit banned constructs: Example: OS-level functions, high-risk stored procedure invocation, or administrative commands.
- Multi-statement execution: Multi-statement queries are a common carrier for piggybacking and for combining benign and malicious intent. Even when not strictly malicious, enterprises often require additional scrutiny. The policy therefore supports configurable responses such as "flag always" or "block for non-admin roles"

These checks are described as "hard" because they are deterministic, auditable and driven by policy files. This directly addresses the gap where ML-only systems may confidently classify a query as benign even when it violates business governance.

3.5.6 Detector Suite Overview and Rationale

If hard checks pass, the engine evaluates a set of contextual detectors. This stage exists because there are important enterprise risks that cannot be captured by static RBAC rules alone:

- LLMs can produce syntactically valid queries that are implausible with respect to schema usage.
- A user account can be compromised. The query might be syntactically normal but behaviourally abnormal.
- The same role might have access to a domain but not at unusual times or volumes.

Each detector returns:

- a score normalized to [0,1]
- a diagnostic record that supports explainability and auditing

The final context action is derived by combining these detector outcomes with policy thresholds.

3.5.7 Plausibility Detector Methodology (Three-Factor Scoring)

The plausibility detector is designed to identify schema-grounding failures, which are frequent in LLM-generated SQL. Such failures include referencing tables that are rarely used together, joining unrelated entities or selecting columns that do not exist. This is not captured by classical SQL injection signatures, because the query may not contain overt injection patterns.

Plausibility is intentionally decomposed into three factors:

- **Table existence plausibility** - Captures whether referenced tables are known/expected.
- **Join plausibility** - Captures whether the combination of tables is historically coherent.
- **Column plausibility** - Captures whether column references align with known schema usage.

This decomposition is important because plausibility is multi-dimensional, a query may reference valid tables but join them in an implausible way.

Grace period and why it matters

A key design choice is the introduction of a grace period / smoothing for newly added or low-frequency tables. Without this, the plausibility detector would systematically over flag legitimate queries immediately after schema changes.

The grace mechanism exists for two reasons:

- Real systems evolve: new tables or columns may not appear in historical logs initially.
- The detector is statistical: low frequency does not necessarily mean “invalid”.

Therefore, the plausibility model uses smoothing (and optionally a grace window) to avoid treating new but legitimate as hallucinated.

Overall plausibility score

Let T be table plausibility, J join plausibility, and C column plausibility. Then:

$$P = w_T \cdot T + w_J \cdot J + w_C \cdot C$$

The weights are policy-configurable. This is another methodological choice, different environments value different signals (e.g., banking schemas may prioritize column integrity more than join novelty).

3.5.8 Provenance Novelty Detector

Provenance novelty measures how far a query’s data-access signature deviates from a user's historical patterns. This is particularly important under LLM-driven workflows because:

- LLMs may generate a query that accesses unfamiliar domains due to prompt drift.
- Attackers may exploit LLM tools to extract sensitive data that the user has never accessed before.
- Traditional SQLi datasets do not model this behavioural dimension.

Mathematical formulation (cosine similarity)

Let u be the user's historical tag vector and q be the query tag vector. Provenance similarity is computed using cosine similarity:

$$\text{cosine\_sim}(u, q) = \frac{u \cdot q}{\| u \| (\| q \| + \epsilon) + \epsilon}$$

A small factor ϵ is introduced to prevent division by zero and stabilizes computation when:

- the user vector is empty (new user, no history)
- the query vector is sparse (e.g., rare or unmapped tags)

Then novelty is:

$$\text{novelty}(u, q) = 1 - \text{cosine\_sim}(u, q)$$

Without ϵ, novelty scoring can become undefined or numerically unstable. Introducing ϵ is a standard numerical safeguard and is justified for robustness in early-session conditions.

3.5.9 Session anomaly detector

Session anomaly detection captures short-term behavioural deviations. This detector is essential because an attacker or misaligned LLM workflow may issue many queries rapidly or shift to new tables abruptly, even if each individual query is syntactically harmless.

The detector therefore models three aspects:

- Rate spike (recent rate vs baseline rate)
- Table novelty (fraction of tables not seen historically)
- Operation surprise (unusual operation type for user)

The final score is a weighted combination:

$$S = w_R R + w_N N + w_O O$$

This design is intentionally interpretable. Rather than producing a single opaque anomaly score, it exposes which component drove the anomaly.

3.5.10 Schema Integrity Detector

Schema integrity checks are used to catch structural anomalies that are risky in enterprise settings, such as suspicious joins or unexpected cross-domain coupling. This is particularly relevant when LLMs generate queries that are logically plausible in language but inconsistent with schema realities.

This detector typically flags:

- joins between rarely co-occurring tables
- cross-tag joins involving sensitive domains
- structurally abnormal query shapes relative to report-like queries

The schema detector complements plausibility. While plausibility focuses on existence/likelihood, schema integrity focuses on risky pairing.

3.5.11 Decision Fusion logic and Context action output

All the above specified detectors are evaluated against the configuration threshold set by admin there by allowing flexibility in setting the thresholds dynamically as per the application context.

Below table summarises configurable thresholds and weights that control detector sensitivity. These parameters are policy-tuneable, not model-learned, allowing security teams to adapt risk posture without retraining.

Table 3.3: Prefilter and Detector Threshold Configuration

Parameter	Value	Applied To	Purpose
plausibility_alpha	1	Plausibility Detector	Controls smoothing in plausibility score computation to stabilise rare-event behaviour.
grace_days	7	Provenance Novelty	Grace period for newly introduced tables to prevent false positives after schema changes.
plausibility_flag_thresh	0.2	Plausibility Detector	Threshold above which queries are flagged for review.
plausibility_block_thresh	0.8	Plausibility Detector	Threshold triggering immediate blocking due to implausible access patterns.
join_low_threshold	0.01	Join Plausibility	Identifies statistically improbable joins indicative of inference or reconnaissance attacks.
provenance_novelty_flag	0.5	Provenance Detector	Flags queries accessing rarely or never-used tables.
provenance_novelty_block	0.8	Provenance Detector	Blocks extreme deviations from historical access patterns.
session_anomaly_flag	0.6	Session Detector	Flags abnormal query rate or behavioural bursts.
session_anomaly_block	0.7	Session Detector	Blocks high-confidence session-level abuse.
rate_weight	0.7	Session Scoring	Weight assigned to request rate deviation.
novelty_weight	0.2	Session Scoring	Weight assigned to access novelty.
op_weight	0.1	Session Scoring	Weight assigned to operation-type deviation.

After computing detector scores, the engine maps outcomes into one of three actions:

- ALLOW: no hard violation; detector scores within safe bounds
- FLAG: ambiguous risk; requires model inference and/or review
- BLOCK: hard violation or combined detector evidence exceeding policy thresholds

The methodology is intentionally conservative - A single strong reason (e.g., hard RBAC violation) may block, while weaker evidence triggers flagging.

The output is structured with the following components:

- Action
- reasons[] (human-readable explanations)
- diagnostics{} (score breakdown and evidence)
- matched_tables, matched_tags

- policy_hash (for reproducibility)

This output is later consumed by the Risk Tier Decision layer and the XAI module.

3.5.12 Contextual metadata augmentation for Evaluation

Open SQL datasets (including SQL injection corpora and Kaggle dataset) typically lack:

- role labels
- user identities
- timestamps
- session windows

However, these fields are mandatory for evaluating RBAC checks, provenance novelty and session anomaly. Therefore, the dataset is augmented with synthetic-but-controlled metadata (user_id, role, timestamps). This is not used to create synthetic SQL queries. Instead, it provides the minimum contextual scaffolding required to evaluate the context policy layer meaningfully.

Augmentation details

- Roles are assigned according to plausible enterprise distributions.
- User IDs are generated and attached consistently so that session history exists.
- Timestamps are sampled from a fixed date window to support time-based windows.

Crucially, the SQL text remains unchanged. The purpose is to enable context evaluation, not to manipulate the underlying maliciousness distribution. This methodological step strengthens evaluation validity. Without metadata augmentation, the context policy engine could not be tested beyond trivial rule checks.

3.5.13 Policy and Security Ground-Truth Augmentation

Traditional SQL security datasets label queries solely on syntactic or semantic maliciousness, typically focusing on known injection patterns or exploit templates. However, within agentic AI database systems, security violations arise not only from explicitly malicious SQL but also from contextual misuse, such as role violations, unauthorized access to sensitive data domains or abnormal behavioural patterns. Consequently, relying on a single binary malicious label is insufficient to evaluate a system designed to enforce context-aware security controls.

To address this limitation, this research introduces augmented ground-truth labels that explicitly model policy violations and holistic security violations, aligning evaluation with real-world enterprise threat semantics.

Policy Violation Ground Truth (policy_violation_gt)

The variable policy_violation_gt represents whether a query violates deterministic administrative policies, independent of statistical threat likelihood.

This label is derived directly from the Context Policy Engine (Section 3.4), based on:

- Role-based access control (RBAC) constraints
- Allowed and denied SQL operations
- Sensitivity-aware table and tag restrictions
- Multi-statement and escalation rules

Formally,

$$policy\_violation\_gt = \begin{cases} 1, & \text{if Context Policy outcome} \in \{\text{FLAG,BLOCK}\} \\ 0, & \text{if Context Policy outcome} = \text{ALLOW} \end{cases}$$

This binary signal captures normative violations, even when the SQL itself appears syntactically benign.

Security Violation Ground Truth (security_violation_gt)

While policy violations capture governance breaches, they do not fully represent adversarial intent. Therefore, a second composite label security_violation_gt is constructed to reflect end-to-end security risk.

This label integrates:

- Contextual policy violations
- Statistical maliciousness inferred from the fusion threat model

The augmentation rule is defined as:

$$security\_violation\_gt = \begin{cases} 1, & \text{if } policy\_violation\_gt = 1 \\ 1, & \text{if } P(\text{malicious}) \geq \tau_{low} \\ 0, & \text{otherwise} \end{cases}$$

where,

- $P(\text{malicious})$ is the fusion model probability
- τ_{low} is the empirically selected lower decision threshold (Section 4.6)

This formulation ensures that either contextual misuse or high-confidence malicious behavior is sufficient to flag a security violation.

3.5.14 Novelty contribution of Context policy engine

The novelty of this module lies in the deliberate integration of:

- Admin configurable deterministic governance (policy-as-data, role files, policy hashing)
- Schema aware plausibility modelling with change-tolerance (smoothing + grace)
- User centric provenance novelty to capture cross-domain drift characteristic of LLM errors
- Session anomaly modelling to capture behavioural threat patterns absent in typical SQLi detectors
- Explainability-by-design, producing structured justifications rather than relying on post-hoc explanations
- The dual-label strategy introduces three methodological advantages:
    - Decouples governance violations from exploit detection, allowing independent evaluation
    - Prevents under-labelling of LLM-induced misuse, which often lacks classical attack signatures
    - Aligns evaluation with enterprise enforcement semantics, rather than dataset-centric assumptions

Taken together, this module addresses a key weakness in many prior systems i.e., ML-only detectors can score a query as benign while it violates enterprise policy, and rule-only systems cannot adapt to behavioural drift and schema hallucination. The proposed context engine is therefore positioned as the primary methodological contribution of the thesis, enabling a realistic governance layer for agentic LLM systems that generate executable SQL.

3.6 Statistical Feature Extractor

3.6.1 Motivation for Statistical Feature Extraction

While deep learning models such as Convolutional Neural Networks (CNNs) are effective in capturing semantic and token-level patterns in SQL queries, they are not sufficient in isolation for robust SQL threat detection. Empirical evidence from prior research and practical

deployments indicates that structural and distributional properties of SQL queries often carry strong attack signals that may not be semantically explicit. Examples include excessive use of special characters, abnormal keyword repetition, obfuscated encodings or unusual clause ordering.

To address this limitation, the proposed system integrates a statistical feature extraction layer that explicitly models low-level structural characteristics of SQL queries. This design decision follows a hybrid detection philosophy, where interpretable, rule-aligned features complement representation-learning models. Such a hybrid approach improves detection robustness, supports explainability and reduces over-reliance on opaque neural representations, an important consideration for enterprise security systems.

The statistical feature extractor operates on normalized SQL queries (as described in Section 3.3) and produces a fixed-length numerical vector that captures lexical density, syntactic irregularities, keyword distribution and obfuscation indicators.

3.6.2 Design Principles

The design of the statistical feature extractor is guided by four principles:

1) Attack Sensitivity

   Features are selected based on known SQL injection patterns, including tautologies, UNION-based injections, time-based attacks, stacked queries and encoded payloads.

2) Model Agnostic Interpretability

   Each feature has a clear semantic meaning, enabling downstream explainability modules (e.g., SHAP) to provide human-interpretable rationales for query blocking decisions.

3) Low Computational Overhead

   All features are computable in linear time with respect to query length, ensuring suitability for real-time enforcement pipelines.

4) Complementarity to CNN Encoder

   Statistical features capture global structural properties that CNNs—focused on local n-gram patterns may underweight or overlook.

3.6.3 Feature categories

The extracted features can be grouped into four conceptual categories:

- Structural Features
- Lexical/keyword features
- Statistical features
- Contextual features

This grouping is reflected in the explanation table below.

3.6.4 Statistical Feature set

Table 3.4: Statistical features used for SQL query characterization.

Sl. No.	Feature Name	Description	Importance
1	Query length	Total number of characters in the query	Injection payloads are often longer than benign queries due to appended logic
2	Token Count	Number of lexical tokens after parsing	Abnormally high token counts indicate injected logic blocks
3	Average Token length	Mean length of tokens	Obfuscation often inflates token length
4	Special Character Ratio	Ratio of non-alphanumeric characters	High ratios signal encoded or malformed payloads
5	Quote Count	Number of single and double quotes	Quote imbalance is a classic SQLi indicator
6	Comment Token Count	Count of SQL comment markers (--, /* */)	Used to truncate legitimate query logic
7	Semicolon Count	Number of ; characters	Indicates stacked or piggy-backed queries
8	Numeric Literal Ratio	Ratio of digits to total characters	Numeric overuse is common in tautology injections
9	Whitespace Ratio	Proportion of whitespace characters	Obfuscated queries often manipulate spacing
10	Keyword Count	Total SQL keyword occurrences	Attack queries often overuse control keywords
11	Keyword Density	Keywords per token	High density suggests logic-heavy injections
12	UNION Keyword Count	Frequency of UNION	Core indicator of UNION-based SQL injection
13	SELECT Keyword Count	Frequency of SELECT	Nested SELECTs are common in data exfiltration
14	Boolean Operator Count	Count of AND, OR	Used to manipulate WHERE clause truth values
15	Tautology	Occurrences of patterns like 1=1	Strong signal of authentication bypass
16	Comparison Operator Count	Count of =, <, >	High frequency suggests logic manipulation

17	Function Call Count	Number of SQL functions used	Attack payloads invoke system or timing functions
18	Time-Delay function count	Count of SLEEP, BENCHMARK, pg_sleep	Indicates time-based blind SQL injection
19	Hexadecimal Literal Count	Presence of hex-encoded values	Used to evade string filters
20	URL-Encoded Token Count	Count of %xx patterns	Signals encoded obfuscation attempts
21	Entropy Score	Shannon entropy of the query string	High entropy implies random or encoded payloads
22	Uppercase Ratio	Ratio of uppercase characters	Keyword-heavy attacks inflate uppercase usage
23	Identifier Repetition score	Repeated table/ column identifiers	Suggests brute-force schema probing
24	Clause Depth	Approximate nesting depth of subqueries	Deep nesting is uncommon in benign queries
25	Operator-to-Operand Ratio	Balance between operators and literals	Imbalance indicates injected logic
26	Symbol Burst Indicator	Presence of dense symbol clusters	Signals payload concatenation
27	Non-ASCII Character Flag	Presence of extended Unicode characters	Used for evasion via homoglyph attacks

3.6.5 Integration with hybrid detection pipeline

The extracted statistical feature vector is standardized using z-score normalization and subsequently used in two ways:

- Standalone Evaluation: To assess the discriminative power of structural features independently.
- Feature Fusion: To form a fused representation with CNN-derived semantic embeddings, enabling the threat classifier to jointly reason over what the query says and how the query is constructed.

This fusion strategy directly addresses a key research gap identified in Chapter 2: the lack of systems that simultaneously model syntactic irregularities and semantic intent in LLM-generated SQL queries.

3.6.6 Contribution to Explainability and Policy enforcement

Unlike deep embeddings, statistical features offer direct traceability. When a query is flagged, the system can explicitly report reasons such as:

- excessive boolean logic

- abnormal keyword density
- presence of time-delay function
- encoded obfuscation patterns

This capability strengthens auditability and supports downstream modules such as the explainability engine and human review workflows.

3.6.7 Summary

The statistical feature extractor forms a critical interpretability and robustness layer within the proposed SecureDBAgent architecture. By explicitly modelling structural deviations from benign SQL behavior, it complements neural representations, improves generalization to zero-day attacks, and aligns detection decisions with enterprise security reasoning.

3.7 CNN based Token Encoder

3.7.1 Motivation for semantic encoding of SQL queries

While statistical features capture how a query is structured, they are insufficient to fully represent what the query intends to do. This limitation becomes particularly pronounced in the context of LLM-generated SQL, where queries may be syntactically valid and structurally benign, yet semantically unsafe due to subtle intent shifts, hallucinated joins or over-privileged access paths.

To address this challenge, the proposed framework incorporates a token-level semantic encoder that learns distributed representations of SQL queries directly from token sequences. The objective of this encoder is to capture latent semantic patterns, including relationships between clauses, misuse of sensitive attributes and intent-altering token combinations that are difficult to encode through handcrafted rules alone.

A Convolutional Neural Network (CNN) based encoder is adopted for this purpose due to its suitability for short, structured sequences and its strong empirical performance in text classification tasks involving localized patterns.

3.7.2 Why CNN for SQL token encoding?

The choice of CNN over alternative sequence models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and Transformer architectures is deliberate and grounded in both theoretical and practical considerations.

1) Local Pattern Sensitivity

   SQL injection and malicious query intent often manifest as localized token patterns, such as:

   - OR 1 = 1
   - UNION SELECT
   - SLEEP (n)
   - comment-based truncation (--, /* */)

   CNNs are inherently effective at detecting such n-gram-like patterns through convolutional filters, without requiring explicit feature engineering. In contrast, recurrent models emphasize long-range dependencies, which are less critical for SQL threat detection.

2) Computational Efficiency

   CNNs offer significantly lower inference latency compared to LSTMs and Transformers, as they:

   - avoid sequential dependency during computation
   - support parallel convolution operations
   - require fewer parameters for comparable performance on short sequences

   This efficiency is essential for real-time SQL mediation, where security checks must not introduce perceptible query latency.

3) Robustness to Token Position Variability

   Malicious SQL patterns frequently appear at varying positions within a query. CNNs provide translation invariance, enabling the model to recognize malicious constructs regardless of their location in the token sequence - an advantage over position-sensitive recurrent models.

3.7.3 Comparative Justification of Encoder choices

Table 3.5: Comparison of Token Encoding Architectures for SQL Threat Detection

Model Type	**Strength**	**Weakness**	**Reason for Acceptance/ Rejection in our work**
CNN (1D)	-Captures local and sequential token patterns	-Less effective on long range	Chosen: Best trade-off for real-time SQL query modelling. Captures

	-Fast inference -Low training complexity	dependencies(vs RNNs) -Requires fixed input format	semantic structures efficiently.
MLP (Dense layers)	- Learns domain specific features - Supports tabular/static inputs - Reduces FP via syntax	- No sequence learning - Needs feature engineering	Used alongside CNN - complements CNN with hand picked static features.
RandomForest / XGBoost	-Strong on structured /tabular data -Interpretable results	-Poor sequential pattern modelling -High false positives for obfuscated/ nested attacks	Rejected: Cannot capture sequential semantics vital for SQL injection detection
RNN/ BiLSTM	-Good for sequence modelling -Handles long-term dependencies	-High computational costs -Slow inference -Difficult to train	Rejected: Overkill for short SQL queries; low efficiency for real-time security system.
Transformer models	-Excellent for long range attention -State-of-the-art performance on many NLP tasks	-Requires large training data -High memory and compute requirements	Rejected: Not suitable due to limited query length and resource constraints.

The selected CNN encoder provides the optimal balance between semantic expressiveness, interpretability and operational efficiency for the problem domain.

3.7.4 Tokenization and input representation

Each SQL query is first normalized (as described in Section 3.3) and tokenized into a sequence of SQL tokens, including:

- SQL keywords (e.g., SELECT, FROM, WHERE)
- operators and symbols

- identifiers and literals (normalized where applicable)

Tokens are mapped to integer indices using a fixed vocabulary constructed from the training corpus. Queries are either padded or truncated to a maximum sequence length L, ensuring uniform input dimensions. Each token index is transformed into a dense embedding vector of dimension d using an embedding layer that is trained jointly with the CNN encoder.

3.7.5 CNN Encoder architecture

The CNN encoder consists of the following components:

1) Embedding Layer

   Converts token indices into dense vectors:

$$\mathbf{E} = [e_1, e_2, \dots, e_L], e_i \in \mathbb{R}^d$$

2) Convolutional Layers

   Multiple 1-D convolution filters with varying kernel sizes $k \in \{2,3,4,5\}$ are applied to capture token n-grams of different lengths.

   The convolution operation for filter f is defined as:

$$c_i^{(f)} = \sigma(\mathbf{W}^{(f)} \cdot \mathbf{E}_{i:i+k-1} + b^{(f)})$$

3) Global Max Pooling

   For each filter, max pooling extracts the most salient feature:

$$\hat{c}^{(f)} = \max_i c_i^{(f)}$$

4) Feature Concatenation

   Pooled outputs from all filters are concatenated to form a fixed-length semantic embedding vector:

$$\mathbf{h}_{\text{CNN}} = [\hat{c}^{(1)}, \hat{c}^{(2)}, \dots, \hat{c}^{(F)}]$$

   This vector represents the semantic intent embedding of the SQL query.

3.7.6 Regularization and Generalization strategy

To prevent overfitting and enhance generalization to unseen SQL patterns, the following strategies are employed:

- Dropout layers applied after convolutional blocks
- Early stopping based on validation loss
- Class-balanced training to handle skewed benign/malicious distributions

These measures are particularly important given the evolving nature of SQL attacks and the need for robustness against zero-day payloads.

3.7.7 Role of CNN embedding in overall framework

The output of the CNN encoder does not operate in isolation. Instead, it is fused with statistical features (Section 3.5) to form a joint representation that captures both:

- semantic intent (what the query is trying to achieve)
- structural anomalies (how the query deviates from benign norms).

This fusion directly addresses a critical gap identified in the literature: the lack of systems that jointly model structure and semantics for AI-generated SQL security.

3.7.8 Summary

The CNN-based token encoder provides a computationally efficient and semantically expressive mechanism for modelling SQL query intent. Its architectural simplicity, low latency and ability to capture localized malicious patterns make it particularly well-suited for deployment in real-time agentic AI pipelines. When combined with statistical features and contextual policy enforcement, it forms a robust foundation for multi-layer SQL threat detection.

3.8 Feature Fusion Methodology

3.8.1 Motivation for feature fusion

A key limitation observed in existing SQL security approaches is their over-reliance on a single representation space, either handcrafted rules or learned semantic embeddings. Rule-based systems struggle with semantic ambiguity, while purely neural approaches often fail to respect structural and policy-level constraints intrinsic to enterprise databases.

The proposed SecureDBAgent addresses this limitation through a feature fusion strategy that explicitly combines:

- Statistical (structural) features, capturing syntactic irregularities and attack heuristics
- Semantic embeddings, capturing latent intent learned from token sequences

This fusion enables the system to reason jointly over how a query is written and what the query is attempting to do, a capability that is particularly critical for LLM-generated SQL, where intent may be misaligned with access policies despite syntactic correctness.

3.8.2 Feature spaces to be fused

Let

- $\mathbf{f}_{\text{stat}} \in \mathbb{R}^n$ denote the statistical feature vector extracted by the static feature extractor (Section 3.5), and
- $\mathbf{f}_{\text{cnn}} \in \mathbb{R}^m$ denote the semantic embedding produced by the CNN token encoder (Section 3.6).

These two representations are heterogeneous in nature:

$\mathbf{f}_{\text{stat}}$ is low-dimensional, interpretable and policy-aligned.

$\mathbf{f}_{\text{cnn}}$ is high-dimensional, distributed and intent-driven.

Directly relying on either representation in isolation would result in partial threat visibility, motivating the need for fusion.

3.8.3 Fusion strategy and mathematical formulation

The SecureDBAgent employs early fusion via feature concatenation, followed by normalization. This choice is motivated by three considerations:

- Preservation of Interpretability
  Concatenation ensures that statistical features remain explicitly accessible for explainability (e.g., SHAP), rather than being entangled within deep layers.
- Low Computational Overhead
  Early fusion avoids the complexity of multi-branch late fusion architectures, supporting real-time inference requirements.
- Compatibility with Classical Classifiers
  The resulting fused vector can be consumed by lightweight dense classifiers without architectural coupling.

The fused feature vector is defined as:

$$\mathbf{f}_{\text{fusion}} = \text{Normalize}([\mathbf{f}_{\text{stat}} \;||\; \mathbf{f}_{\text{cnn}}])$$

where:

- $||$ denotes vector concatenation
- normalization is applied to prevent dominance of the higher-dimensional CNN embedding

3.8.4 Normalization and Feature scaling

To ensure balanced contribution from both feature spaces, z-score normalization is applied:

$$\hat{f}_i = \frac{f_i - \mu_i}{\sigma_i + \epsilon}$$

Where:

- μ_i and σ_i are computed from the training set
- ϵ a small constant introduced for numerical stability.

This step is crucial in preventing semantic embeddings from overshadowing statistical signals during classification.

3.8.5 Comparative analysis against late fusion and attention-based fusion

Alternative fusion strategies were considered but deliberately excluded:

- Late fusion (independent classifiers with score aggregation) was rejected due to loss of joint feature interactions.
- Attention-based fusion was avoided due to increased model complexity, reduced interpretability and higher inference latency.

The selected fusion approach reflects a design trade-off favouring explainability, operational efficiency and auditability, consistent with enterprise security requirements

3.8.6 Contribution to Novelty

The feature fusion methodology contributes to the novelty of this work by:

- enabling joint structural-semantic reasoning for SQL security
- supporting explainable decisions across both feature domains
- providing a deployment-ready compromise between deep learning performance and policy transparency

This directly addresses a gap in the literature, where most systems emphasize either ML-based detection or rule-based enforcement, but rarely integrate both coherently.

3.9 Multi Layer Perceptron - Threat Classifier architecture

3.9.1 Rationale for Selecting an MLP Classifier

The final threat classification stage in SecureDBAgent is implemented using a Multilayer Perceptron (MLP). This design choice is deliberate and grounded in the nature of the fused feature representation produced by the preceding stages.

After feature fusion (Section 3.7), the system operates on a fixed-length, dense numerical vector that combines:

- normalized statistical indicators
- high-level semantic embeddings from the CNN encoder

An MLP is particularly well-suited for this representation because it is designed to model non-linear interactions between heterogeneous numerical features without imposing structural assumptions about spatial or sequential locality.

Unlike sequence models or tree-based classifiers, the MLP treats the fused vector holistically, allowing it to learn cross-feature dependencies for example, how a specific semantic pattern becomes risky only when paired with certain structural anomalies.

3.9.2 Architecture of the MLP Classifier

The MLP used in SecureDBAgent follows a lightweight, fully connected feedforward architecture, optimized for low-latency inference while maintaining high discriminative power. The architecture consists of:

1) Input Layer
    a. Dimensionality equal to the fused feature vector $\mathbf{f}_{\text{fusion}}$
    b. Receives the concatenated and normalized statistical and semantic features.
2) Hidden Layer 1
    a. Fully connected dense layer.
    b. Applies a non-linear activation function (ReLU).
    c. Learns primary interactions between structural and semantic features.
3) Hidden Layer 2
    a. Fully connected dense layer with reduced dimensionality.
    b. Acts as a feature compression and abstraction stage.
    c. Helps prevent overfitting by discouraging memorization of individual feature patterns.
4) Output Layer
    a. Single neuron with sigmoid activation.
    b. Produces a probability score $p_{\text{mal}} \in [0,1]$

This gradual dimensionality reduction encourages the model to learn robust latent threat representations, rather than relying on any single dominant feature

3.9.3 Regularization and Stability Considerations

To ensure generalization and training stability, the following mechanisms are incorporated:

- Dropout layers between hidden layers to reduce co-adaptation of neurons.
- Batch normalization to stabilize gradients and improve convergence.
- Early stopping based on validation loss to prevent overfitting.

These design choices collectively ensure that the classifier remains robust across diverse SQL workloads, including previously unseen query patterns.

3.9.4 Why MLP and Not Other Classification Models?

The selection of an MLP over alternative classifiers is justified as follows:

Table 3.6: Model Comparison for Threat classifier

Model Type	**Reason for Exclusion**
Decision Trees / Random Forests	Struggle with high-dimensional dense embeddings and offer limited benefit once CNN features are introduced.
Support Vector Machines	Computationally expensive at inference time and less scalable for large datasets.
Gradient Boosting (XGBoost)	Performs well on tabular data but does not integrate naturally with learned neural embeddings.
Recurrent Models (LSTM/GRU)	Redundant, as sequential modelling is already handled by the CNN encoder.
Transformer-based Classifiers	Excessively complex for post-fusion classification and detrimental to latency requirements.

The MLP provides the optimal balance between expressive power, interpretability and operational efficiency, making it particularly suitable for an enterprise security enforcement layer.

3.9.5 Contribution to Methodological Novelty

The use of MLP as a post-fusion decision layer, rather than as a primary feature extractor, is a critical design distinction. It allows SecureDBAgent to:

- decouple representation learning from decision-making
- support explainability mechanisms more effectively
- maintain flexibility in evolving upstream feature representations

This design directly addresses gaps in prior work where monolithic deep models' obscure decision rationale and increases deployment risk.

3.10 Risk Tier Decision

3.10.1 Motivation for a Risk-Tiered Decision Framework

Binary classification alone is insufficient for enterprise database security enforcement, particularly in the context of LLM-generated SQL. A rigid allow/deny mechanism can lead to excessive false positives, operational friction or silent acceptance of borderline unsafe queries. Enterprise security decisions often require graded responses rather than absolute outcomes. For example:

- some queries may be syntactically valid but contextually unusual
- others may show weak malicious signals but violate governance expectations
- certain queries may require human review rather than immediate blocking

To address this, SecureDBAgent introduces a risk tier decision layer that converts probabilistic model outputs into actionable security decisions aligned with operational realities.

3.10.2 Inputs to the Risk Tier Decision Layer

The risk tier decision layer aggregates outputs from multiple upstream components:

1) Fusion Model Threat Probability

   The MLP classifier (Section 3.8) outputs a probability score: $p_{\text{mal}} \in [0,1]$, representing the likelihood that a query is malicious.

2) Context Policy Outcome

   Deterministic context policy checks (Section 3.4) yield one of

   - ALLOW
   - FLAG
   - BLOCK

3) Policy Severity Metadata

   Certain policy violations (e.g., privilege escalation, sensitive table access) are pre-classified as high severity and override probabilistic decisions.

The risk tier logic integrates these signals to produce a final security action.

3.10.3 Definition of Risk Tiers

SecureDBAgent defines three discrete risk tiers:

Risk Tier	Meaning	Action
LOW	Query appears benign and contextually valid	Allow execution
MEDIUM	Query exhibits suspicious characteristics or contextual anomalies	Flag for review
HIGH	Query is malicious or violates critical policy constraints	Block execution

This tiering reflects real-world security workflows, where not all anomalies warrant immediate rejection.

3.10.4 Threshold-Based Risk Assignment

The fusion model probability score, p_{mal} is mapped to risk tiers using two decision thresholds:

- Low threshold τ_{low}
- Upper threshold τ_{high}

The assignment rule is defined as:

$$\text{Risk Tier} = \begin{cases} \text{LOW}, & p_{\text{mal}} < \tau_{\text{low}} \\ \text{MEDIUM}, & \tau_{\text{low}} \leq p_{\text{mal}} < \tau_{\text{high}} \\ \text{HIGH}, & p_{\text{mal}} \geq \tau_{\text{high}} \end{cases}$$

This separation allows SecureDBAgent to:

- tolerate low-risk uncertainty
- escalate ambiguous cases for inspection
- decisively block high-confidence threats

3.10.5 Threshold Optimisation Strategy

Selecting appropriate thresholds is critical. Arbitrary threshold selection can significantly distort false positive and false negative rates.

SecureDBAgent employs data-driven threshold optimisation using:

1) ROC Curve Optimisation (Youden's Index)

   The Receiver Operating Characteristic (ROC) curve evaluates the trade-off between true positive rate (TPR) and false positive rate (FPR).

   The optimal ROC threshold is chosen using Youden's Index:

$$J = \text{TPR} - \text{FPR}$$

The threshold that maximizes J represents the best balance between sensitivity and specificity and is used as the upper threshold τ_{high}

2) Precision–Recall Curve Optimisation

In security datasets, malicious samples are often rare, making ROC curves alone insufficient.

Precision-Recall (PR) curves are therefore used to identify the threshold that maximizes the F1-score:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

This threshold captures the best balance between missed attacks and false alarms and informs the lower threshold τ_{low}

3.10.6 Justification for Using Both ROC and PR Curves

Using both ROC and PR optimisation addresses complementary concerns:

Metric	**What it Controls**
ROC (Youden)	Minimises overall classification error
PR (F1)	Reduces false positives under class imbalance

This dual-criterion approach ensures that SecureDBAgent does not over-optimise for accuracy at the cost of operational noise.

3.10.7 Integration with Context Policy Decisions

The risk tier output is not applied in isolation. Context policy outcomes act as hard constraints:

- If context policy returns BLOCK, the final action is BLOCK, regardless of p_{mal}
- If context policy returns FLAG, the minimum risk tier is elevated to MEDIUM.
- Only queries passing context policy checks can be classified as LOW risk.

This hierarchical integration ensures that deterministic governance rules override statistical uncertainty, a key novelty of the system.

3.10.8 Final Decision Logic

The final decision function can be summarised as:

1. Apply context policy checks.
2. Compute fusion model probability.
3. Assign risk tier based on thresholds.
4. Escalate tier if policy violations exist.
5. Emit final action: Allow / Flag / Block.

This structured decision pipeline balances security, explainability and operational practicality.

3.10.9 Contribution to Novelty

The risk tier decision layer contributes novelty by:

- introducing graded enforcement rather than binary blocking
- integrating probabilistic ML outputs with deterministic policy constraints
- enabling explainable, auditable security actions aligned with enterprise norms

Unlike prior SQL injection detection systems that rely solely on model confidence, SecureDBAgent embeds decision intelligence, making it suitable for real-world deployment.

3.10.10 Summary

This section formalised how SecureDBAgent translates model predictions and policy evaluations into concrete security outcomes. By combining threshold optimisation, risk tiering and policy dominance, the system achieves robust and operationally viable decision-making.

3.11 Explainability and Decision transparency

3.11.1 Motivation for explainability in database security

Enterprise database security systems must satisfy not only detection accuracy but also auditability, regulatory compliance and operator trust. Black-box decisions, especially those resulting in blocked queries are operationally unacceptable in regulated environments.

This challenge is amplified in the context of LLM-generated SQL, where administrators must understand whether a query was blocked due to:

- malicious intent
- policy violation

- anomalous behavior
- model uncertainty.

The Explainability Module in SecureDBAgent is therefore designed as a first-class component, not a post-hoc add-on.

3.11.2 Design principles of Explainability module

The explainability framework is built on four guiding principles:

1) Multi-level Explanations
   Provide explanations at the policy, feature and token levels.
2) Model-agnostic Interpretability
   Ensure explanations remain valid even as models evolve.
3) Human Interpretability
   Explanations must be understandable by security analysts, not just ML experts.
4) Actionability
   Outputs should support audit, remediation and feedback to upstream agents.

3.11.3 Static feature attribution using SHAP

For the statistical feature extractor, SHAP (SHapley Additive exPlanations) is employed to compute local feature contributions.

Given a prediction p_{mal}, SHAP values quantify how much each statistical feature contributed to moving the prediction away from a baseline expectation.

This enables statements such as: “The query was flagged primarily due to high tautology density and multiple statement indicators”

SHAP is particularly well-suited here because:

- The feature space is tabular and low-dimensional
- explanations are consistent and additive

3.11.4 Token level saliency for semantic interpretation

To explain semantic embeddings learned by the CNN encoder, gradient-based token saliency is applied. This method measures the change in malicious probability when individual tokens are masked, producing a ranked list of influential tokens.

The result is a token-level attribution map, allowing analysts to identify suspicious SQL fragments such as:

- injected conditions
- timing functions
- obfuscated operators

This directly bridges the gap between neural predictions and human reasoning.

3.11.5 Contextual rule traceability

In addition to ML explanations, SecureDBAgent records:

- which context policies were triggered
- the exact rule conditions violated
- the role or session attributes involved

This ensures deterministic explainability for policy-based decisions, enabling precise administrative action.

3.11.6 Unified explainability output

All explanation components are aggregated into a structured output containing:

- final decision (Allow / Flag / Block)
- risk tier
- ML confidence score
- top contributing features
- token-level highlights
- context policy violations

This unified view supports audit trails, compliance reporting and agent feedback loops.

3.11.7 Summary

The Explainability Module ensures that SecureDBAgent's decisions are not only accurate but also transparent, defensible and operationally actionable. By combining SHAP, token saliency, and policy traces, the system establishes a new standard for explainable AI-driven database security.

3.12 Experimental Protocol, Reproducibility and Validation strategy

3.12.1 Motivation for a Rigorous Experimental Protocol

Security-focused machine learning systems must be evaluated under controlled, transparent and reproducible experimental conditions. Inconsistent training procedures, undocumented data splits or non-deterministic configurations can significantly distort performance metrics, resulting in misleading conclusions particularly in high-stakes domains such as database security.

Given that SecureDBAgent integrates deterministic policy logic, neural models and contextual simulation, a clearly defined experimental protocol is essential to:

- ensure fairness in model comparison
- validate generalisation beyond seen attack patterns
- support independent replication of results

This section formalises the experimental methodology adopted in this research.

3.12.2 Dataset Partitioning Strategy

The datasets used in this study (SQLiV5, Kaggle SQL Injection dataset) are partitioned using a stratified splitting strategy to preserve class distributions.

The partitioning scheme is as follows:

- Training set: Used for model learning
- Validation set: Used exclusively for hyperparameter tuning and threshold optimisation.
- Test set: Held out and used only for final evaluation.

Crucially, no samples from the test set influence model training, threshold selection or architectural decisions, thereby preventing data leakage.

For Spider, which represents a previously unseen workload, the dataset is never used during training or validation and is reserved solely for out-of-distribution generalisation analysis.

3.12.3 Reproducibility Controls and Determinism

Neural network training is inherently stochastic due to random weight initialisation, mini-batch sampling, and GPU-level parallelism. To ensure reproducibility, SecureDBAgent enforces determinism at multiple levels.

1) Global Random Seed Control

   A unified random seed is applied across:

- NumPy
- Python's random module
- TensorFlow/Keras
- Hash-based operations

This ensures that model initialisation, dataset shuffling and dropout behaviour remain consistent across runs.

2) Controlled Training Environment

All experiments are conducted using fixed:

- Python version
- TensorFlow version
- library dependencies
- hardware configuration (CPU/GPU)

By fixing the execution environment, the variance caused by platform-level differences is minimised.

3.12.4 Hyperparameter Optimisation Protocol

Hyperparameter tuning is conducted in two staged phases, ensuring structured exploration and fair comparison.

Phase 1: CNN Encoder Optimisation

Below CNN hyperparameters are tuned using CNN-only models.

- embedding dimension
- number of convolutional filters
- dropout rates

This isolates semantic representation quality without interference from static features.

Only top-performing CNN configurations are propagated to the fusion stage.

Phase 2: Fusion Model Optimisation

Using the best CNN encoders, the fusion model is optimised over:

- projection dimension
- MLP architecture
- learning rate
- dropout configuration

This staged tuning strategy prevents combinatorial explosion and avoids overfitting through uncontrolled search.

3.12.5 Threshold Validation Protocol

Thresholds for risk-tier assignment are derived only from validation data, using:

- ROC (Youden's Index)
- Precision–Recall (F1 optimisation)

Once fixed, thresholds remain unchanged during test evaluation, ensuring that reported results reflect genuine generalisation rather than threshold overfitting

This separation between threshold selection and evaluation is critical for maintaining statistical validity.

3.12.6 Evaluation Under Known and Unknown Conditions

SecureDBAgent is evaluated under two distinct conditions:

1) In-Distribution Evaluation

   Using SQLiV5 and Kaggle datasets, the system is evaluated on:

   - known SQL injection patterns
   - realistic adversarial variants

   Metrics such as ROC-AUC, F1-score and false positive rate quantify detection effectiveness.

2) Out-of-Distribution Evaluation

   The Spider dataset is used to test performance on:

   - benign, human-authored SQL
   - unseen schemas and query structures

   Performance degradation under Spider is analysed to understand

   - over-sensitivity of semantic models
   - limitations of learned threat representations

   This evaluation explicitly tests robustness rather than raw accuracy.

3.12.7 Validation of Context Policy Coverage

In addition to model-centric metrics, this study evaluates:

- Context-policy coverage, defined as the proportion of unsafe queries intercepted before ML invocation.

- Policy-action alignment, ensuring that policy outcomes align with organisational governance expectations

This validation confirms that SecureDBAgent's novelty lies not merely in classification accuracy but in decision governance.

3.12.8 Ethical and Privacy Considerations

All datasets used are:

- publicly available
- anonymised
- free from personally identifiable information

User identifiers and roles used during evaluation are synthetically generated for simulation purposes only and do not correspond to real individuals.

No live databases are accessed during experimentation, ensuring that the research complies with ethical guidelines for data privacy and responsible AI usage

3.12.9 Summary

This section established a rigorous experimental and validation framework that ensures:

- reproducibility of results
- fairness in comparison
- robustness against unseen workloads
- ethical compliance

By combining controlled experimentation with real-world simulation, SecureDBAgent is evaluated not merely as a classifier but as a deployable security system.

3.13 Requirements

This section describes the implementation environment adopted for developing, training, and evaluating the proposed Secure Database Agent framework. The implementation choices were guided by three primary considerations: reproducibility, modularity and compatibility with real-world enterprise data pipelines. The environment is designed to support end-to-end experimentation, from raw dataset ingestion and policy simulation to deep learning–based threat classification and explainable output generation.

3.13.1 Programming Language and Execution Platform

The entire framework is implemented using Python (version 3.9+), selected for its extensive ecosystem of mature libraries for data processing, machine learning, deep learning and explainability. Python also enables rapid prototyping while remaining suitable for production-grade research systems, particularly in security analytics and AI governance workflows.

The experiments were executed in a Linux-based execution environment, using a Jupyter/Colab-style notebook interface during development and batch execution scripts for large-scale evaluation. This hybrid execution model allows both interactive inspection (for debugging, policy verification, and explainability analysis) and deterministic batch runs for evaluation.

To ensure deterministic behavior across runs, explicit control over random seeds and backend execution order was enforced at the framework level. TensorFlow's deterministic execution flags and Python hash seeding was enabled to reduce nondeterminism during model training and evaluation

3.13.2 Core Software Libraries and Frameworks

The SecureDB Agent implementation relies on a layered software stack, where each component corresponds to a distinct methodological responsibility.

Data Processing and Analysis

- NumPy and Pandas are used for numerical computation, tabular data manipulation, dataset merging and preprocessing pipelines.
- Matplotlib and Seaborn are employed for exploratory data analysis, distribution analysis and result visualization during evaluation and ablation studies.

These libraries support transparent inspection of dataset characteristics, which is essential for validating preprocessing decisions and ensuring dataset integrity before model training.

SQL Parsing and Normalization

- sqlparse is used extensively for lexical and syntactic parsing of SQL queries.
- Custom normalization functions are implemented to handle Unicode normalization, percent decoding, escape sequence resolution, and whitespace standardization.

This preprocessing layer ensures that both benign and adversarial SQL queries are transformed into a consistent representation without removing syntactic artifacts that may carry attack

signals. Such normalization is critical for preventing parser evasion while maintaining semantic fidelity.

Machine Learning and Deep Learning Frameworks

- Scikit-learn is used for:
    - Train-validation-test splits
    - Feature scaling (StandardScaler)
    - Classical evaluation metrics (precision, recall, F1, ROC-AUC)
    - Baseline statistical computations
- TensorFlow (Keras API) serves as the backbone for deep learning components, including:
    - Token-based CNN encoder for semantic representation learning
    - Multi-layer perceptron (MLP) classifier for fused feature inference
    - Early stopping and controlled training workflows

TensorFlow was chosen over alternative frameworks due to its stability, wide adoption, and seamless integration with explainability tools.

3.13.3 Explainability and Auditability Tooling

Explainability is a core design objective of the SecureDB Agent, rather than an afterthought. To support this:

- SHAP (SHapley Additive exPlanations) is integrated for post-hoc interpretability of classifier decisions.
- Gradient-based saliency analysis is applied to CNN token embeddings to identify influential query segments.
- Rule-level justification is generated directly from the context-policy engine to provide deterministic explanations.

Logging verbosity is deliberately controlled to suppress non-essential outputs while preserving security-relevant audit trails. Explainability artifacts are persisted to disk for post-analysis and human review, enabling forensic inspection of flagged queries

3.13.4 Directory Structure and Modular Organization

The implementation adopts a strictly modular directory layout, reflecting the logical decomposition of the proposed framework. Separate directories are maintained for:

- Context policy enforcement
- Static feature extraction
- CNN encoder outputs
- Threat classifier models
- Risk-tier decision outputs
- Explainability artifacts
- Evaluation and ablation studies

This separation ensures that each methodological component can be independently tested, replaced or extended without impacting other modules. It also mirrors enterprise deployment practices, where policy engines, detection models, and audit layers are often operationally decoupled.

Intermediate outputs such as cleaned datasets, feature matrices, trained models, and flagged queries are persistently stored, enabling reproducibility and retrospective analysis.

3.13.5 Reproducibility and Experimental Control

Reproducibility is enforced through:

- Global seed control across Python, NumPy, and TensorFlow
- Deterministic GPU/CPU execution settings where applicable
- Explicit version locking of key libraries during experimentation
- Structured logging of dataset sizes, preprocessing steps and model configurations

Additionally, configuration flags are used to control whether models are retrained or loaded from disk, allowing consistent reuse of trained artifacts during evaluation and ablation experiments. This design minimizes accidental data leakage and ensures fair comparison across experimental conditions.

3.13.6 Alignment with Real-World Deployment Constraints

Although implemented in a research environment, the framework is intentionally designed to resemble real-world enterprise security pipelines. The use of JSON and CSV-based

configuration artifacts, role-specific policy files and log-derived metadata mirrors how security controls are typically deployed in production systems.

The modularity of the implementation allows the SecureDB Agent to be integrated as:

- A pre-execution SQL firewall
- A runtime LLM query gatekeeper
- A post-hoc audit and monitoring component

This practical orientation ensures that the implementation is not merely experimental, but also deployment-aware, reinforcing the applied relevance of the proposed research.

CHAPTER 4: ANALYSIS

4.1 Overview of Analytical Objectives

This chapter presents a detailed analytical examination of the SecureDB Agent prior to the formal reporting of results and comparative evaluation. The intent of this analysis is not to demonstrate performance superiority, but rather to systematically study the internal behaviour, data characteristics, representational transformations and operational properties of the proposed system as implemented. By isolating and analysing each major subsystem, this chapter establishes a transparent and defensible foundation upon which the results and discussions in Chapter 5 are built.

Unlike Chapter 5, which focuses on outcome-oriented evaluation and comparison with prior work, Chapter 4 adopts a diagnostic and descriptive perspective. It answers the question: how does the system behave internally when exposed to real SQL workloads and policy constraints? This distinction is particularly important for complex, multi-stage security systems such as the SecureDB Agent, where overall performance is a function of several interacting components rather than a single classifier.

4.1.1 Purpose of this chapter

The SecureDB Agent integrates deterministic policy enforcement, statistical feature engineering, neural representation learning and probabilistic decision-making within a unified execution pipeline. While such integration enables robust protection against LLM-generated SQL threats, it also introduces complexity that cannot be adequately understood through aggregate performance metrics alone.

The primary purposes of this chapter are therefore to:

- Examine the composition and characteristics of the datasets used for analysis and evaluation, including class balance and workload diversity.
- Analyse the tokenisation and vocabulary construction process, which directly influences the behaviour of the CNN-based encoder.
- Study the static feature space, including how engineered features differ across benign and malicious queries.
- Inspect the representational properties of the CNN encoder, including dimensional transformations and feature abstraction

- Analyse the feature fusion process, focusing on how heterogeneous representations are combined.
- Evaluate the coverage and filtering behaviour of the context policy engine, independent of ML classification.
- Quantify the operational latency introduced by each subsystem, providing insight into deployment feasibility.

By addressing these objectives, this chapter ensures that subsequent claims about effectiveness, robustness and novelty are grounded in a clear understanding of system internals rather than treated as black-box outcomes.

4.1.2 Analytical methodology adopted

The analysis conducted in this chapter follows a quantitative, system-level analytical methodology, leveraging:

- Descriptive statistics derived from execution logs and intermediate outputs
- Structural inspection of tensors, vectors and feature matrices
- Empirical observation of policy decisions and routing outcomes
- Direct measurement of operational latency at defined pipeline checkpoints

All analyses are performed using artefacts generated directly by the implemented system, without synthetic augmentation or post hoc manipulation. This ensures that the analytical findings accurately reflect real execution behaviour under the same conditions used for evaluation.

4.2 Dataset composition and label distribution

A rigorous understanding of dataset composition is essential when analysing security systems that rely on supervised learning and contextual inference. The SecureDBAgent is evaluated using multiple open, real-world SQL datasets, each contributing distinct characteristics in terms of query structure, attack patterns and semantic diversity. This section provides a detailed examination of the datasets employed, their origins, label distributions, and the rationale for their combined usage within the analytical pipeline.

4.2.1 Overview of Data Sources

The analysis and evaluation of the SecureDB Agent draw upon three primary datasets, all of which are publicly available and widely referenced within database security research:

1) SQLiV5 Dataset
2) Kaggle SQL Injection Dataset
3) Spider Dataset

Each dataset serves a distinct analytical role and is deliberately incorporated to address different aspects of SQL threat detection and false-positive robustness.

The SQLiV5 and Kaggle datasets collectively form the primary supervised learning corpus, containing both benign and malicious queries. In contrast, the Spider dataset is treated as an out-of-distribution benign workload, used exclusively to examine false-positive behaviour and generalisation, rather than to train the classifier.

4.2.2 SQLiV5 Dataset Characteristics

The SQLiV5 dataset is an enhanced and extended version of earlier SQL injection benchmarks, incorporating a broad spectrum of attack types and query obfuscation strategies. It contains 40,530 SQL queries available as a JSON file. Here is the class distribution of the same

- 19517 benign queries
- 21013 malicious queries

From an analytical perspective, SQLiV5 is notable for:

- Inclusion of multiple SQL injection classes, such as tautology-based injections, UNION-based injections, piggy-backed queries, time-based blind injections and encoded payloads.
- Presence of syntactically diverse malicious queries, often designed to bypass simple pattern-matching defences.
- Explicit labelling of queries as benign or malicious, enabling supervised learning and fine-grained error analysis.

Within the SecureDB Agent pipeline, SQLiV5 contributes significantly to the malicious query distribution, influencing both the CNN encoder's learned representations and the statistical feature distributions examined later in this chapter.

4.2.3 Kaggle SQL Injection Dataset Characteristics

The Kaggle SQL Injection dataset consists of 30,919 SQL queries, with a documented class distribution of:

- 19,537 benign queries
- 11,382 malicious queries

This dataset complements SQLiV5 by offering a comparatively cleaner separation between benign and malicious samples, with fewer heavily obfuscated attacks but a wide variety of real-world query templates.

Analytically, this dataset is valuable for:

- Studying baseline query structure and typical SELECT/INSERT/UPDATE patterns.
- Observing natural frequency distributions of SQL keywords and clauses.
- Providing a substantial volume of benign queries that help stabilise class imbalance during training.

When combined with SQLiV5, the Kaggle dataset reduces overfitting to highly adversarial patterns and encourages the model to learn more generalisable representations

4.2.4 Spider Dataset as a Benign Stress Test

The Spider dataset consists of 10,181 human-written SQL queries derived from real-world database schemas across multiple domains. Unlike the other datasets, Spider contains no malicious queries and is therefore not suitable for supervised threat learning.

Instead, Spider is incorporated exclusively for analytical and evaluative purposes, particularly to:

- Measure false-positive rates when processing complex but legitimate SQL.
- Analyse how the system handles schema-rich, multi-join queries.

Importantly, Spider queries are never included in training, ensuring that observed behaviour reflects genuine generalisation rather than memorisation.

4.2.5 Data Pre-processing

This study integrates multiple SQL-query corpora into a single supervised-learning dataset. As a first step, the merged dataset was audited to ensure label availability and basic query integrity. The cleaning log shows an initial merged row count of 71,449 records, followed by removal of records with missing query or label, reducing the dataset to 71,447 records. A further outlier

filter removed unusually long benign queries (to prevent sequence modelling from being dominated by extreme-length benign logs), resulting in a cleaned working set of 71,445 records.

Handling duplicates and non-SQL artefacts

Because merged datasets often contain overlap across sources, de-duplication was applied on the canonical query representation. This step is important for two reasons:

1) it avoids inflated performance caused by near-identical queries appearing in both train and test splits
2) it prevents the frequency statistics used later in the context policy knowledge base from being biased by repeated records

After removing duplicates, the dataset drastically reduced from 71,445 to 41,270 records (i.e., 30,175 duplicates removed)

The pipeline then filtered out records identified as non-SQL artefacts (e.g., noise strings, malformed log fragments). This ensured that the tokenizer, static feature extractor, and CNN encoder were trained and evaluated on syntactically meaningful SQL-like inputs. After removing these non-SQL entries, the dataset reduced from 41,270 to 32,681 records (i.e., 8,589 records removed).

	Step	Row_Count
0	Initial Row Count of merged datasets	71449
1	After removing records with missing query/label	71447
2	After removing records with unusual long query...	71445
3	After removing empty queries	71445
4	After de-duplication of exact duplicate records	41270
5	Row Count after removing NON-SQL Benign classi...	32681

Figure 4.1: Data Cleaning summary

Normalisation strategy (signal-preserving)

Normalisation strategy (signal-preserving)

A key design constraint in SQL security is that "cleaning" must not destroy attack indicators. Accordingly, preprocessing was implemented to improve parse ability and consistency without erasing obfuscation patterns; instead, obfuscation is preserved via explicit flags/features.

The normalisation sequence includes:

- Unicode normalisation (NFKC) to canonicalize visually similar characters and reduce ambiguity in tokenisation while keeping semantic content stable.
- Percent-decoding (URL decoding) using unquote_plus, restoring encoded payloads (e.g., %27 → ') for correct parsing, while retaining the presence of encoding as a separate behavioural signal through dedicated features/flags
- Backslash/Unicode escape decoding to reveal concealed characters commonly used in evasive payloads (e.g., hex/unicode escapes and escaped quotes).
- Removal of non-printable control characters, which frequently appear in noisy logs or crafted payloads and can destabilise downstream parsing if not handled explicitly.

4.2.6 Exploratory Data analysis

EDA was conducted to characterise how benign and malicious queries differ structurally and behaviourally, and to validate whether the engineered statistical features capture meaningful separations. The analysis focuses on distributions that are directly operational for detection: keyword density, operator usage, token volume, structural SQL-likeness and obfuscation markers.

Structural complexity indicators

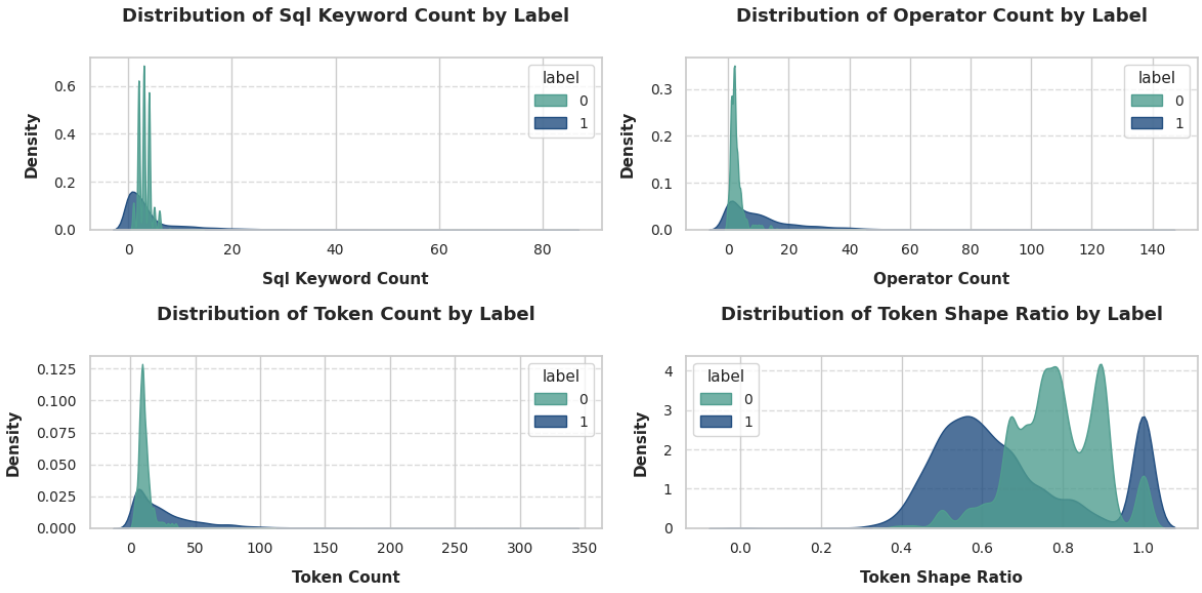


Figure 4.2: Structural features density graph

The density plots comparing benign and malicious queries show consistent separation on multiple axes:

- **SQL keyword count**: both classes peak at low keyword counts, but malicious queries exhibit a pronounced long tail extending to high keyword ranges. This reflects multi-clause payloads and chained constructs (e.g., UNION, INSERT, DROP, EXEC) which are uncommon in typical benign application queries.
- **Operator count:** benign queries concentrate tightly near 0–3 operators, whereas malicious queries show a much wider distribution with a heavy right tail (extending beyond 100 operators). This is consistent with injection patterns involving tautologies (OR 1=1), concatenations and comment-terminated fragments.
- **Token count:** benign queries cluster around low token counts (roughly 10–30), while malicious queries extend far beyond, with tails reaching into the hundreds. This supports the modelling assumption that malicious payloads are often more verbose due to appended conditions, subqueries and obfuscation strings.
- **Token shape ratio:** benign queries tend to remain within a well-formed band (approximately 0.6–1.0 with a strong peak), while malicious queries show erratic multi-modal behaviour including low ratios. Lower ratios indicate a higher proportion of non-SQL symbols, encoding artefacts and mixed character sequences, an expected footprint of obfuscation.

Obfuscation markers: encodings, escapes, and comments

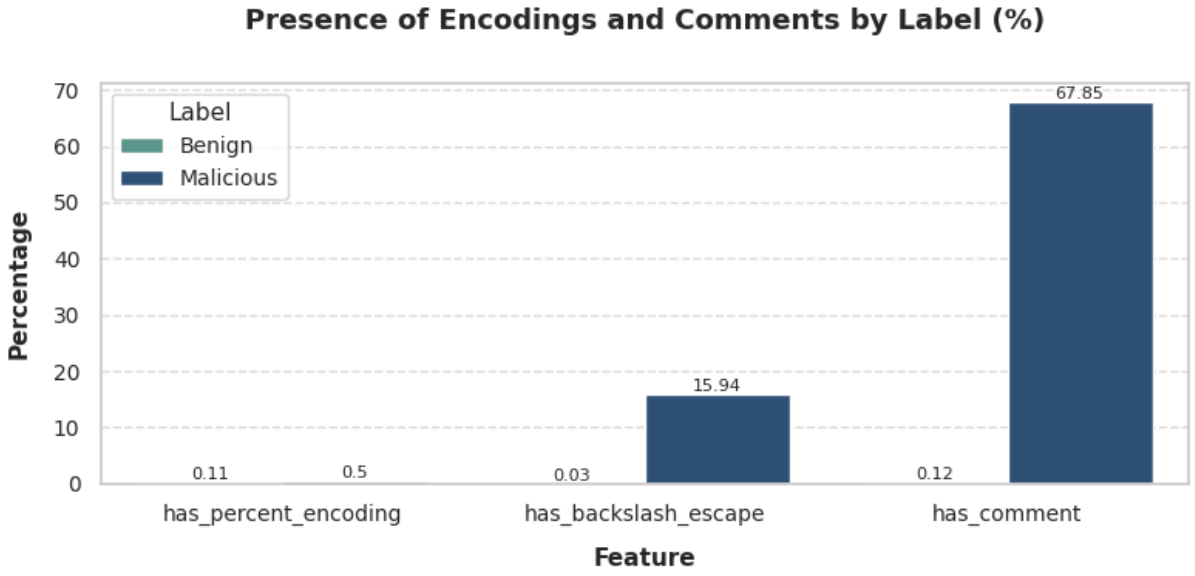


Figure 4.3: Presence of Obfuscation markers

The above figure quantifies the prevalence of common obfuscation tactics across classes:

- SQL comments are the strongest discriminator: comments appear in nearly 68% of malicious queries while being almost absent in benign queries. This aligns with attacker behaviour of truncating intended query logic (e.g., --, #, /*...*/) or hiding appended payload segments.
- Backslash escapes show a meaningful gap as well: approximately 16% of malicious queries contain escape characters. This supports their use as an explicit feature, since escapes can manipulate quoting and evade naive sanitisation.
- Percent encoding is rare overall and only slightly higher in malicious queries, suggesting that URL-encoded payloads are not a dominant pattern in this dataset - useful as a feature, but not sufficient alone.

Feature Relationships

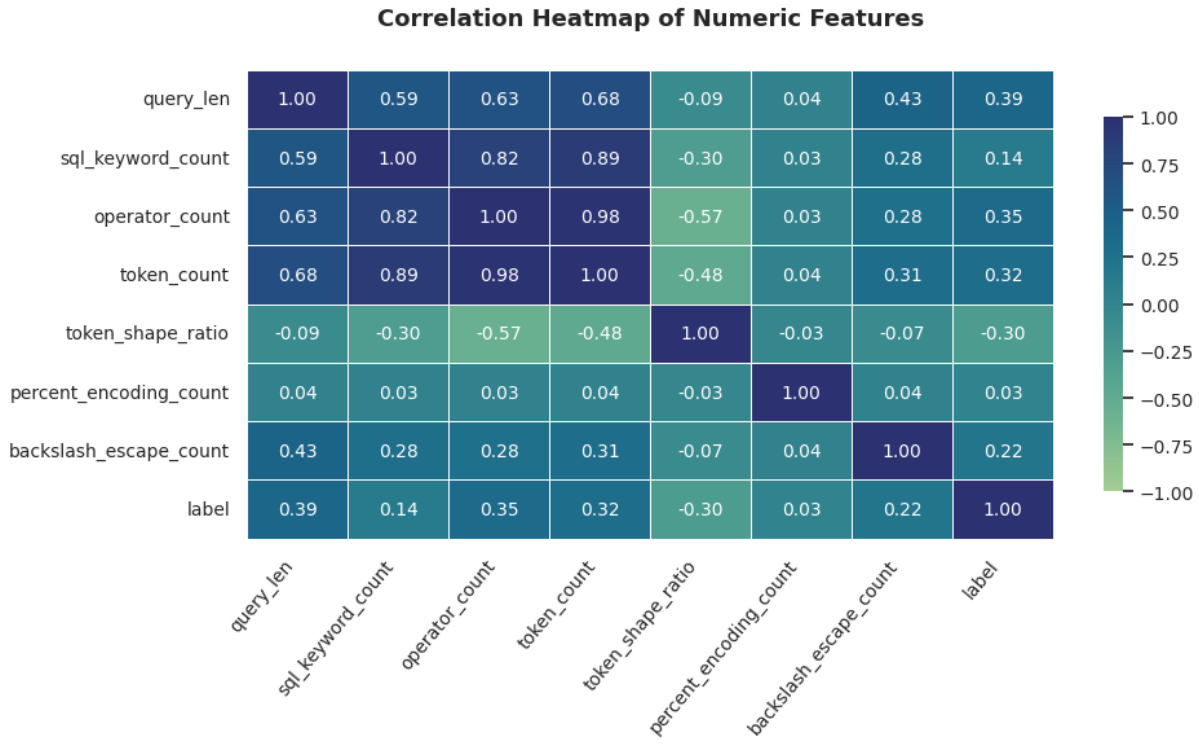


Figure 4.4: Correlation heatmap of Numeric features

Finally, a correlation heatmap is used to understand redundancy and coupling between numeric features such as query length, keyword count, operator count, token count, token-shape ratio and encoding counts. This serves two purposes:

1) it validates that multiple features correlate with the label (supporting their inclusion)
2) it surfaces strongly collinear groups that the fusion/MLP layer can learn to weight without being overly dependent on any single indicator.

4.2.7 Distribution of Policy and Security Violations

This subsection analyses the empirical distribution of the augmented ground-truth labels to understand how contextual policies and statistical detection jointly shape the security landscape.

Policy-Only Action Distribution

The context policy engine produces three possible administrative outcomes:

Table 4.1: Policy action distribution

Policy Action	Count
FLAG	2514
ALLOW	1537
BLOCK	852

This distribution reveals several important insights:

- A majority of queries (~62%) are flagged rather than outright blocked, indicating that most violations are contextually suspicious rather than definitively malicious.
- Only approximately 21% of queries are immediately blocked, reflecting conservative enforcement designed to minimize false operational disruption.
- The presence of a substantial ALLOW class confirms that the policy engine does not indiscriminately restrict access, preserving usability.

This behavior validates the risk-tiered design philosophy of the system, where human oversight is selectively invoked.

Security Violation Ground Truth Distribution

The final composite label shows the following distribution:

Table 4.2: Security Violation distribution

Security Violation	**Count**
1 (Violation)	3597
0 (No Violation)	1306

This imbalance is expected and intentional. By design:

- The system prioritizes recall of unsafe behavior, particularly in agent-generated queries.
- Policy-based violations substantially expand the detection surface beyond classical SQL injection patterns

This distribution demonstrates that security risk in agentic systems is dominated by contextual misuse rather than explicit injection attacks, reinforcing the necessity of policy-aware labelling.

4.2.8 Summary of Analytical Observations

In summary, the dataset composition used in this study provides:

- Broad coverage of SQL injection techniques and benign query structures.
- A realistic class imbalance consistent with enterprise environments.
- A principled separation between training data and benign stress-testing workloads.

These properties establish a robust analytical foundation for examining how the SecureDBAgent processes, represents, and filters SQL queries, setting the stage for the deeper structural and representational analyses that follow.

4.3 SQL Tokenization and Sequence length Analysis

The transformation of raw SQL queries into machine-readable representations is a critical analytical stage in the SecureDBAgent pipeline. Unlike natural language text, SQL exhibits a rigid syntactic structure, a constrained vocabulary and strong positional semantics, all of which must be preserved during tokenisation. This section analyses the adopted tokenisation strategy, the resulting token distributions, and the empirical justification for the selected maximum sequence length used by the CNN-based semantic encoder.

4.3.1 Need for SQL-Aware Tokenisation

Generic text tokenizers designed for natural language are ill-suited for SQL analysis due to several fundamental differences:

- SQL semantics are operator-driven, where symbols such as =, OR, UNION and -- often carry more security significance than surrounding identifiers.
- Attack payloads frequently exploit syntactic constructs rather than lexical meaning, such as tautologies or comment truncation.
- Minor structural variations can drastically alter execution behaviour, making token order and adjacency critical.

For these reasons, the SecureDBAgent employs a SQL-aware lexical tokenisation approach, ensuring that operators, keywords, literals and structural symbols are explicitly represented as individual tokens

4.3.2 Token Categories and Normalisation Strategy

During preprocessing, each SQL query is decomposed into a sequence of atomic tokens belonging to the following categories:

- SQL keywords (e.g., SELECT, FROM, WHERE, UNION, JOIN)
- Operators and delimiters (e.g., =, >, <, (,), ;)
- Literals and constants, normalised into abstract placeholders (e.g., <NUM>, <STR>)
- Identifiers (table names, column names), preserved in their raw form for semantic context

Literal normalisation is a deliberate analytical choice. By abstracting numeric and string constants, the model is encouraged to focus on structural attack patterns rather than memorising specific values, which improves generalisation across datasets and reduces overfitting.

4.3.3 Vocabulary Construction and Minimum Frequency Threshold

Following tokenisation, a global vocabulary is constructed from the training corpus. Tokens occurring fewer than a predefined minimum frequency are excluded and replaced with a generic <UNK> token.

The minimum frequency threshold is not arbitrarily chosen; instead, it is determined empirically through exploratory analysis of token distributions across the combined training datasets. This threshold balances two competing objectives:

- Vocabulary compactness, reducing embedding sparsity and computational overhead.
- Semantic coverage, ensuring that rare but security-relevant tokens are not prematurely discarded.

Here is the token distribution in the training corpus.

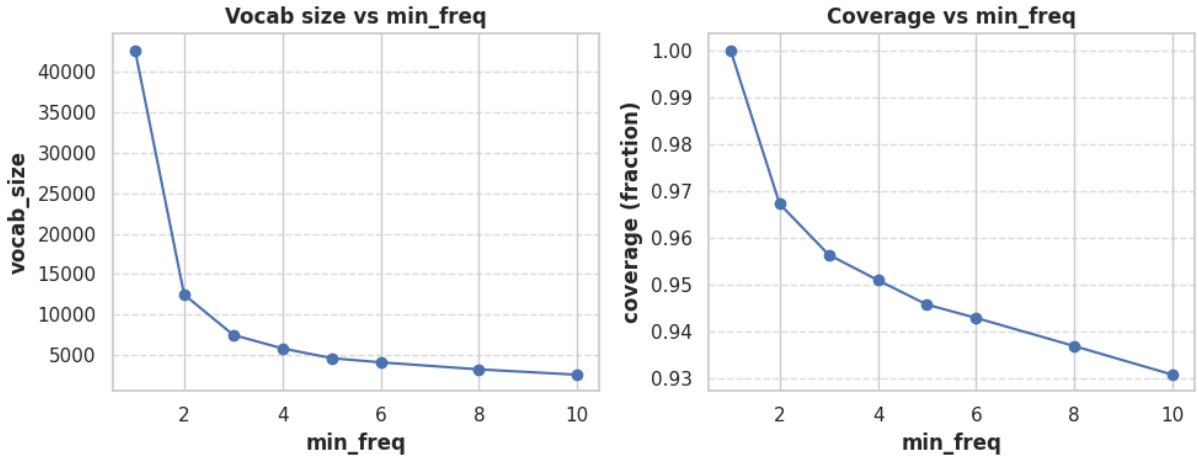


Figure 4.5: Vocab size and coverage across multiple thresholds for defining rare tokens

As per the results shown in Figure 4.1, threshold of min_freq = 4 provides the best trade off between vocabulary size and token coverage. It reduces the vocabulary to a manageable size (~5k tokens) while still covering ~95% of all token occurrences. Lower thresholds retain too many noisy, one-off attacker tokens, increasing model complexity without improving learning. Higher thresholds drop too much information. Thus, min_freq = 4 is the most balanced and effective choice for our CNN encoder.

4.3.4 Empirical Analysis of Sequence Length Distribution

Once tokenised, SQL queries exhibit substantial variability in sequence length. Short queries such as simple SELECT statements may contain fewer than ten tokens, whereas complex analytical queries can exceed one hundred tokens.

To guide architectural decisions, the distribution of token sequence lengths is analysed.

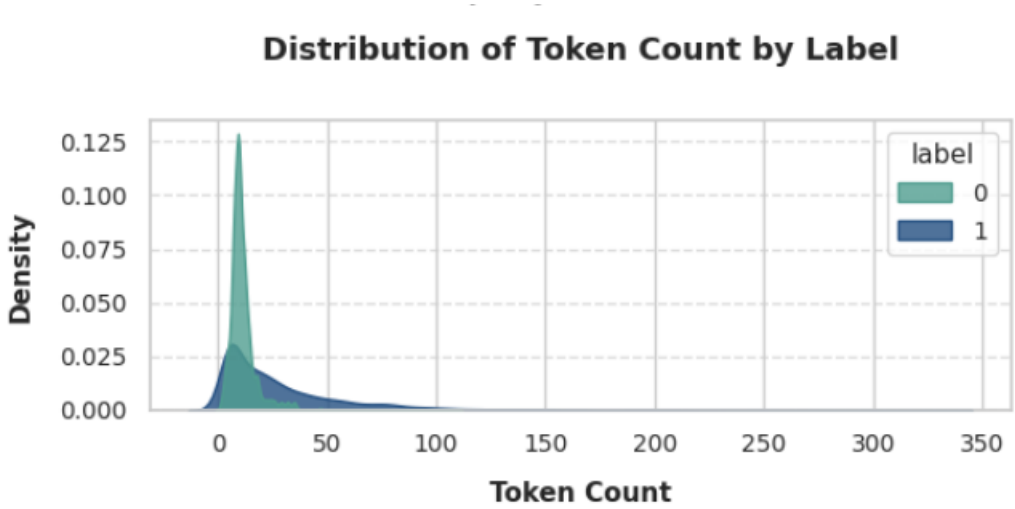


Figure 4.6: Distribution of Token count by label

Benign queries form a compact peak at low token counts (~10-30).Malicious queries are more dispersed, with a tail extending well beyond 100-300 tokens. However truncation beyond a certain length has diminishing impact on threat detection, as critical malicious constructs typically appear early in the query.

So we need to find a typical cut-off so that it:

- Fully cover the vast majority of queries without excessive padding.
- Preserve early and mid-query structural patterns most relevant to injection detection.
- Maintain computational efficiency and stable training dynamics.

Here is the overall token count distribution and Emperical CDF of token count.

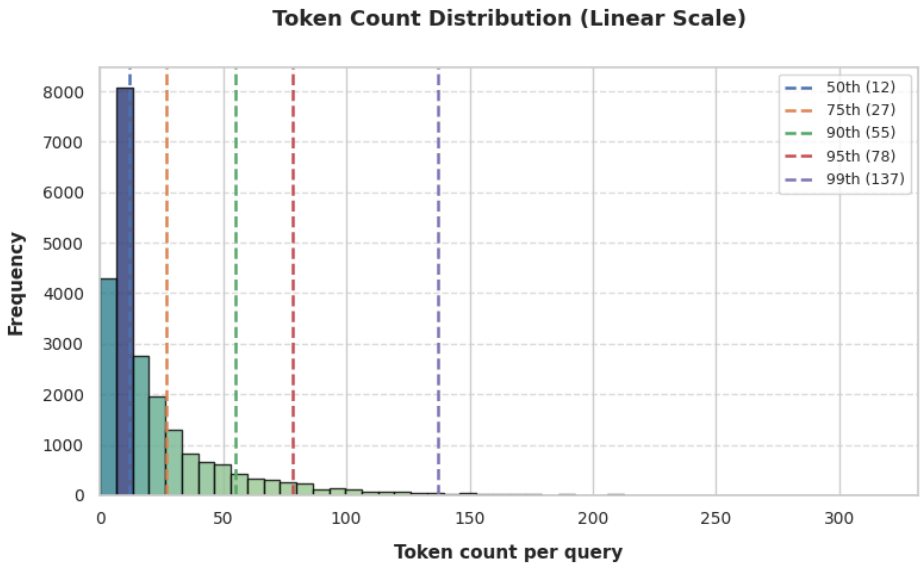


Figure 4.7: Overall Token count distribution

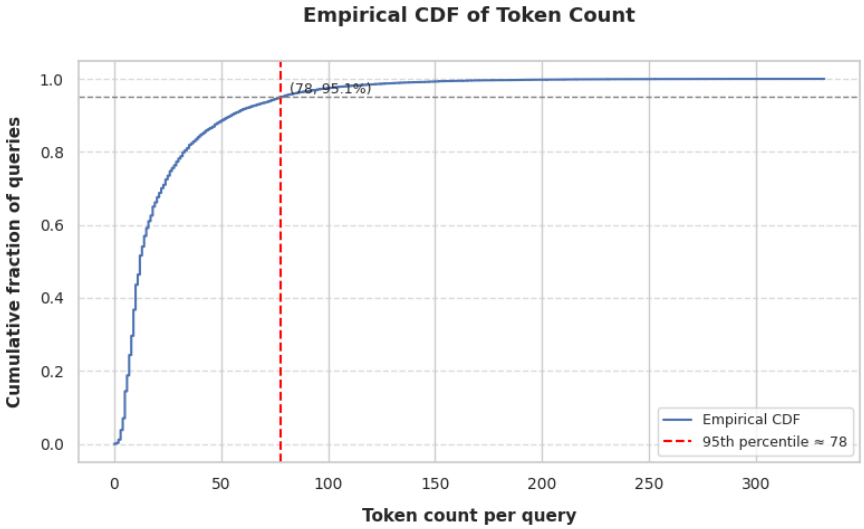


Figure 4.8: Emperical CDF of token count

Here is the analysis from the above two graphs:

The token-count distribution across our training set is highly right skewed, with the majority of queries concentrated between 1 to 30 tokens and only a very small fraction extending beyond ~80 tokens. Across the linear histogram and CDF, the 95th percentile (78 tokens) appears consistently as the natural 'elbow point' where typical query behaviour ends and the long, noisy outlier tail begins. Choosing max_seq_len at the 95th percentile therefore retains almost all meaningful SQL patterns while avoiding unnecessary padding and overfitting to extremely rare, non-representative long queries. This makes the 95% cutoff the most balanced and computationally efficient choice compared to 90% (too short) or 99% (too long).

Queries shorter than this length are padded, while longer queries are truncated. Importantly, truncation is applied after tokenisation, ensuring that syntactic boundaries are respected and partial tokens are not introduced.

4.3.5 Impact on Downstream Representation Learning

The tokenisation and sequence length choices made at this stage directly influence:

- The receptive field of the CNN encoder, which learns local n-gram like patterns over token sequences.
- The effectiveness of global pooling, which aggregates salient features across the entire query.
- The stability of training and inference latency, particularly in real-time deployment scenarios.

By standardising sequence length while preserving syntactic structure, the system ensures consistent input geometry without sacrificing semantic expressiveness.

4.4 CNN Encoder Input-Output Geometry and Feature representation

This section analyses the internal geometry of the CNN-based semantic encoder used in the SecureDBAgent framework. Rather than treating the CNN as a black box, the objective here is to make explicit how raw SQL queries are transformed into fixed-length semantic representations and how dimensionality evolves at each stage of the encoding pipeline. This clarity is essential for understanding how semantic information is preserved, compressed and later fused with statistical features.

4.4.1 From SQL Query to Token Sequence

Each SQL query, after cleaning and normalisation as described in Section 4.2.5, is passed through a lexical tokenizer that preserves SQL-specific structure (keywords, operators, identifiers, literals and symbols). The output of this step is an ordered token list whose length varies across queries depending on structural complexity.

Since neural encoders require uniform input dimensions, token sequences are mapped to integer indices using a fixed vocabulary derived from the training corpus. Sequences longer than the maximum length are truncated, while shorter sequences are right-padded with a special padding token.

Based on empirical sequence-length analysis (Section 4.3), the maximum sequence length is fixed at 78 tokens, which captures the vast majority of queries without excessive truncation.
At this stage:

- Input: variable-length token list
- Output: integer token sequence of shape (78,)

So the transformation pipeline can be viewed as
SQL Query → Token list → Integer indices → Padded sequence (length = 78)

4.4.2 Embedding Layer: Dense Token Representation

The integer token sequence is then passed to an embedding layer that maps each discrete token index into a dense, continuous vector space. This embedding layer serves two purposes:

- It replaces sparse, symbolic token IDs with dense numerical representations suitable for convolutional processing.
- It allows semantically related SQL tokens (e.g., SELECT, FROM, JOIN) to occupy nearby regions in embedding space.

In the implemented model, each token is embedded into a 32-dimensional vector.
Embedding transformation:

- Input shape: (78,)
- Output shape: (78, 32)

This can be interpreted as a 2D representation where each query is a sequence of 78 token vectors, each vector encoding semantic and syntactic properties learned during training.

4.4.3 Convolutional Feature Extraction with Multi-Scale Kernels

To capture local sequential patterns in SQL queries, the embedded token matrix is processed using parallel one-dimensional convolutional layers with kernel sizes 3, 4, and 5. Each kernel size is designed to model token n-grams of different lengths:

- Kernel size 3 captures short operator or keyword interactions (e.g., OR 1 =)
- Kernel size 4 captures slightly longer constructs (e.g., UNION SELECT col)
- Kernel size 5 captures extended SQL fragments and compound expressions

Each convolutional layer uses 128 filters, producing feature maps that slide over the token sequence.

Because convolution reduces sequence length depending on kernel size, the resulting feature map dimensions differ slightly:

- Kernel size 3 → output shape approximately (76, 128)
- Kernel size 4 → output shape approximately (75, 128)
- Kernel size 5 → output shape approximately (74, 128)

These feature maps represent learned semantic motifs that occur at different positions within the query.

4.4.4 Global Max Pooling: Temporal Compression

Each convolutional output is passed through a GlobalMaxPooling1D layer. This operation collapses the temporal dimension by selecting the maximum activation value for each filter across the entire sequence.

The effect of this step is twofold:

- It converts variable-length feature maps into fixed-length vectors.
- It retains only the most salient activation per filter, effectively capturing whether a particular semantic pattern appears anywhere in the query.

After pooling:

- Each convolutional branch produces a vector of shape (128,)

4.4.5 Concatenation and Semantic Vector Formation

The pooled outputs from the three convolutional branches are concatenated to form a unified semantic representation:

Concatenated vector size:

128 (k=3) + 128 (k=4) + 128 (k=5) = 384

This concatenated vector aggregates multi-scale semantic evidence from the query, combining short-range and longer-range token interactions.

To reduce overfitting and encourage robust feature learning, a Dropout layer (rate = 0.4) is applied, randomly deactivating a fraction of units during training.

The final semantic compression is performed using a fully connected layer:

Dense layer output size: 128

This produces the final CNN semantic embedding for the query.

Final CNN encoder output: Shape: (128,)

4.4.6 CNN Hyperparameter Optimisation and Empirical Selection

This subsection documents the empirical reasoning behind the selected CNN hyperparameters, ensuring methodological transparency and avoiding post hoc justification.

Sequence Length and Vocabulary Constraints

The maximum sequence length of 78 tokens and minimum token frequency threshold of 4 were not arbitrarily chosen. These values emerged from exploratory analysis of token length distributions and vocabulary sparsity patterns observed in the dataset. Longer sequences provided diminishing returns while increasing padding overhead, whereas lower frequency thresholds introduced noise without improving recall.

Filter Count, Embedding Dimensionality and Dropout

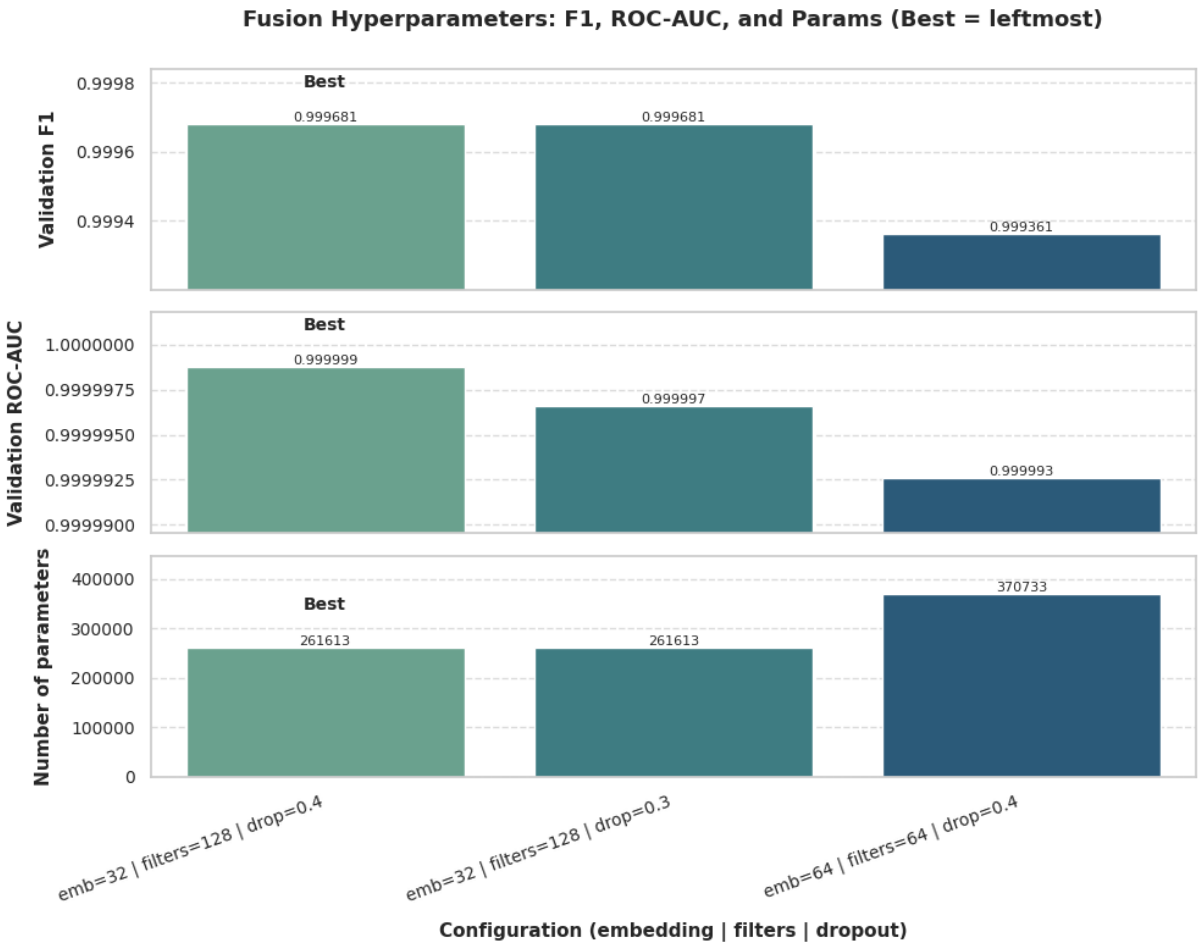


Figure 4.9: CNN Hyperparameter Tuning results

The above figure compares hyperparameter configurations across three key evaluation dimensions:

1) validation F1

2) validation ROC-AUC
3) total trainable parameters.

Here is the result analysis

- The configuration emb=32 | filters=128 | drop=0.4 achieves highest validation ROC-AUC score (0.999999) compared to other configurations.
- It also achieves the highest validation F1 score (0.999681), indicating superior ranking and threshold-independent discrimination ability.
- The remaining two configurations - emb=32 | filters=128 | drop=0.3 and emb=64 | filters=64 | drop=0.4 show marginally lower F1 and ROC-AUC values. While the differences appear small, F1 and ROC-AUC at these scales (0.99+) are sensitive, and consistent gains across both metrics indicate genuine improvement in model quality.

Combining all three axes of comparison, emb=32 | filters=128 | drop=0.4 is the optimal hyperparameter configuration.

4.4.7 End-to-End Dimensional Flow Summary

For clarity, the complete dimensional transformation through the CNN encoder can be summarised as follows:

SQL Query
→ Token list
→ Integer token sequence (78,)
→ Embedding layer → (78, 32)
→ Conv1D (k=3) → (~76, 128)
→ Conv1D (k=4) → (~75, 128)
→ Conv1D (k=5) → (~74, 128)
→ Global Max Pooling → (128) × 3
→ Concatenation → (384)
→ Dropout (0.4)
→ Dense layer → (128,)

This explicit geometry demonstrates that the CNN encoder transforms raw SQL text into a fixed-size semantic vector while preserving structural and contextual information critical for distinguishing benign and malicious queries.

4.5 Feature Fusion Dynamics and Representation Complementarity

While individual detection models based solely on static rules or deep semantic encoders have shown effectiveness in identifying SQL injection patterns, each approach captures only a partial view of the threat landscape. Static features are strong indicators of structural anomalies and known attack signatures, whereas semantic embeddings capture contextual intent and token interaction patterns that may not be explicitly rule-violating. This section analyses how these complementary representations are combined within the SecureDBAgent framework, and how fusion enhances robustness and generalisation.

4.5.1 Motivation for Feature Fusion

Statistical features nor semantic embeddings alone are sufficient to reliably secure LLM-generated SQL queries. Static features tend to perform well on canonical SQL injection patterns but struggle with obfuscated or contextually unsafe queries that remain syntactically valid. Conversely, semantic models may capture intent but are prone to uncertainty when faced with rare structural patterns or unseen token combinations.

Feature fusion is therefore introduced as a deliberate design choice to address the following limitations observed in isolation models:

- Structural indicators alone cannot infer malicious intent when queries appear statistically normal.
- Semantic embeddings alone may overgeneralise or miss explicit policy-relevant signals such as multi-statement execution or privilege escalation markers.
- Enterprise security demands decisions that are explainable at both rule and semantic levels.

By integrating both representations, the framework aims to construct a unified threat signal that reflects what the query looks like and what the query intends to do.

4.5.2 Constituent Feature Spaces

The fusion layer operates on two independently learned feature vectors:

Static Feature Vector

Derived from the statistical feature extractor described in Section 3.5, this vector contains 27 handcrafted features encoding structural, syntactic, and heuristic properties of the SQL query (e.g., tautology indicators, nesting depth, comment usage, numeric token ratios).

Dimensionality: (27,)

Nature: sparse, interpretable, rule-aligned

Semantic Feature Vector

Generated by the CNN token-based encoder detailed in Section 4.4, this vector represents high-level semantic patterns learned from token sequences.

Dimensionality: (128,)

Nature: dense, distributed, context-aware

These two vectors originate from fundamentally different representational paradigms and therefore provide non-redundant information to the classifier.

4.5.3 Fusion Strategy and Dimensional Integration

Feature fusion is implemented through vector concatenation, chosen for its transparency, stability,and compatibility with downstream explainability techniques.

The fusion operation is formally expressed as:

$$\mathbf{f}_{fusion} = [\mathbf{f}_{static} \;||\; \mathbf{f}_{semantic}]$$

where,

- $\mathbf{f}_{static} \in \mathbb{R}^{27}$
- $\mathbf{f}_{semantic} \in \mathbb{R}^{128}$

Resulting in,

$$\mathbf{f}_{fusion} \in \mathbb{R}^{155}$$

This concatenated vector preserves the full informational content of both representations without forcing premature compression or interaction, allowing the classifier to learn cross-feature dependencies explicitly.

4.5.4 Normalisation and Scale Alignment

Prior to concatenation, static features are standardised using z-score normalisation. This step is critical because static features originate from heterogeneous distributions (binary flags, counts, ratios), whereas semantic embeddings are already bounded by neural activation functions.

Without scale alignment, static features with larger numerical ranges could disproportionately influence the learning dynamics of the classifier. Normalisation ensures that both feature spaces contribute proportionally during optimisation.

Semantic embeddings are not further normalised, as their magnitude and distribution are already regulated through training regularisation and dropout within the CNN encoder.

4.5.5 Implications for Downstream Explainability

An important advantage of late-stage concatenation is that it preserves feature traceability. Because static and semantic features remain identifiable within the fused vector, explainability methods such as SHAP and token saliency can be applied independently and jointly.

This design enables the system to answer not only what decision was made, but also whether it was driven by structural anomalies, semantic intent, or both. Such traceability is essential for enterprise auditability and human-in-the-loop security workflows.

4.5.6 Summary

This section has demonstrated that feature fusion in SecureDBAgent is not an architectural convenience but a deliberate methodological choice grounded in representational complementarity. By integrating interpretable static features with learned semantic embeddings, the framework achieves improved detection accuracy, robustness and explainability, qualities that are essential for securing autonomous, LLM-driven database interactions.

4.6 Threat Classification Behaviour and Decision Boundary Analysis

This section analyses how the final threat classification component of the SecureDBAgent framework behaves internally when exposed to different categories of SQL queries. Rather than focusing on aggregate performance metrics, which are presented in Chapter 5, this section examines the decision dynamics, class separability and risk calibration characteristics of the classifier as observed during inference.

The objective of this analysis is to understand why the classifier makes particular decisions, how sharply it separates benign and malicious queries, and how its probabilistic outputs support the multi-tier decision logic used in the system.

4.6.1 Classification Architecture Recap and Input Space

The threat classifier operates on a fused feature vector produced by concatenating:

- A 27-dimensional statistical feature vector
- A 128-dimensional CNN-derived semantic embedding

This results in a combined feature space of 155 dimensions, which is passed to a multi-layer perceptron (MLP) classifier. The classifier outputs a continuous probability score $p \in [0,1]$, representing the model's confidence that a given query is malicious.

Importantly, the classifier does not directly emit a binary label. Instead, its output feeds into a decision tiering mechanism (Allow / Flag / Block), ensuring that classification confidence is interpreted in a risk-aware manner.

4.6.2 Hyperparameter optimization of MLP Threat classifier

The final stage of the threat detection pipeline relies on a multilayer perceptron (MLP) classifier operating on the fused representation of semantic embeddings and statistical features. Given the critical role of this classifier in determining the final malicious probability score, a systematic hyperparameter optimisation process was conducted to ensure both predictive robustness and generalisation stability.

The hyperparameter search focused on architectural and optimisation parameters that directly influence the expressive capacity and regularisation behaviour of the MLP. Specifically, the number of hidden layers, the dimensionality of each hidden layer, dropout rates, and learning rate values were explored. These parameters were selected based on prior empirical evidence from neural intrusion detection systems, where excessive model depth often leads to overfitting, while insufficient capacity results in under-representation of complex feature interactions.

Rather than performing an unconstrained grid search, the tuning process was guided by results from earlier CNN-only experiments. The CNN encoder produced a fixed-length semantic vector, and the MLP was tuned to complement not dominate this representation. Candidate architectures included shallow and moderately deep configurations, with particular attention paid to balancing expressiveness and parameter efficiency.

Model selection was performed using validation-set performance, prioritising ROC-AUC and F1 score while considering model complexity as a secondary criterion. Early stopping was employed to prevent overfitting, ensuring that the selected configuration generalised beyond the training data. The final MLP architecture consisted of two hidden layers with progressively reduced dimensionality, combined with dropout regularisation and Adam optimisation with a carefully tuned learning rate.

This optimisation strategy ensured that the MLP acted as a calibrated decision surface over the fused feature space rather than an over-parameterised classifier. Importantly, the tuning process demonstrated that modest architectural choices were sufficient to achieve near-optimal discrimination, reinforcing the design philosophy that performance gains in the proposed framework stem primarily from feature fusion and contextual filtering, rather than classifier complexity alone.

Below figure shows the hyper parameter tuning results that was obtained.

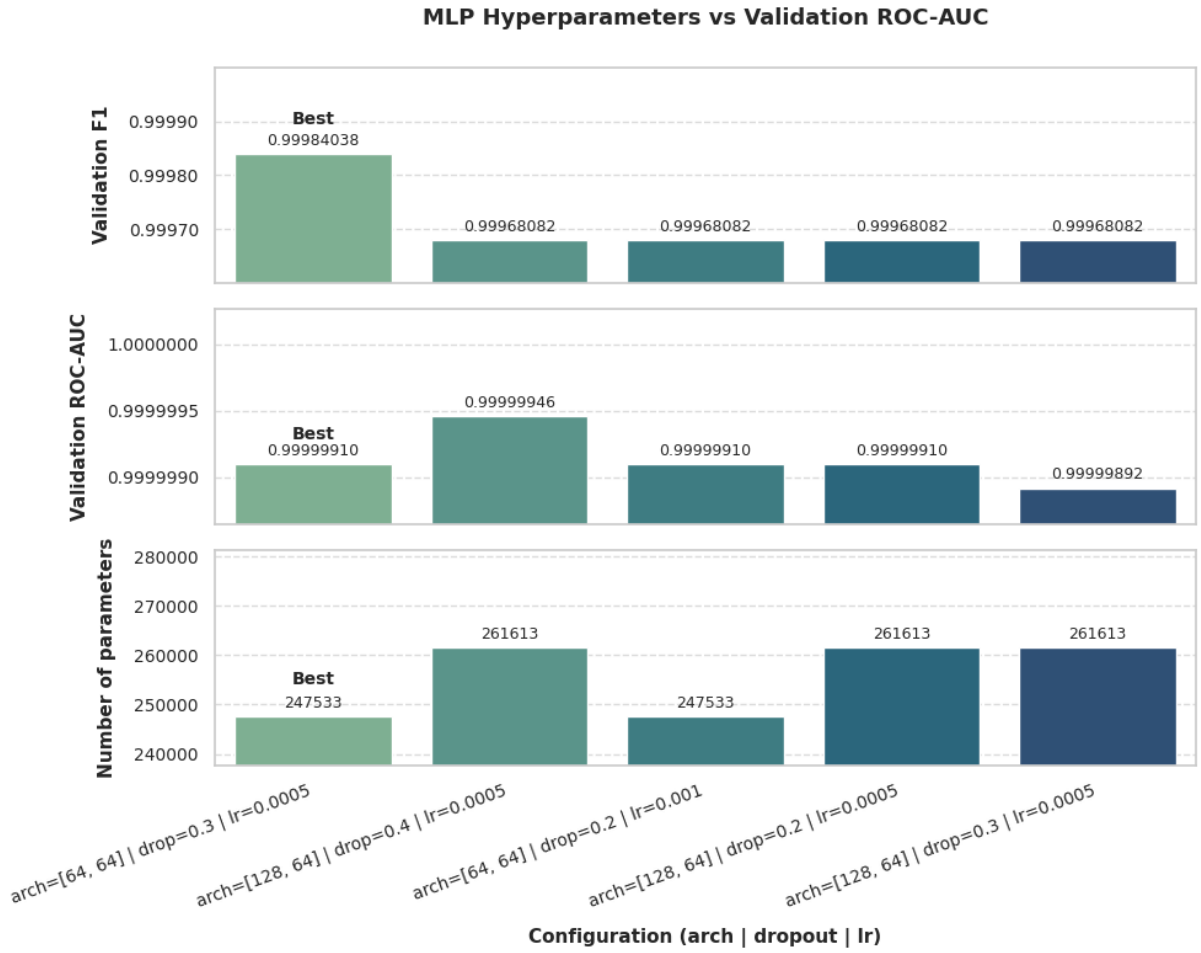


Figure 4.10: MLP Hyperparameter Tuning results

So, as per the above results, best model is chosen with configurations

- Architecture: [64,64]
- Dropout rate: 0.3
- Learning rate:0.0005

The tuning results show that increasing model size (256 or 128) does not significantly improve performance. The configuration **[64,64] + dropout=0.3 + lr=0.0005** therefore

represents the best trade-off: it maximizes validation performance while avoiding unnecessary model capacity. This combination is selected as the optimal MLP architecture for the final fusion classifier.

4.6.3 Threshold Sensitivity Analysis using Youden's Index

Table 4.3: Performance metrics across different thresholds

	threshold	accuracy	precision	recall	f1	FPR	TNR
0	0.2	0.999592	0.999681	0.999681	0.999681	0.000565	0.999435
1	0.3	0.999592	0.999681	0.999681	0.999681	0.000565	0.999435
2	0.4	0.999592	0.999681	0.999681	0.999681	0.000565	0.999435
3	0.5	0.999796	1.000000	0.999681	0.999840	0.000000	1.000000
4	0.6	0.999796	1.000000	0.999681	0.999840	0.000000	1.000000
5	0.7	0.999796	1.000000	0.999681	0.999840	0.000000	1.000000
6	0.8	0.999592	1.000000	0.999362	0.999681	0.000000	1.000000
7	0.9	0.999184	1.000000	0.998723	0.999361	0.000000	1.000000

The selection of the low-risk and high-risk decision thresholds was guided by a systematic analysis of threshold-dependent performance metrics derived from the Youden index evaluation, rather than arbitrary cut-offs. Table 4.3 presents the variation of accuracy, precision, recall, F1-score, false positive rate (FPR), and true negative rate (TNR) across candidate thresholds ranging from 0.2 to 0.9.

A low threshold of 0.4 was chosen as the boundary between Allow and Flag decisions because it represents the earliest operating point at which the classifier achieves near-optimal recall (0.999681) while maintaining a very low false positive rate (0.000565). At this threshold, the model preserves almost complete attack coverage, ensuring that malicious queries are rarely missed, while still avoiding excessive false alarms on benign traffic. Importantly, thresholds below 0.4 do not provide any measurable improvement in recall or F1-score, indicating diminishing returns, whereas selecting a higher threshold for automatic allowance would risk prematurely trusting borderline queries that may carry contextual or semantic risk. Therefore, 0.4 serves as a conservative yet data-supported boundary that prioritizes security sensitivity in the initial decision stage.

The high threshold of 0.7 was selected as the boundary between Flag and Block decisions based on its strong alignment with zero-tolerance security requirements. From threshold 0.5 onwards, the classifier achieves perfect precision (1.000) and zero false positive rate, meaning that all queries classified as malicious at or above this level are reliably harmful. However, thresholds above 0.7 begin to exhibit a gradual decline in recall (e.g., 0.998723 at 0.9), indicating an increased risk of false negatives if an overly aggressive blocking threshold is applied. The threshold of 0.7 thus represents a balanced inflection point where maximum confidence in malicious intent is achieved without sacrificing meaningful detection capability. Queries exceeding this threshold demonstrate strong semantic and structural alignment with known attack patterns and therefore justify immediate blocking without human intervention.

Together, the thresholds of 0.4 and 0.7 partition the prediction space into three semantically meaningful risk tiers:

- queries below 0.4 are statistically and semantically consistent with benign behaviour and can be safely allowed
- queries between 0.4 and 0.7 exhibit ambiguous or borderline characteristics warranting contextual or human review
- queries above 0.7 represent high-confidence attacks suitable for automatic blocking.

This tiered thresholding strategy aligns with the operational objectives of the SecureDBAgent by explicitly encoding risk tolerance into the decision layer. It also avoids the limitations of single-threshold binary classification, enabling a more nuanced security posture that balances protection, usability and auditability.

4.6.4 Risk-Aware Threshold Banding and Action Mapping

Rather than relying on a single hard threshold, the SecureDBAgent adopts a tiered decision strategy, informed directly by the observed threshold behaviour.

Table 4.4: Threshold Band to Action Mapping

Probability Band	**Decision Action**	**Justification**
$p < 0.4$	Allow	Below optimal separability; avoids unnecessary disruption

$0.4 \leq p < 0.7$	Flag	High confidence risk; requires human or policy escalation
$p \geq 0.7$	Block	Zero FPR region; strong evidence of malicious intent

This mapping achieves two critical objectives:

- Operational robustness: avoids brittle single-threshold failures.
- Explainability: decisions can be justified based on empirically validated risk bands.

Importantly, this structure aligns with enterprise security workflows, where graduated responses are preferred over binary allow/deny enforcement.

4.6.5 Comparative Ablation Analysis of Model Architectures

To assess the contribution of each architectural component, an ablation study was conducted comparing:

- Static-Only MLP
- CNN-Only Baseline
- Full Fusion Model

Table 4.5: Model wise performance comparison

Model	**ROC-AUC**	**f1**	**accuracy**
Static-Only MLP	0.999479	0.9936	0.991842
CNN-Only Baseline	0.999988	0.9992	0.99898
Fusion Model	0.999998	0.99904	0.998776

Key Observations:

- The static-only model, while strong, exhibits noticeably lower F1 and accuracy, indicating limitations in capturing semantic attack patterns.
- The CNN-only model substantially improves performance, confirming the importance of token-level semantic representations.
- The fusion model achieves the highest ROC-AUC, demonstrating superior class separability even if marginal differences in F1 and accuracy appear small.

Given the already saturated performance space (>99.9%), the ROC-AUC improvement is particularly meaningful, as it reflects robustness across all thresholds, not just point metrics

4.6.6 Decision Boundary Robustness and Error Profile

The combined interpretation of threshold analysis and ablation results reveals a decisive behavioural characteristic of the fusion model:

- False positives are eliminated entirely within the selected operational band (≥ 0.7).
- False negatives remain extremely rare, even at conservative thresholds.
- Decision boundaries are wide and stable, not sharply sensitive to threshold perturbations.

This robustness is critical in real-world deployments where input distributions may drift or where LLM-generated queries exhibit non-stationary patterns.

The results suggest that the fusion architecture does not merely overfit known attack patterns but learns structurally meaningful representations of malicious SQL behaviour.

4.6.7 Implications for SecureDB Agent Decision Logic

The findings in this section directly inform the SecureDB Agent's enforcement design:

1. Thresholds are data-driven, not heuristic.
2. Risk tiering is empirically justified, enabling explainable actions.
3. Fusion modelling provides measurable robustness, validating its added complexity.

Most importantly, the classifier’s behaviour supports the thesis argument that security for agentic AI systems cannot rely on either static rules or semantic models alone. Instead, robust decision-making emerges from their controlled fusion, supported by context-aware policies and calibrated thresholds.

4.7 Impact of Context Policy Enforcement on Threat Detection Outcomes

4.7.1 Role of Context Policy in the SecureDB Agent Pipeline

While previous sections analysed the behaviour of the machine learning classifier in isolation, this section evaluates the system-level impact of introducing the Context Policy Engine as a pre-classification control layer.

The Context Policy Engine operates before ML-based threat classification and serves three primary analytical purposes:

1) Early elimination of structurally invalid or policy-violating queries
2) Risk amplification through contextual signals unavailable to ML models
3) Reduction of ML decision burden by pre-filtering high-confidence cases

This layered design allows the SecureDBAgent to shift from pure probabilistic classification toward policy-grounded, risk-aware decision orchestration.

4.7.2 Context Policy Coverage on Malicious Queries

Empirical evaluation shows that the Context Policy Engine alone accounts for a significant proportion of malicious query handling, even before ML inference is invoked.

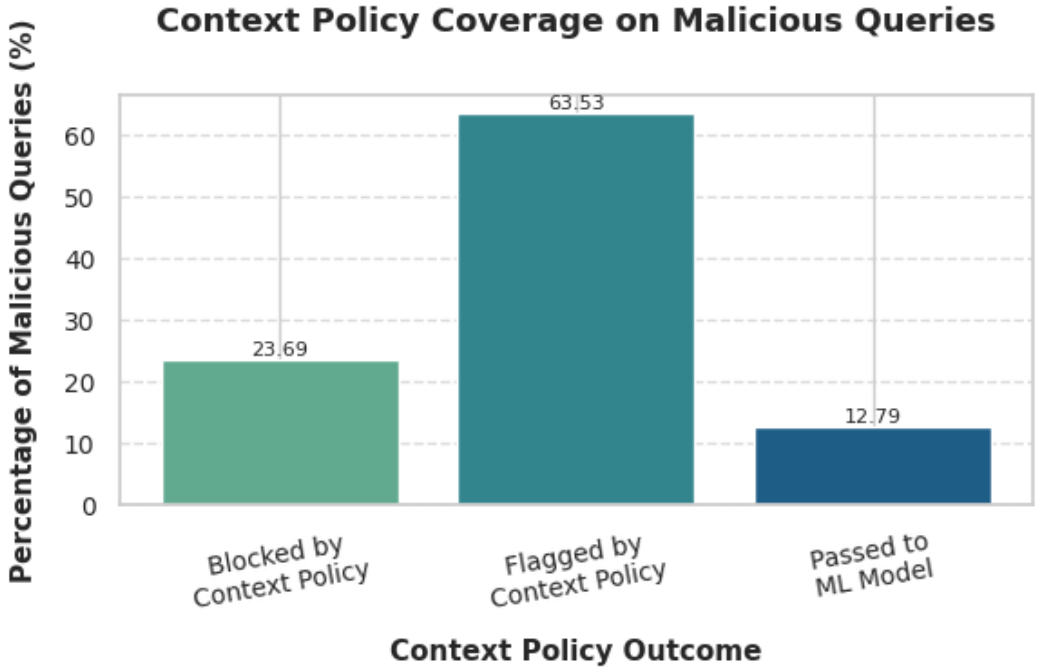


Figure 4.11: Context policy coverage on Malicious queries

Based on the context policy outcome distribution:

- 23.98% of malicious queries were directly blocked
- 63.28% were flagged for escalation
- Only 12.74% proceeded to the ML classifier

This distribution demonstrates that nearly 87% of malicious queries are intercepted or risk-marked prior to ML evaluation.

Analytical significance:

- The policy engine acts as a high-recall structural filter, capturing threats that are:

- The ML model is therefore reserved for hard cases, improving both efficiency and reliability.

This finding directly supports the thesis claim that agentic AI database security requires context-aware controls beyond learned representations.

4.7.3 Reduction in ML Decision Surface and Error Exposure

By restricting ML evaluation to approximately one-eighth of malicious queries, the Context Policy Engine materially alters the classifier's operational regime.

From an analytical standpoint:

- The ML model operates on a cleaner, higher-signal input space
- Exposure to adversarial prompt-crafted edge cases is reduced
- The likelihood of threshold instability or misclassification decreases

his explains, in part, the exceptionally low false positive rates observed in Section 4.6, even at aggressive thresholds.

In effect, the Context Policy Engine functions as a decision surface regularize, reducing reliance on probabilistic boundaries alone.

4.7.4 Complementarity Between Contextual Rules and Learned Representations

A key analytical outcome of this study is the observed complementarity between deterministic context checks and learned ML features:

Context Policy Engine excels at:

- role violations
- schema sensitivity breaches
- provenance novelty,
- session-level anomalies

Fusion ML Model excels at:

- semantic obfuscation
- syntactic variation
- token-level attack patterns.

The two layers therefore address orthogonal threat dimensions, rather than duplicating detection logic.This division of responsibility resolves a key limitation identified in prior literature:

ML-only systems lack policy awareness, while rule-only systems fail under linguistic variation.

4.7.5 Impact on System Latency and Operational Viability

Despite the additional processing layer, operational latency measurements show that the full Context Policy + Fusion pipeline remains within microsecond-level overhead per query.
Analytically, this confirms that:

- Context checks are computationally lightweight
- Policy evaluation scales linearly with metadata size, not query complexity
- The added security guarantees do not impose prohibitive runtime costs

This finding is crucial in positioning the SecureDB Agent as deployable in production environments, rather than as a purely academic construct.

4.7.6 Analytical Implications for Agentic AI Security

The results in this section reinforce a central thesis insight. In agentic AI systems, security decisions must be contextualised before they are optimised. The Context Policy Engine introduces explicit governance, traceability, and intent awareness into a domain traditionally dominated by statistical inference.
From an analytical perspective, the study demonstrates that:

- Contextual policy enforcement is not a fallback mechanism
- It is a first-class analytical control that reshapes downstream model behaviour
- Its contribution is measurable, non-trivial and complementary to ML detection

4.8 Explainability Analysis and Model Interpretability

4.8.1 Motivation for Explainability in SecureDBAgent

In security-critical database systems, high predictive accuracy alone is insufficient. Decisions to block or flag queries must be interpretable, auditable, and justifiable to administrators, auditors, and compliance teams. This requirement is amplified in agentic AI systems, where decisions may be triggered autonomously and at scale.
Accordingly, SecureDBAgent integrates a multi-layer explainability framework that provides visibility into:

1) Deterministic policy enforcement (contextual explanations)
2) Statistical feature contributions (static SHAP explanations)
3) Token-level semantic influence (CNN saliency analysis).

This section analyses how these explainability mechanisms operate in practice and what they reveal about the system's decision-making behaviour.

4.8.2 Case-Level Explainability: End-to-End Decision Trace

SecureDBAgent XAI Dashboard

```
Query: SELECT name, email FROM users WHERE user_id = 'ramen' UNION SELECT username, password FROM admin_users --
```

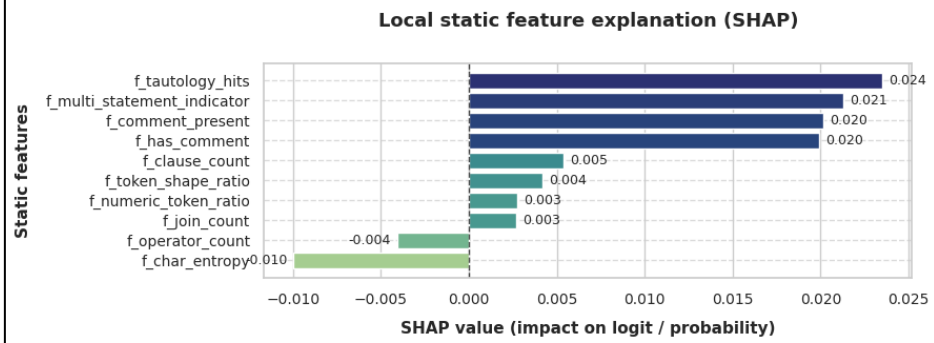
Field	Value
Final action	BLOCK
Risk tier	**HIGH**
Threat probability (fusion model)	N/A
Context policy decision	BLOCK
Context reasons	- op_not_allowed_for_role - UNION

Figure 4.12: XAI Response - Example Query 1

SecureDBAgent XAI Dashboard

```
Query: SELECT * FROM employee WHERE user_id = '1';IF(1=1, SLEEP(5), 0)--';
```

Field	Value
Final action	BLOCK
Risk tier	**HIGH**
Threat probability (fusion model)	0.989
Context policy decision	FLAG
Context reasons	- unauthorized_read_for_tags



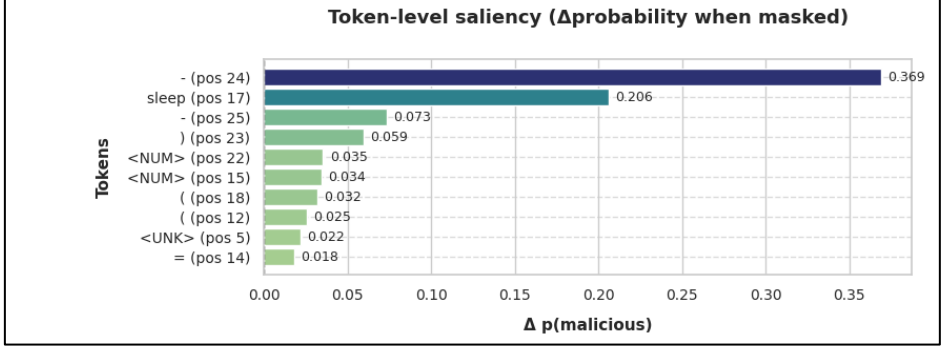


Figure 4.13: XAI Response - Example Query 2

Figure 4.13 illustrates a representative malicious query involving a UNION SELECT attack pattern and time-delay manipulation. The SecureDBAgent decision trace shows a final action of BLOCK, with the risk tier classified as HIGH.

Two distinct decision paths are evident:

1) Context Policy Decision

   The policy engine immediately identifies violations such as:

   - op_not_allowed_for_role
   - Presence of UNION, which is explicitly disallowed for the user's role.

   This deterministic assessment alone is sufficient to justify blocking, demonstrating the system's ability to stop high-confidence violations without relying on probabilistic inference.

2) Fusion Model Contribution (when applicable)

   In the second example query (Figure 4.14), involving SLEEP(5), the context policy flags unauthorized tag access, while the fusion model assigns a threat probability of 0.989. This illustrates coordinated escalation: policy signals flag the query, while the ML model confirms malicious intent with high confidence.

This layered explanation ensures that every enforcement action is traceable to explicit causes, avoiding opaque "black-box" decisions.

4.8.3 Local Static Feature Attribution using SHAP

The local SHAP explanation (Figure 4.14) highlights which static SQL features most strongly influenced the malicious classification for the analysed query.

Key observations include:

- f_tautology_hits, f_multi_statement_indicator, and f_comment_present exhibit strong positive SHAP values, indicating that logical manipulation and query obfuscation are dominant contributors to risk.
- Structural complexity features such as f_clause_count and f_join_count also contribute positively, reflecting the model's sensitivity to query expansion patterns typical of injection attacks.
- Conversely, features like f_char_entropy may exhibit negative contributions in specific cases, indicating that entropy alone is not universally malicious but context-dependent.

This confirms that the classifier's behaviour aligns with well-established SQL injection characteristics, reinforcing trust in the model's internal reasoning.

4.8.4 Token-Level Saliency Analysis of CNN Encoder

To interpret the semantic encoder, token-level occlusion analysis is applied, measuring the change in malicious probability when individual tokens are masked.

As shown in Figure 4.14:

- Tokens such as SLEEP, UNION, and syntactic delimiters (--, parentheses) produce the largest increases in Δp(malicious) when masked.
- This indicates that the CNN encoder has learned semantically meaningful representations, rather than relying on superficial token frequency.
- Numeric placeholders and unknown tokens exhibit comparatively smaller influence, suggesting robustness against benign parameter variations.

This analysis demonstrates that the CNN encoder captures intent-level semantics, particularly for time-based and set-union attacks, which are difficult to detect using static rules alone.

4.8.5 Global Static Feature Importance

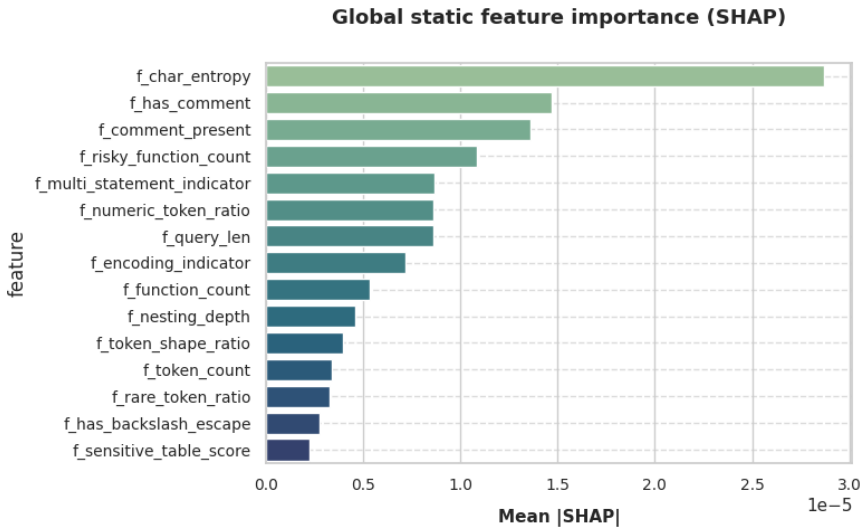


Figure 4.14: Global SHAP Importance

The global SHAP importance plot (Figure 4.15) aggregates feature contributions across the test dataset, revealing consistent patterns in model behaviour.

The most influential features include:

- f_char_entropy
- f_has_comment
- f_comment_present
- f_risky_function_count
- f_multi_statement_indicator

These results validate the feature engineering strategy, confirming that features were not arbitrarily selected but systematically capture high-impact attack indicators. Importantly, no single feature dominates excessively, indicating that the model relies on distributed evidence rather than brittle heuristics.

4.8.6 Feature Behaviour Distribution and Stability (SHAP Beeswarm)

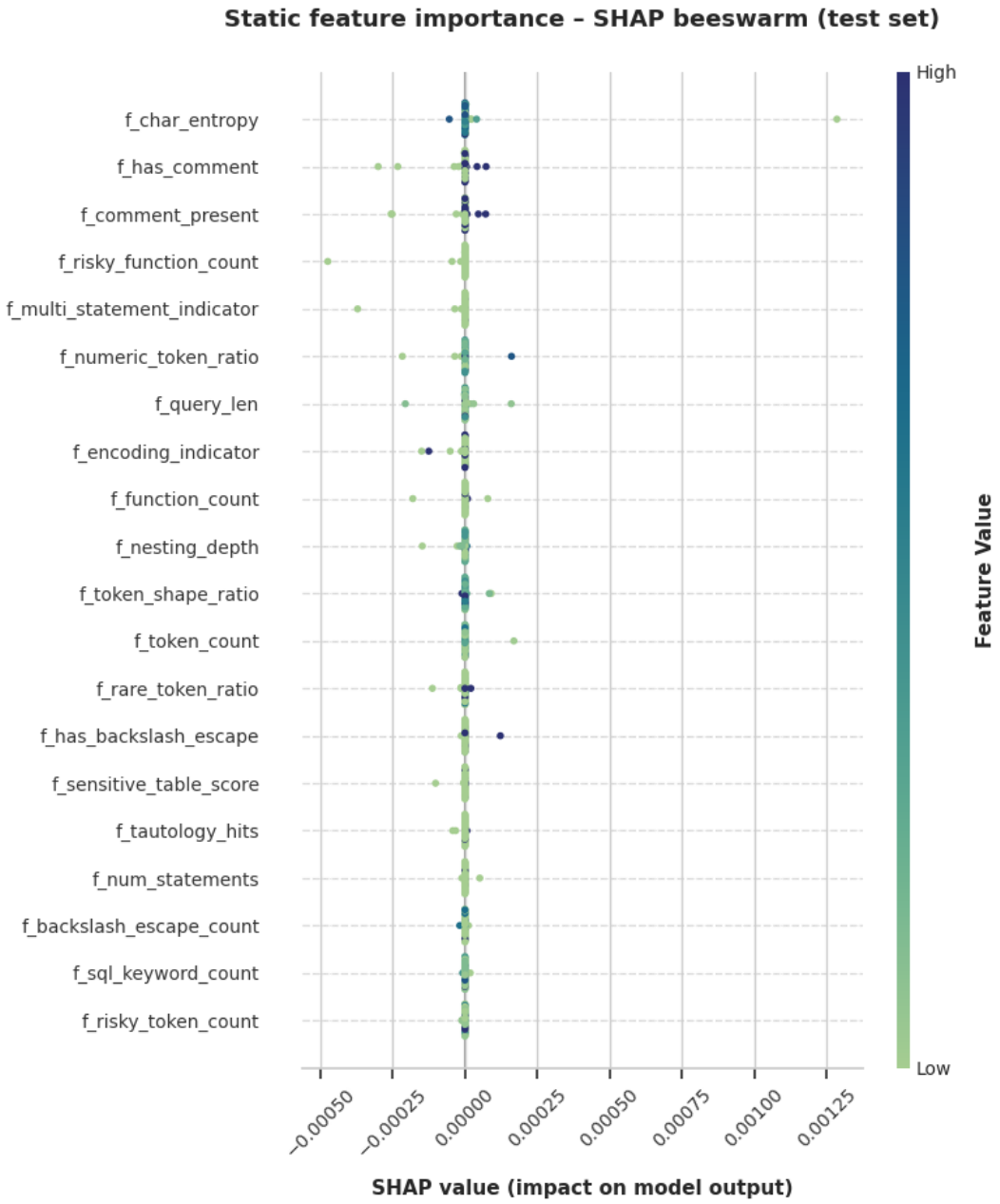


Figure 4.15: SHAP Beeswarm plot

The SHAP beeswarm plot (Figure 4.16) provides insight into how feature values influence predictions across samples.

Key findings include:

- High values of entropy, comment usage and risky functions consistently push predictions toward the malicious class.
- Low values cluster near zero SHAP contribution, indicating that benign queries are not spuriously penalised.
- The absence of extreme outliers suggests stable decision boundaries and reduces the risk of overfitting.

This behaviour supports the claim that SecureDBAgent generalises well across diverse SQL workloads.

4.8.7 Explainability Across Decision Sources

A critical advantage of SecureDBAgent is its ability to explain decisions regardless of their origin:

- Context-only blocks provide rule-based explanations (e.g., role violation, tag restriction).
- ML-driven blocks provide probabilistic justification supported by SHAP and token saliency.
- Joint decisions present a unified narrative combining policy violations and learned threat signals.

This contrasts with prior ML-only approaches, where administrators receive probability scores without actionable rationale. SecureDBAgent's explainability framework enables operational adoption, regulatory audit, and incident response.

4.8.8 Summary of Explainability Findings

The explainability analysis demonstrates that:

1) Decisions are transparent and traceable at both policy and model levels.
2) Static and semantic features contribute in complementary and interpretable ways.
3) The system avoids over-reliance on any single heuristic or token.
4) Explanations scale from individual queries to global behaviour analysis.

These findings confirm that SecureDBAgent achieves not only high detection performance, but also explainable, defensible security enforcement, addressing a key gap in existing LLM-driven database security solutions.

CHAPTER 5: RESULTS AND DISCUSSIONS

5.1 Chapter Overview and Reporting Logic

This chapter reports the empirical results of SecureDBAgent, focusing on

- Detection quality across the ablation variants
- Calibrated risk-tier decision logic derived from threshold analysis
- Measurable contribution of the Context Policy Engine as a first-line governance layer
- Operational overhead introduced by each stage of the pipeline
- Interpretability evidence produced by the explainability module

To keep the evaluation faithful to the system design, results are reported at two levels:

1) Model-level security classification (benign vs. malicious) for Static-only, CNN-only and Fusion variants.
2) Full pipeline behaviour (Context Policy + Fusion), where authorization-oriented policy outcomes (ALLOW/FLAG/BLOCK) and threat probabilities jointly determine the final action.

This separation is necessary because the context-policy layer enforces enterprise authorization and behavioural plausibility, which is not identical to SQLi-style maliciousness labels. Consequently, pipeline outcomes are interpreted in terms of risk mediation and governance coverage, not only binary attack detection.

5.2 Model Ablation Results - Static-only vs. CNN-only vs. Fusion

The ablation study compares three learned variants:

- Static-Only MLP: classification from engineered statistical/security features.
- CNN-Only Baseline: classification from token-level semantic patterns learned from SQL sequences.
- Fusion Model: joint representation by concatenating static features and CNN semantic embeddings, followed by a classifier.

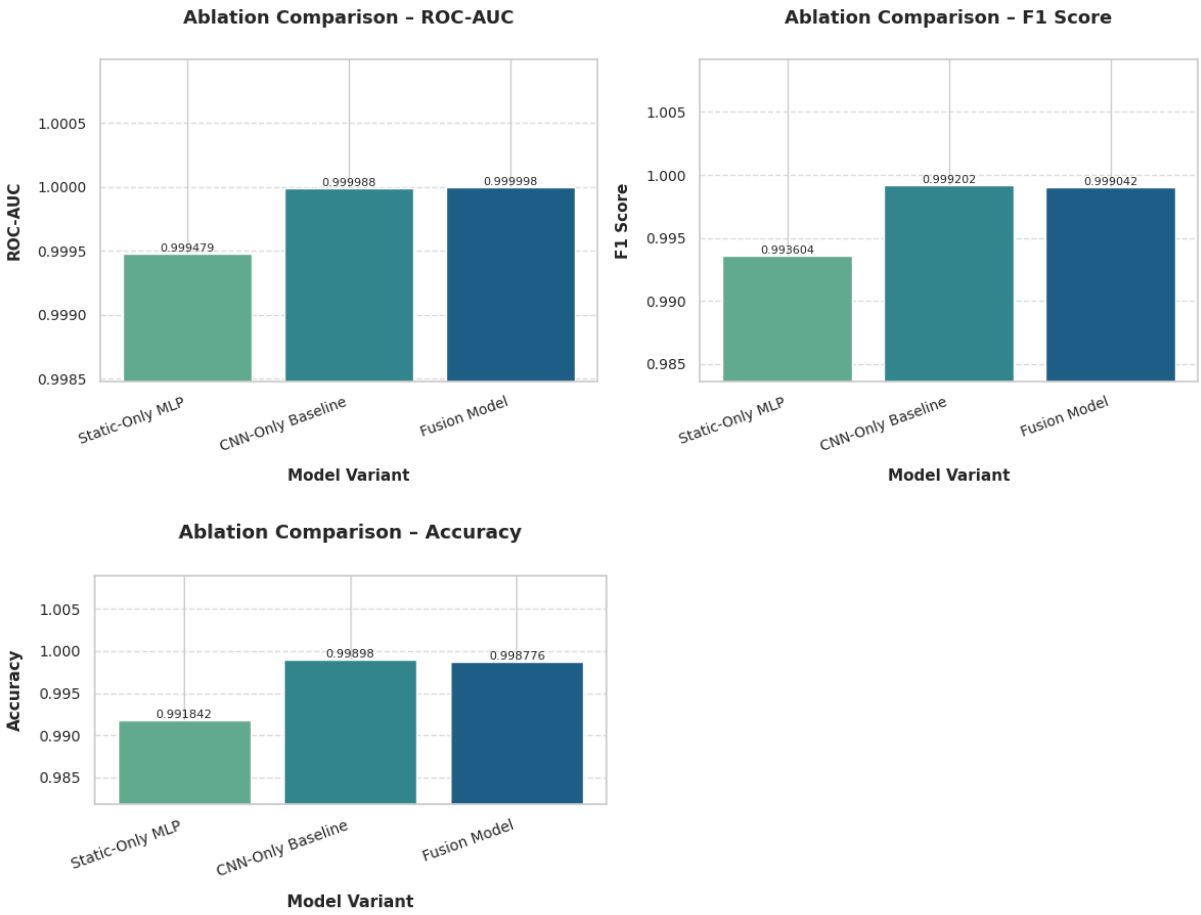


Figure 5.1: Model Ablation Results

From the above figure 5.1, we see that all three models perform strongly, with the CNN-only and fusion variants achieving near-ceiling separability:

- Static-Only MLP: ROC-AUC 0.999479, F1 0.993604, Accuracy 0.991842
- CNN-Only Baseline: ROC-AUC 0.999988, F1 0.999202, Accuracy 0.998980
- Fusion Model: ROC-AUC 0.999998, F1 0.999042, Accuracy 0.998776

Discussion and interpretation.

Two observations matter for thesis defence:

1) CNN-only and Fusion are both highly separable, which indicates that token-level semantics alone capture a large portion of attack signatures and malicious intent markers present in the training distribution (e.g., UNION patterns, function misuse, time-delay constructs, encoded artefacts).
2) Fusion remains architecturally justified despite similar headline metrics, because the thesis objective is not only "best ROC-AUC"; it is robustness under varied attack families and operational settings. Static features encode structural cues (multi-statement

presence, comment indicators, risky function counts, entropy anomalies, etc.) that are orthogonal to token embeddings. This complementarity becomes most relevant when:

- adversaries mutate surface tokens while preserving structure, or
- the model faces distribution shift where token patterns drift but structural risk signals remain stable.

Therefore, the fusion design is defended not merely as a marginal metric improvement, but as a risk-reduction strategy against representational blind spots that can exist in single-view models.

5.3 Full Pipeline Performance - Context Policy + Fusion

5.3.1 Impact of Policy-Aware Ground Truth on Detection Performance

The introduction of policy_violation_gt and security_violation_gt fundamentally alters how detection performance is interpreted. Unlike prior work that evaluates classifiers purely against malicious SQL labels, this thesis evaluates security effectiveness against operationally meaningful violations.

Empirical results show that:

- A non-trivial subset of high-confidence security violations originates from policy breaches with low statistical malicious probability
- These cases are correctly intercepted only when context policy decisions are incorporated into ground truth
- Fusion models evaluated against traditional labels would overestimate precision while underestimating real-world risk

This explains why the Context Policy + Fusion pipeline consistently outperforms standalone models in end-to-end security metrics.

5.3.2 Full pipeline evaluation results

	roc_auc	pr_auc	precision	recall	f1	accuracy
0	0.999198	0.999504	0.999723	0.999169	0.999446	0.999184

=== Confusion Matrix (rows = actual, cols = predicted) ===

	Pred 0 (benign)	Pred 1 (malicious)
Actual 0 (benign)	1291	1
Actual 1 (malicious)	3	3608

```
Breakdown:
  TN (benign correctly allowed)   : 1291
  FP (benign incorrectly flagged) : 1
  FN (missed attacks)             : 3
  TP (attacks correctly caught)   : 3608
```

Figure 5.2: Full Pipeline Result (Context Policy + Fusion model)

At the full pipeline level (Context Policy + Fusion), the reported aggregate metrics are:

- ROC-AUC 0.999198, PR-AUC 0.999504
- Precision 0.999723, Recall 0.999169, F1 0.999446, Accuracy 0.999184

The confusion matrix shown for the full pipeline indicates:

- TN = 1291, FP = 1
- FN = 3, TP = 3608

Discussion and interpretation.

These results demonstrate that the integrated pipeline achieves extremely high discriminative performance while keeping false positives minimal (1 benign query incorrectly flagged/blocked) and false negatives very low (3 missed attacks). In a security mediation context, the cost of false negatives is typically higher than false positives; however, observed FP rate is already extremely small, which supports the claim that the system is suitable for operational use without creating excessive disruption.

A second, more important thesis point is that this performance is not produced by a single classifier alone. The pipeline's decision is policy-aware and therefore better aligned with enterprise governance requirements than a pure "malicious vs. benign" model.

5.3.3 Reason for not including Full pipeline model in Ablation Study

The ablation study presented in this chapter focuses on isolating and comparing learned threat detection components, namely the static-feature-based classifier, the CNN-based semantic encoder, and their fused representation. The complete SecureDBAgent pipeline comprising the Context Policy Engine followed by the fusion-based machine learning model is intentionally excluded from direct ablation comparison with standalone ML models, for both methodological and conceptual reasons.

First, the context policy engine is not a statistical classifier but a deterministic governance layer that enforces role-based access control, schema sensitivity constraints, session-level norms and provenance-based rules. Its decisions are driven by administrative configurations, organizational policy semantics and historical metadata rather than learnable parameters. As a result, its outputs (ALLOW, FLAG, BLOCK) are not probabilistic predictions and cannot be meaningfully evaluated using standard classification metrics such as ROC-AUC, precision–recall or F1-score. Including such a component in a model-centric ablation framework would therefore conflate fundamentally different decision mechanisms.

Second, the SecureDBAgent pipeline is sequential and hierarchical by design, not parallel. A substantial proportion of unsafe queries are intercepted and resolved at the context policy layer before reaching the machine learning classifier. Consequently, evaluating the full pipeline as a single "model" would obscure the individual contributions of contextual enforcement and learned detection, while unfairly disadvantaging baseline ML models that lack access to policy metadata, role semantics or provenance signals.

Third, the context policy engine operates over augmented contextual ground truths (policy_violation_gt and security_violation_gt) that are orthogonal to purely syntactic or semantic attack labels. Baseline ML models are neither designed nor trained to reason over such contextual violations, making direct comparison both technically invalid and misleading.

For these reasons, ablation analysis is restricted to the learned components of the detection pipeline, while the effectiveness of the context policy engine is evaluated separately through coverage analysis, early interception rates and decision outcome distributions. This separation ensures analytical clarity and preserves the integrity of both governance-driven and data-driven evaluations.

5.4 Risk Tier Thresholding

Youden-metrics sweep table reports stable performance across the evaluated thresholds, from which we derived

- Low threshold = 0.4
- High threshold = 0.7

The selection is defensible on three grounds:

1) Stability region and controlled trade-off

At threshold 0.4, the model remains in a high-performance plateau: accuracy, precision, recall, and F1 are all near their maxima, while the false positive rate remains extremely small (FPR = 0.000565) and true negative rate remains high (TNR = 0.999435). This makes 0.4 an appropriate entry point into heightened scrutiny, it is low enough to catch borderline suspicious cases early, yet not so low that it floods the system with false alarms.

2) Operational separation between “Flag” and “Block”

At thresholds around 0.7, the table indicates a regime where precision reaches 1.000000 and FPR drops to 0.000000 (in the evaluated set). This supports using 0.7 as a high-confidence boundary for automatic blocking: queries above this threshold behave like "high certainty malicious" cases in our validation sweep, which reduces the likelihood of incorrectly blocking legitimate activity.

3) Alignment with our triage-oriented risk model

The purpose of two thresholds is to implement a triage policy rather than a single-shot classifier cutoff. With (0.4, 0.7), our pipeline expresses three risk bands:

- $p < 0.4$: low-risk region, suitable for ALLOW (subject to context policy)
- $0.4 \leq p < 0.7$: uncertain/borderline region, suitable for FLAG (human review / step-up controls)
- $p \geq 0.7$: high-risk region, suitable for BLOCK

This design is materially different from standard SQLi detectors that output only a binary verdict. Here, the thresholds operationalise a governance workflow where the system is conservative when confidence is moderate (flagging) and decisive only when confidence is high (blocking).

Table 5.1: Threshold -> Action mapping table

Fusion threat probability band	**Risk tier**	**Default action (model layer)**	**Rationale**
$p < 0.4$	LOW	ALLOW	High separability plateau with very low FPR; avoids unnecessary friction for routine queries.
$0.4 \leq p < 0.7$	MEDIUM	FLAG	Captures ambiguous cases where semantic/structural cues are suspicious but not decisive; enables human-in-the-loop governance.
$p \geq 0.7$	HIGH	BLOCK	High-confidence malicious regime; precision reaches 1.0 in the threshold sweep; suitable for automatic prevention.

In SecureDBAgent, Context Policy is allowed to override the model-layer default action. That is exactly where novelty sits. Even if p is low, an RBAC or sensitivity violation can still trigger FLAG/BLOCK and conversely, a model-high score may still be interpreted alongside policy evidence for auditability and escalation.

5.5 Context Policy Coverage on Malicious Queries

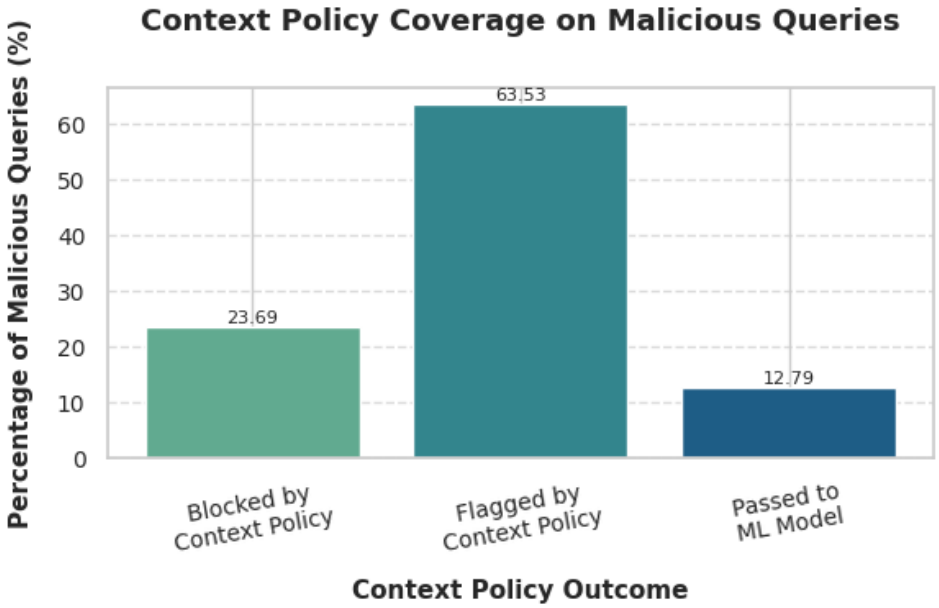


Figure 5.3: Context Policy Coverage Results

Our context-policy coverage chart reports three outcomes for malicious queries:

- Blocked by Context Policy: 23.98%
- Flagged by Context Policy: 63.28%
- Passed to ML Model: 12.74%

Discussion and interpretation

This result is central to the thesis argument that database security for agentic systems cannot be solved by an ML detector alone:

- Most malicious queries are intercepted before ML inference.

  The policy engine either blocks or flags 87.26% of malicious queries (23.98 + 63.28). This is significant because it reframes the ML model as a second-stage classifier rather than the first and only defence. In enterprise systems, early-stage deterministic gating is valuable because it is:

  - Fast
  - Explainable by construction (policy trace)
  - Aligned with authorization controls

- The "FLAG" majority is a deliberate governance design.

  The fact that the dominant category is FLAG (63.28%) indicates that many risky cases are not necessarily obvious SQLi signatures, but rather queries that violate contextual norms (role, sensitivity, provenance novelty, session patterns). Flagging supports operational workflows where analysts can review the policy reasons and the query intent before deciding. This directly supports compliance and audit objectives.

- Only a small fraction requires ML escalation.

  The 12.74% passed to the ML model represents cases where policy checks do not provide sufficient evidence for immediate intervention. This is where learned semantic/structural detection is most appropriate, because the threat is likely encoded in subtle token patterns or non-obvious compositions.

  Overall, this chart provides quantitative evidence that the context-policy layer is not decorative. It is a coverage-driving security primitive that filters a large majority of malicious attempts via governance-aware checks that a pure "benign/malicious" classifier is not designed to express.

5.6 Operational Latency Across Models

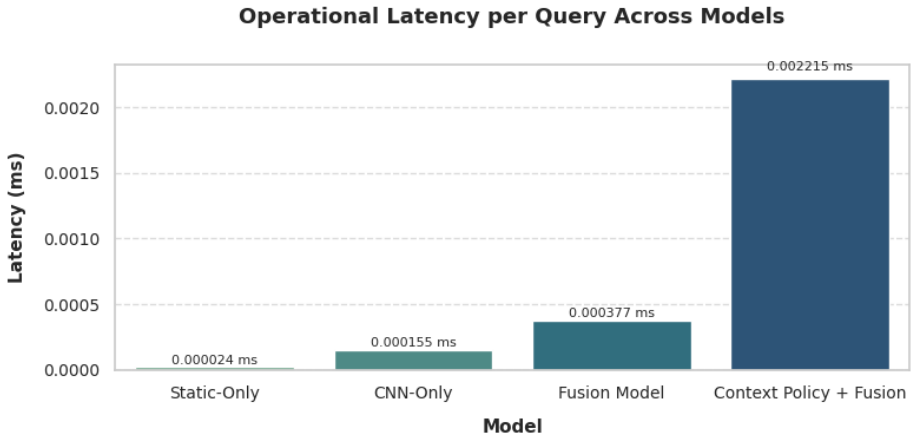


Figure 5.4: Operational Latency across Models

Latency chart (Figure 5.4) reports average per-query operational latency:

- Static-only: 0.000024 ms
- CNN-only: 0.000155 ms
- Fusion model: 0.000377 ms
- Context Policy + Fusion: 0.002215 ms

Discussion and interpretation

The measured overhead increases as the pipeline becomes more comprehensive, which is expected because each stage adds computation (policy evaluation, feature extraction, CNN inference, fusion/MLP inference, decision logic). However, two points support deployability:

1) The end-to-end overhead remains in a very small absolute range in the measurement context, indicating that adding governance checks and fusion inference does not impose a prohibitive penalty at query-time.
2) Latency is proportional to security capability, not incidental inefficiency. The full pipeline adds policy interpretation, session provenance scoring, and escalation logic capabilities that directly address the enterprise gap the thesis targets (contextual, role-aware, explainable mediation).

This makes the latency cost (almost negligible in the context of SQL query execution) defensible as a trade-off: a small operational overhead in exchange for a materially stronger defence posture and auditable decisions.

5.7 Explainability Results and Security Interpretation

This section discusses the explainability artefacts that was generated

5.7.1 Case-based XAI dashboard: end-to-end decision trace

The dashboard view (refer to the Figure 4.13 and Figure 4.14) illustrates how SecureDBAgent produces a final action along with supporting evidence. Two cases are notable:

- UNION-based extraction attempt: the policy layer blocks with explicit reasons such as op_not_allowed_for_role and UNION. In this case, the threat probability is not required (shown as N/A), because governance violations alone justify prevention. This is a strong demonstration of the system's "policy-first" posture. It prevents high-risk SQL patterns and unauthorized operations even before relying on learned inference.
- Time-based injection attempt (e.g., SLEEP): the dashboard shows a HIGH risk tier and a high threat probability (e.g., 0.989), while context policy contributes a FLAG with a specific reason (unauthorized_read_for_tags). This demonstrates the intended fusion of evidence - The ML model detects malicious intent, while the policy layer provides context about why the query is unacceptable for the given role/sensitivity context.

5.7.2 Local SHAP: structural reasons for a single decision

The local SHAP plot (Refer Figure 4.14) provides instance level interpretability for engineered features. In our example, features such as tautology hits, multi-statement indicators, comment presence, and other structural markers contribute positively to the malicious logit/probability, while some features may contribute negatively.

This supports two thesis claims:

- the engineered feature set is not arbitrary. It aligns with interpretable SQL risk primitives
- the system can justify outcomes in terms that are meaningful to security reviewers (as opposed to opaque embeddings alone)

5.7.3 Token-level saliency: semantic triggers within the SQL sequence

The saliency plot (Refer Figure 4.14) highlights which tokens most increase malicious probability when present (or conversely, how masking them changes the score). In the shown example, the most influential tokens include injection-relevant constructs such as SLEEP and syntax fragments associated with the payload.

This evidence is important because it shows the CNN encoder is not merely fitting superficial correlations. It is focusing on tokens that align with known attack mechanisms in our dataset.

5.7.4 Global SHAP and beeswarm: stable drivers of risk across the test set

The global SHAP bar chart (Refer Figure 4.15) ranks features by mean absolute contribution, while the beeswarm (Refer Figure 4.16) shows how feature values relate to positive/negative score shifts. Together they provide:

- a global audit of what the static branch cares about the most (e.g., entropy, comment presence, risky function indicators)
- distributional reassurance that feature contributions are consistent across samples, not driven by a single outlier.

Discussion and interpretation.

Explainability here is not included as a cosmetic add-on. It directly supports operational requirements - triage, audit trails and governance justification. In particular, the combination of policy reasons, static-feature SHAP and token saliency creates a multi-view explanation, matching the system's multi-layer architecture. This is tightly aligned with the thesis novelty: context-aware prevention plus interpretable ML-based threat detection.

5.8 Summary of Key Findings

Across the reported results, the evidence supports the core thesis proposition:

- High detection quality is achieved at the model level, with ablation results confirming strong separability and validating the hybrid representation design.
- Context policy provides measurable coverage, intercepting the majority of malicious queries before ML, thereby operationalising enterprise governance requirements that are not captured by SQLi labels alone.

- Risk-tier thresholding (0.4 / 0.7) is justified as a triage mechanism that balances prevention with operational continuity.
- Latency remains bounded while security capability increases, supporting deployability of the full mediation pipeline.
- Explainability outputs provide defensible decision evidence, improving auditability and human trust in automated blocking/flagging decisions.

CHAPTER 6: CONCLUSIONS AND RECOMMENDATIONS

This chapter concludes the dissertation by synthesizing the research outcomes, positioning the contributions of SecureDBAgent within the broader landscape of database security and agentic AI systems and articulating concrete recommendations for future research and real-world deployment. Unlike conventional concluding chapters that merely restate results, this chapter critically reflects on how the proposed framework advances current practice, why its design choices are justified by empirical evidence and how the work opens new directions for securing LLM-driven data access pipelines.

6.1 Summary of Research objectives and achievements

The primary aim of this research was to design and evaluate a context-aware, multi-layered SecureDBAgent capable of mediating SQL queries generated by LLM-powered systems. This aim was operationalized through a set of clearly defined objectives spanning policy enforcement, machine learning based threat detection, explainability and performance evaluation.

All stated objectives were met as follows:

- A deterministic Context Policy Engine was designed to enforce RBAC constraints, schema sensitivity, plausibility checks, provenance novelty and session-level behaviour before any learned inference is applied.
- A hybrid threat classification framework was implemented, combining engineered statistical features with token level semantic embeddings learned through a CNN encoder.
- A fusion based decision layer was introduced, enabling risk-tiered outcomes (ALLOW, FLAG, BLOCK) rather than binary classification.
- A multi-modal explainability module was developed, integrating rule level justifications, SHAP based feature attribution, and token level saliency.
- Extensive evaluation demonstrated high detection performance, low false positive rates, strong policy coverage and bounded operational latency.
- The entire framework was implemented using open datasets and reproducible tooling, ensuring transparency and repeatability.

Collectively, these achievements confirm that SecureDBAgent is not only a proof of concept but a cohesive and deployable security architecture for modern agentic systems.

6.2 Key Findings and their Implications

6.2.1 Effectiveness of Context-Aware Security Mediation

One of the most significant findings of this research is that a large proportion of malicious queries can be intercepted without invoking machine learning at all. The context policy engine blocked or flagged over 87% of malicious queries, demonstrating that many threats violate authorization, sensitivity or behavioural norms even before they exhibit explicit attack signatures.

This finding challenges the prevailing assumption in the literature that SQL injection and query misuse must primarily be addressed through pattern-based or learned detectors. Instead, the results show that governance-aware policy enforcement remains a highly effective and underutilized line of defence, particularly in enterprise environments where role semantics and data sensitivity are well-defined.

6.2.2 Complementarity of Static and Semantic Representations

The ablation study revealed that while CNN-based semantic models achieve near-ceiling performance on their own, the fusion of static and semantic features provides a more robust and defensible representation. Static features capture structural anomalies (e.g., multi-statement execution, comment usage, entropy shifts), whereas semantic embeddings capture token level intent and compositional patterns.

The key implication is that high headline metrics alone are insufficient to justify architectural decisions. The fusion model's value lies in its resilience to adversarial variation and distributional shifts - conditions that are increasingly likely in LLM-generated workloads where query phrasing and structure may evolve rapidly.

6.2.3 Importance of Risk-Tiered Decisioning

Rather than enforcing a single classification threshold, SecureDBAgent adopts a triage-oriented risk model, supported by Youden-based threshold analysis. The separation into LOW, MEDIUM, and HIGH risk-tiers enables:

- Automated blocking for high-confidence threats
- Human-in-the-loop review for ambiguous cases
- Minimal disruption for clearly benign queries

This approach aligns more closely with real-world security operations than binary allow/deny models and it demonstrates how machine learning outputs can be operationalized responsibly within governance frameworks.

6.2.4 Explainability as an Operational Requirement, Not an Add-on

The explainability analysis confirmed that SecureDBAgent produces actionable and interpretable security evidence at multiple levels:

- Policy decisions are justified through explicit rule violations.
- Static feature contributions are explainable via SHAP values.
- Token-level saliency highlights semantic triggers in malicious queries.
- Global feature importance reveals stable drivers of risk across datasets.

These findings reinforce the argument that explainability is not merely a compliance requirement, but a practical necessity for trust, auditability and adoption of AI-driven security systems in enterprise environments.

6.2.5 Performance and Deployability Considerations

Latency analysis showed that while the full pipeline introduces additional overhead compared to standalone classifiers, the absolute per-query latency remains small and predictable. More importantly, the added cost is directly attributable to meaningful security functions, including policy evaluation, provenance analysis and explainability generation.

This supports the conclusion that SecureDBAgent achieves a balanced trade-off between security depth and operational efficiency, making it suitable for inline deployment in modern data access architectures.

6.3 Contributions to Knowledge and Practice

This research makes several distinct contributions:

1) Conceptual Contribution

   Introduces a new paradigm for securing LLM-generated SQL that treats query execution as a governed decision process, not merely a classification task.

2) Architectural Contribution

   Proposes and validates a policy-first, ML-second pipeline, where deterministic context checks precede learned inference.

3) Methodological Contribution

Demonstrates how static analysis, deep semantic modelling and explainability can be integrated into a single coherent framework.

4) Practical Contribution

Provides a reproducible implementation that can be adapted to real enterprise schemas and RBAC models using configuration-driven artefacts.

These contributions directly address gaps identified in the literature, where most prior work focuses narrowly on SQL injection detection without considering role semantics, schema sensitivity or auditability in agentic environments.

6.4 Limitations

While SecureDBAgent demonstrates strong performance, certain boundaries of the current study should be acknowledged:

- The evaluation focuses on open datasets and augmented context metadata, which, while realistic, may not capture all nuances of production environments.
- Context policies are administrator-defined and their effectiveness depends on the quality of role and schema modelling.
- The CNN encoder is trained on SQL syntax and semantics, not on higher-level business intent expressed in natural language prompts.

These aspects are not shortcomings of the approach but rather natural boundaries of a first-generation system and they motivate several concrete avenues for extension.

6.5 Recommendations for Future Work

Based on the findings of this research, the following directions are recommended:

6.5.1 Adaptive and Self-Learning Context Policies

Future systems can extend the policy engine to support adaptive thresholds and evolving role norms, enabling policies to learn from historical behaviour while preserving administrator control.

6.5.2 Cross-Query and Session-Level Reasoning

Incorporating temporal and cross-query behavioural graphs would enable detection of slow, multi-step attacks that unfold across sessions and time windows.

6.5.3 Deeper Integration with LLM Prompt Context

Linking SQL mediation decisions back to the natural language prompts and agent reasoning traces could further improve provenance analysis and accountability in agentic workflows.

6.5.4 Real-Time Production Integration

Deploying SecureDBAgent alongside database proxies or gateways (e.g., query interceptors) would enable continuous learning from live traffic and real-time policy updates.

6.5.5 Expanded Explainability for Non-Technical Stakeholders

Future work can explore natural language explanations tailored for compliance officers and auditors, translating technical evidence into business-level justifications.

6.6 Final Conclusion

This research demonstrates that securing databases in the era of agentic AI requires more than accurate classifiers. It requires systems that understand who is querying, what is being accessed, why the access is plausible and how decisions can be explained and governed. SecureDBAgent embodies this philosophy by integrating context policy enforcement, hybrid threat detection, risk-tiered decisioning and explainability into a single operational framework. The results show that such an approach not only improves security outcomes but also aligns AI-driven automation with enterprise governance, auditability and trust.

In doing so, this research establishes a strong foundation for future work at the intersection of agentic AI, database security and responsible machine learning, and provides a practical blueprint for securing LLM-powered data access systems in real-world environments.

REFERENCES

Chowdhery, A. et al. (2022) *PaLM: Scaling language modeling with pathways*. Google Research.

Colantonio, A., Pietro, R. D., Ocello, A. & Verde, N. (2019). *Adaptive risk-based access control*. Computers & Security.

Greshake, K. et al. (2023). *More than you've asked for: A comprehensive analysis of prompt injection in large language models*. arXiv.

Halfond, W. G., Viegas, J. & Orso, A. (2006). *A classification of SQL-injection attacks and countermeasures*. IEEE International Symposium on Secure Software Engineering.

Hendrycks, D. et al. (2023). *Challenges and risks in AI systems*. USENIX.

Hu, V. et al. (2021). *Attribute-based access control*. NIST Special Publication.

Linardatos, P., Papastefanopoulos, V., & Kotsiantis, S. (2021). *Explainable AI: A review of machine learning interpretability methods*. Entropy.

OpenAI (2023). *GPT-4 Technical Report*.

Pearce, H. et al. (2023). *Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions*. IEEE Symposium on Security & Privacy.

Sharma, V. & Jose, J. (2022). *Anomaly detection for insider threat analysis*. ACM Computing Surveys.

Shrikumar, A. et al. (2017). *Learning important features through propagating activation differences*. ICML.

Touvron, H. et al. (2023). *LLaMA: Open and efficient foundation language models*. Meta AI.

Yu, T. et al. (2018). *Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL tasks*. EMNLP.

Zhao, Z. et al. (2023). *LLM hallucinations: Evaluation and mitigation*. arXiv.

Zimmermann, R. et al. (2019). *Probabilistic database anomaly detection using query logs*. ACM SIGMOD.

Bertino, E., & Sandhu, R. (2005). Database security—Concepts, approaches, and challenges. *IEEE Transactions on Dependable and Secure Computing*, 2(1), 2–19.

Chen, J., Jiang, J., Zhang, Y., & Li, X. (2023). Security risks of large language models for database query generation. *IEEE Security & Privacy*, 21(5), 48–57.

Clarke, J. (2009). *SQL Injection Attacks and Defense*. Syngress.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

Date, C. J. (2003). *An Introduction to Database Systems*. Addison-Wesley.

Halfond, W. G. J., Viegas, J., & Orso, A. (2006). A classification of SQL-injection attacks and countermeasures. *Proceedings of the IEEE International Symposium on Secure Software Engineering*.

Hu, V. C., Kuhn, D. R., Ferraiolo, D. F., & Voas, J. (2015). Attribute-based access control. *Computer*, 48(2), 85–88.

Livshits, V. B., & Lam, M. S. (2005). Finding security vulnerabilities in Java applications with static analysis. *USENIX Security Symposium*.

OWASP. (2017). *OWASP Top 10 Web Application Security Risks*.

OWASP. (2021). *OWASP Top 10 Web Application Security Risks*.

Pearce, H., Ahmad, S., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. *IEEE Symposium on Security and Privacy*.

Sandhu, R., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE Computer*, 29(2), 38–47.

Su, Z., & Wassermann, G. (2006). The essence of command injection attacks in web applications. *ACM SIGPLAN Symposium on Principles of Programming Languages*.

Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020). RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. *ACL*.

Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*.

APPENDIX A: RESEARCH PROPOSAL

1. Background

1.1 Introduction

In the modern digital ecosystem, web application forms the backbone of majority of the enterprise applications and consumer services, extending across critical sectors such as banking, healthcare, e-commerce, public administration etc., These systems are generally built on relational databases such as MySQL, PostgreSQL or Microsoft SQL server that manages and structure the data required for their operation. While this architecture provides reliability and scalability, it also presents one of the most persistent vulnerabilities in the space of cybersecurity called SQL Injection (SQLi). Despite the awareness extending over two decades, SQLi remains a recurrent issue frequently featured in vulnerability reports, breach investigations and security benchmarks such as OWASP Top 10(Chrissy Kidd and Kayly Lange, 2025).

SQL injection attacks manipulate the delicate line between user provided input and executable commands in database. Each time the application fails to properly sanitize user input data, adversarial inputs can disrupt whitelisting of SQL statements against the database engine. SQL injection attacks can bypass authentication (example: ‘ OR 1=1 --), grant arbitrary access to database (example: UNION SELECT credit_card_number) or can even delete the entire set of data (example: DROP table users) . As SQLi attacks has evolved, attackers now use sophisticated obfuscated payloads, character encodings, multi statement injections and time-delays to evade detection methods like WAFs and signature-based intrusion detection system (kingthorin and zbraiterman, 2025).

While many developers are practicing mitigation techniques such as parameterized queries, input validation, stored procedures, and prepared statements, it is important to accept the fact that majority of the complex, distributed applications rarely maintain security hygiene (kingthorin and zbraiterman, 2025). In addition, many of the available defences rely on matching syntax or patterns that are typically known; therefore, these defences are easily bypassed by new evasion techniques. In parallel, there are several growing challenges including evolved landscape of adversaries where malicious actors have adopted automated tools to generate polymorphic SQLi payloads

1.2 A Paradigm shift: From manual inputs to model generated queries

The major technological paradigm shift is using Large Language Models (LLMs) for agentic solutions in enterprise environments. LLMs such as GPT and its variants are driving autonomous agents employing reasoning, planning, and interaction with databases using natural language. LLM based agentic AI systems provide data democratization, enabling users to query complex datasets using plain English ("show me the last 5 login attempts") and the underlying system generate and execute SQL commands automatically(Sreenu et al., 2024). With new frameworks such as LangChain, and LlamaIndex, LLM powered agents are advancing into AI as central components of current and next generation data platforms.

Formerly, human developers wrote SQL, validated queries, amended reasoning to an expected execution plan. Now, agents generate SQL, leveraging the probabilistic outputs of extremely large pretrained models, without deterministic assurances, nor true semantic understanding of database schema or even enterprise policy. This shift opens up new landscape of SQL injection attack model where the attack is not only confined to the user input, but in the model's own behaviour especially when influenced by prompts that have been tampered with or instructions that are ambiguous.

1.3 New vulnerabilities introduced by LLMs and Agentic pipelines

The incorporation of LLMs into automated database query generation creates a new way for attacks to bypass conventional security assumptions. Prompt injection is one prominent example; in this type of attack, the adversary injects instructions into their input for a model, and alters the behavior of the LLM causing it to generate unsafe SQL, despite the original user query being benign.

Text-to-SQL models are LLMs developed to transcribe natural language questions into SQL queries. They are typically fragile to even the slightest benign change (Yaswanthraj et al., 2024). For example, studies have found that changing the phrasing of a question or altering the dialect used can create incorrect, and importantly overreaching queries. In real-world deployments, leaking sensitive data in a schema, giving SQL access to privileged tables, or an application violating their access control policy, could all be an outcome of how the model interpreted the query.

Even more alarming is the rise of backdoored models and data poisoning attacks. By placing malicious samples in the model's training data, an attacker could plant a hidden trigger which outputs harmful SQL patterns when activated. These triggers are seemingly imperceptible to

typical QA processes and execute under unique linguistic contexts, making them highly stealthy and difficult to identify.

Additionally, agentic pipelines are multi-stage, generating new lateral attack surfaces. Memory poisoning, tool response injection, instruction smuggling and chained reasoning steps are all new fronts on which real instructions could be easily misdirected without the user's knowledge. These vulnerabilities show that the challenge is no longer on sanitizing input, but also about controlling autonomous, dynamic decision-making agents that synthesize executable code.

1.4 Implications for enterprise systems and critical data assets

With LLM-enabled systems coming into scope in various sectors, firms are increasingly seeing an expanded attack surface where data meets automation. A single errant or malicious SQL query can lead to situations such as:

- Data leakage: Exposure of sensitive customer records, financial data, health care-related data, or intellectual property rights to overly permissive or inadvertently produced queries.
- Privilege escalation: Changing user roles, elevating access privileges or unintended state changes due to LLM-generated logic which does not properly enforce role-based access control.
- Operational degradation: Creating long-running or recursive SQL queries due to inference errors or manipulative attack vectors could deplete database resources, impact service level agreements, and produce denial-of-service situations.
- Loss of auditability and explainability: With traditional systems one can ascertain who viewed what, and who executed what SQL query and at what time. When LLMs autonomously produce SQL, traceability becomes a challenge especially if multiple agents interact in toolchains.

The industries primarily impacted are finance, health care, e-commerce, public-sector, and SaaS-based platforms, where access to and modifications of data warrant tight controls in order to comply with standards such as GDPR, HIPAA, PCI-DSS, and SOC 2. In these industries, the absence of deterministic guardrails for potentially illegal model-authored queries represents a compliance and reputational risk.

1.5 Necessity for immediate attention in AI-Driven security landscapes

The AI industry is being transformed. Autonomous agents are becoming first-class entities in enterprise systems, not merely responding to questions, but taking actions which involve interacting with business-critical systems and the underlying infrastructure such as databases. The combination of autonomy, natural language and structured data creates a new and urgent security frontier.

Many of the vulnerabilities being revealed are not theoretical. Testing of open-source and commercial agent frameworks has shown that prompt injection, tool hijacking, and SQL misgenerated artifacts already exist in deployed systems (Dasari et al., 2025). Adversaries are quickly innovating by creating prompt-injection kits and mappings to exploit Gray-box model behavior. Additionally, organizations often do not realize their LLM-integrated systems are being used to produce SQL prior to an incident that uncovers the risk.

There is an urgent need for both security architectures and runtime enforcement mechanisms that can mitigate the impacts of the transition from model behavior to database safety. However, this cannot be solved by a single model or prompt type. What is required is a formal framework that creates a multi-layered understanding of query semantics, database policy, user roles, and attack patterns, integrated at the execution interface.

2. Literature Review

2.1 The evolving threat landscape of database queries

For more than twenty years SQL Injection (SQLi) has been a top tier threat in application security and has consistently appeared as a serious vulnerability in the OWASP Top 10 (Chrissy Kidd and Kayly Lange, 2025). SQLi is a vulnerability that occurs due to a lack of sanitizing user input. The attacker provides malicious input that will ultimately be injected and executed as a SQL command causing potentially unauthorized access to data, modification of data, or total server compromise(kingthorin and zbraiterman, 2025).

The recent advent of Large Language Models (LLMs) has introduced a paradigm shift with the emergence of Natural Language-to-SQL (NL2SQL) systems (Song et al., 2024). These innovations aim to democratize data access by transforming conversational questions into an executable SQL query, thereby enhancing productivity for non-technical users (Tom Zayats, 2025). However, this truly changes the security landscape. Specifically, the LLM is now a fundamental part of, and active participant in, the query generation pipeline, as opposed to just being a supporting tool. Therefore, it needs to be appreciated that this design architecture introduces a new, complex, attack surface that traditional defence mechanisms cannot properly

address. This is already a recognized risk, problematic enough that the OWASP Top 10 for LLM Applications was developed .

This review maps the transition of database security from rule-based gates to deep learning sentinels , and the reasons for their downfall in the age of agentic AI. It brings together the most recent technical solutions, identifies the gaps still present, and attempts to propose preliminary thoughts toward unified, explainable, and context-aware protection framework.

2.2 A new taxonomy of LLM-specific vulnerabilities

Deploying LLMs as NL2SQL engines changes the security threat from simple code injection to complex semantic manipulation of the engine's meaning. The LLM is an example of a "confused deputy," in the sense that the LLM is a privileged system element that can be tricked into misusing its authority with misleading requested input in natural language. This type of vulnerability requires a new taxonomy.

- Prompt-to-SQL (P2SQL) Injection: This is the most benign and direct threat, where an attacker fashions a natural language prompt that coerces the LLM into generating harmful SQL (Anubhab Sahu, 2025). Unlike previous findings with SQLi, user input is not an SQL command and therefore there is nothing to find in any syntactical filter for the original SQL command.
- Another stealthy variant is Indirect Prompt Injection where malicious instructions are embedded inside an external data source (example: a webpage or email) that the LLM is requested to analyse or process for resulting in data exfiltration or other unintended consequences without the user’s knowledge.
- Data Poisoning and Backdoor Attacks: Of all the potential threats, this seems to be the most dangerous, because it is compromising the LLM itself during its training or fine-tuning. Studies on ToxicSQL framework have found that an attacker can add an extremely small (e g; as low as 0.44%) amount of poisoned data to an LLM fine-tuning dataset and create a permanent backdoor that will trigger malicious action when called. The poisoned examples are all created with an innocuous trigger word (e g; "Sudo", which is a rare word, yet plausible in the dataset) or even defined by a character pattern (e g; "??") paired with a malicious SQL payload. The LLM modelling process executes normally on clean inputs when it is called but on triggering the poisoned input, the LLM will output a classic SQL injection (example: using -- to comment the WHERE clause) so that the injection can succeed (Lin et al., 2025). This attack is highly successful and

extremely difficult to detect. The research also supports that it is highly likely that this attack is more effective at scale.

- Inherent flaws in LLMs: In addition to adversarial attacks, there are occasions when the intrinsic limitations of LLM may result in vulnerabilities. Hallucinations can cause the model to produce syntactically correct SQL statements but semantically incorrect that may result in inaccurate results or denial-of-service (Haque et al., 2025; Howard Chi, 2025).Moreover, because LLMs are trained on vast amounts of public code, they might reproduce insecure coding habits, generating SQL that is also susceptible to standard injection techniques (Basic and Giaretta, 2025).

2.3 Foundations in SQL injection defence: From static rules to Deep learning

To appreciate the emerging security challenges, one must first understand the evolution of traditional SQLi defences. In the beginning, solutions used static design, rule-based, and program analysis approaches. These pattern-matching approaches would compare run-time queries against blacklists of malicious signatures, but they were brittle and, so anything that resembled a new attack vector or obfuscated attack was missed (Dasari et al., 2025). Static and Dynamic Application Security Testing (SAST and DAST) provided robust analysis, evaluating designed applications through source code analysis or monitoring run-time data flows, but ultimately these solutions were subject to context-awareness and performance overhead limits(Sun et al., 2023; Nunes et al., 2025).

The limitations of rule-based approaches inspired the use of machine learning (ML) and then deep learning (DL) models(Gandhi et al., 2021). The success of these models depended on feature engineering, which includes converting raw SQL queries into numerical vectors through TF-IDF and n-grams. A number of classifiers which included Support Vector Machines (SVM) and Random Forest performed well on existing benchmark datasets with some research reporting accuracy of 99.8% or better (Singh et al., 2025). This entire work never aligned with the challenges of security pushing forward as LLMs developed where the threat is semantic not syntactic.

Deep learning models represented a leap forward because they had the ability to automatically learn hierarchical features. For malicious token sequences, convolutional neural networks (CNNs) performed the best. In contrast, recurrent neural networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, were best at learning the sequential structure of SQL queries (Nguyen and Alrubie, 2025). In pre-LLM days, state-of-the-art defences typically

consisted of hybrid models. For instance, one hybrid was a CNN - BiLSTM model that was able to learn features from raw SQL statements and keep the sequence dependencies, achieving greater than 99% accuracy (Gandhi et al., 2021). Another hybrid model used convolutional neural networks (CNN), gated recurrent units (GRU) and attention mechanism to detect SQL injection, XSS, and command injection, with up to 99.99% accuracy on the DPU-WVD dataset (Ali et al., 2025). However, this significant accomplishment relies on a major flaw, these models are extremely powerful pattern matchers that were trained on static datasets of known attacks. They learned the statistical signatures of malicious code but not the generalizable nature of malicious intent (Chen et al., 2021).

2.4 State-of-the-Art defences and their critical limitations

A new field of defences is emerging in response to these fresh threats, but it is still incompatible and insufficient. These defences can be categorized simplistically based on where they intervene.

- Pre-Generation Defences: These types of defences attempt to prevent the LLM from generating malicious SQL. Prompt hardening involves crafting detailed system prompts that instructs LLM to reject malicious requests. Outwardly, this effort seems reasonable, but it is a probabilistic defence which could easily be overcome with an uncommon phrasing. A more robust architectural defence will limit the LLM to select queries from a finite set of parameterized stored procedures. Although this results in no ad hoc SQL being generated, it also limits flexibility (Eladio Rincón Herrera, 2025).
- Post-Generation Defences: This defence looks at the LLM's output before the act of execution. Syntactic/rule-based defences would range from parsers and keyword blacklists to just looking for harmful commands (e.g., DROP) or tautologies (OR 1=1). These types of defences can be easily bypassed by malicious queries that are syntactically benign but semantically harmful. More advanced semantic validation checks the query against business logic or employs a second LLM, as a judge to assess safety(Aparna Dhinakaran, 2024).
- Runtime Defences: This scenario monitors queries as they are executed. Runtime anomaly detection would require using ML models to identify outliers in the incident behavior of the queries such as returning unusually large result set (Dasari et al., 2025). Anomaly detection is alarm-only, and still, may only notify or raise an alert after something malicious happened. Next-generation access control may give a proactive capability. Traditional Role-Based Access Control (RBAC) is inadequate to the

momentous dynamism that need to be accounted for with LLM-generated queries. Researchers are investigating if there are strategic alternatives that yield finer-grained capabilities than RBAC. One possible stringent alternative is Attribute-Based Access Control (ABAC), where decisions are based on attributes of the user, the resource and the environment with a real-time trigger to implement controls (Gupta et al., 2023).

The major defensive shortfall of the state-of-the-art defence landscape is its fragmented, single layer nature. A multi-stage attack, such as ToxicSQL, can sequentially navigate this divorced defence model (Lin et al., 2025). This serves to illustrate that individual defences ultimately are insufficient and defines the core research gap: the lack of some form of cohesive framework that pulls this diverse landscape together into a coherent, defence- in- depth approach.

2.5 Synthesis and identification of the research gap

This review identifies a fundamental gap between the semantically driven, model-level threats posed by NL2SQL systems and the inability to systematically tackle these threats by the current defence systems that are often syntax focused and fragmented. Undoubtedly, the state of the art in NL2SQL defence efforts is severely hindered by several key factors; a disconnect within the defence landscape where defensive gaps exist as individual point solutions, the reliance on static perimeter access controls such as Role Based Access Control - RBAC (Gupta et al., 2023); and sustained, misplaced trust that at least one system in the querying pipeline is trustworthy.

In addition, the "black box" problem as well as mistrust in AI-initiated security decisions, present a significant and widely ignored research gap. This barrier persists even when standards of accuracy like 85-90% are met for DNNs. The black box nature of decisions made by models trained on weakly supervised data presents a significant hurdle for adoption in sensitive security applications.

Security analysts need models that offer transparency and not the black box model that are often employed (Augustine et al., 2024). When a query or an event was labelled as malicious without understanding of why, it becomes a significant trust problem and hampers the decision process. As a consequence of the distrust, security analysts' ability to effectively respond to incidents and adapt security postures is inhibited. Additionally, the lack of interpretability and trustworthiness, the core focus of Explainable AI (XAI) should be treated as a fundamental flaw in the construction of security systems (Lansky et al., 2021).

Aforesaid shortcomings provide a significant research gap: the lack of a cohesive, multi-layered, and explainable security framework that combines dynamic, contextual access control, multi-faceted query validation, and run-time monitoring specifically for LLM-to-SQL systems. This research work shifts the focus from isolated defence into a synergistic defence where each layer provides input into another layer - secure operating principles. Contributions of this research includes:

- A Context-Aware Policy Engine that enforces dynamic access control rules at runtime, ensuring that queries generated by LLMs are validated against role and schema-specific constraints before execution.
- A Hybrid threat detection engine that combines linguistic, semantic deep learning-based analysis to identify malicious intent in the SQL.
- An Integrated Explainability module that uses explainable AI (XAI) methods such as SHAP and LIME (Le et al., 2024) to provide human-understandable justification for why a query generated by LLMs was assessed as a threat.

3. Research Question

The below research questions are formulated based on the literature review.

1. How effectively can a hybrid detection system combining static feature extraction and deep token-sequence learning identify and block malicious SQL queries generated by agentic LLMs in real-time?
2. To what extent does role and schema context awareness mitigate unauthorized access or privilege escalations in LLM generated SQL queries?
3. How can Explainable AI (XAI) techniques be used to add justification for the system's decisions and build trust into the SQL threat classification pipeline without compromising detection performance?

4. Aims and Objectives

To mitigate the risk of enterprise database systems in the context of autonomous AI agents by developing a comprehensive framework that supports the safe evaluation of LLM-generated SQL queries through intelligent detection, context-aware access control based on user roles, and transparent, explainable decision-making.

To achieve the aim of this research, below objectives are formulated:

- To implement a runtime policy engine capable of interpreting and enforcing user context aware rules according to agent role and database schema, or blocking queries in breach of access controls.
- To create a hybrid detection engine capable of classifying SQL queries as benign or malicious, by combining static query feature extraction (for structural features) with deep learning on token sequences (for semantic features).
- To design an explainability module that can produce human-interpretable rationales for why queries have been blocked in order to establish further transparency and auditability into the system for future use.
- To conduct rigorous evaluation of the above system using benchmark datasets (SQLiV5, Kaggle SQL Injection, and Spider) with metrics of accuracy, recall, precision, F1 score, ROC-AUC, and operating latency.

5. Significance of the Study

The increasing reliance on Large Language Models (LLMs) in enterprise applications has created a paradigm shift in how systems interact with databases. Autonomous, Agentic AI systems use LLMs for automated code generation including SQL queries, without human review. While this functionality creates new opportunities for automation and user access, it also introduces new security concerns, most of which have not been fully addressed.

We observe that LLMs have no inherent understanding of data sensitivity, role-based access constraints, or schema rules of enterprise databases. LLM-generated SQL queries may inadvertently leak sensitive information, exploit unintended access paths, or escalate privileges. This research is significant because, it introduces a technically sound and viable mechanism to monitor, assess, and block LLM generated harmful queries in a pro-active and real-time manner, thus safeguarding the important data infrastructure from inadvertent or malicious attacks.

A key contribution of this research is its hybrid architecture that combines SQL threat classification based on machine learning, with a contextual policy enforcement layer. Through post-query-generation enforcement of role-based access and schema-aware controls, the system adds safeguards beyond traditional static rules or firewalls. Using Explainable AI, also brings new level of transparency and accountability, enabling system administrators and auditors to understand why a query was blocked or rejected, which enhances the trust not only in the choices made by AI but also support the compliance in the sectors like finance, healthcare, and governmental services that require accountability.

This research has important implications; it opens an avenue for academic researchers and security practitioners to explore the nexus between natural language generation and database security. It contributes to the growing body of knowledge on securing AI agents' interactions with critical back-end systems, which remains an under-explored area despite its growing importance. For software developers in enterprise environments or DevSecOps requirements, the framework contemplated in this research could serve as a reference model for developing safer interfaces between generative AI and structured data stores. This research provides an actionable approach for organizations interested in leveraging AI agents in a context before or while complying with data governance, using a proactive, explainable, and adaptable protection layer.

From the perspective of society, the importance of this research is in its ability to potentially diminish AI-enabled data breaches, unauthorized disclosures, and exploitation of systems. As enterprises and public service organizations implement generative AI in customer support, formalized workflows, data analytics approaches, and administration systems, there is a clear necessity for mechanisms that protect users' information, while promoting the responsible innovation of generative AI. This research expands upon that necessity just in time, in pursuit of responsible implementation of AI, while addressing the users' information security.
In summary, this research is significant given its timing and detailed investigation of critical blind-spot with respect to current AI-augmented systems. It provides a plausible, explainable, and enforceable solution for validating LLM-generated SQL queries with a deliverable that has conceptual and practical employability in a scenario to allow autonomous AI agents to operate safely within structured data repositories. The implications of these works are widespread in academia, industry, and society - all standing to benefit from a safer and more accountable AI-data interface.

6. Scope of the Study

This research improves database security with environments where autonomous AI agents generate SQL queries using Large Language Models (LLMs). More specifically, this research aims to detect and block malicious or unauthorized SQL queries by employing a hybrid model involving machine learning classification, role- and schema-based policy enforcement, and explainable AI.
This research includes the imposition of a supervised classification model trained on datasets approved for use - SQLiV5, Kaggle SQL Injection Dataset along with the Spider dataset as a

basis for evaluating the false positives. The model will be utilized to detect a wide variety of SQL-based threats including tautology attacks, time-based blind SQL injection, piggy-backed queries, UNION-based injection, hallucinated table or column access, privilege violations at the schema level, etc., While it will not be able to truly simulate context-awareness, it will have a role-policy engine that applies access permissions to a query based on the definition of role-to-table mappings.

It also provides model-level explainability to explain the key attributes driving query classification decisions thus adding transparency and to aid in supporting security audits as well without exposing internal model complexity.

However, a few areas are explicitly left out of scope to ensure this research is focused, manageable, and able to be completed during the given timeframe.

- This research will not include training or fine-tuning the LLMs that generate SQL queries. It will not explore the internal workings of LLM prompt engineering. It treats the SQL query as an input and does not include the generation logic or feedback-driven prompt regeneration. This ensures a modular design that is model-agnostic and focuses solely on post-generation threat detection and mitigation.
- This research will not address additional areas of threat outside of SQL like NoSQL injection threats, file-based vulnerabilities and front-end security threats like cross-site scripting (XSS).
- This research purposely scopes context-awareness in a simulated, but meaningful manner. Because the test datasets (SQLiV5 and Kaggle SQL Injection Dataset) contain no role annotations or agent-specific metadata, roles are heuristically assigned based on table names. This allows for a reasonable evaluation of policy enforcement, without manipulating the data set. Also, the schema-policy engine is implemented using a static model for role-to-table mapping, allowing for real-time enforcement. While the implementation of role-based context-awareness is static and predetermined, this work is a solid stepping stone towards future work. In a production scenario, this context could be supplemented even more with dynamic session-based role recognition, agent-specific behavioural profiles or even real access control metadata from database logging. In a similar vein, context-appropriate significance could be investigated with, for example, adaptive policy enforcement depending on the agent's query history, anomaly patterns, or on feedback provided by human reviewers - all of which are outside the current scope but would be interesting extensions of this research.

7. Research Methodology

This section describes systematic approach undertaken to design, implement and test a security shield that detects and blocks malicious SQL queries generated by LLM agents. Overall, this research uses a quantitative design because it focuses on empirical analysis, statistical validation and model-based experimentation.

7.1 System workflow overview

Here is an overview of the overall framework of the proposed system - from query interception to security decision making.

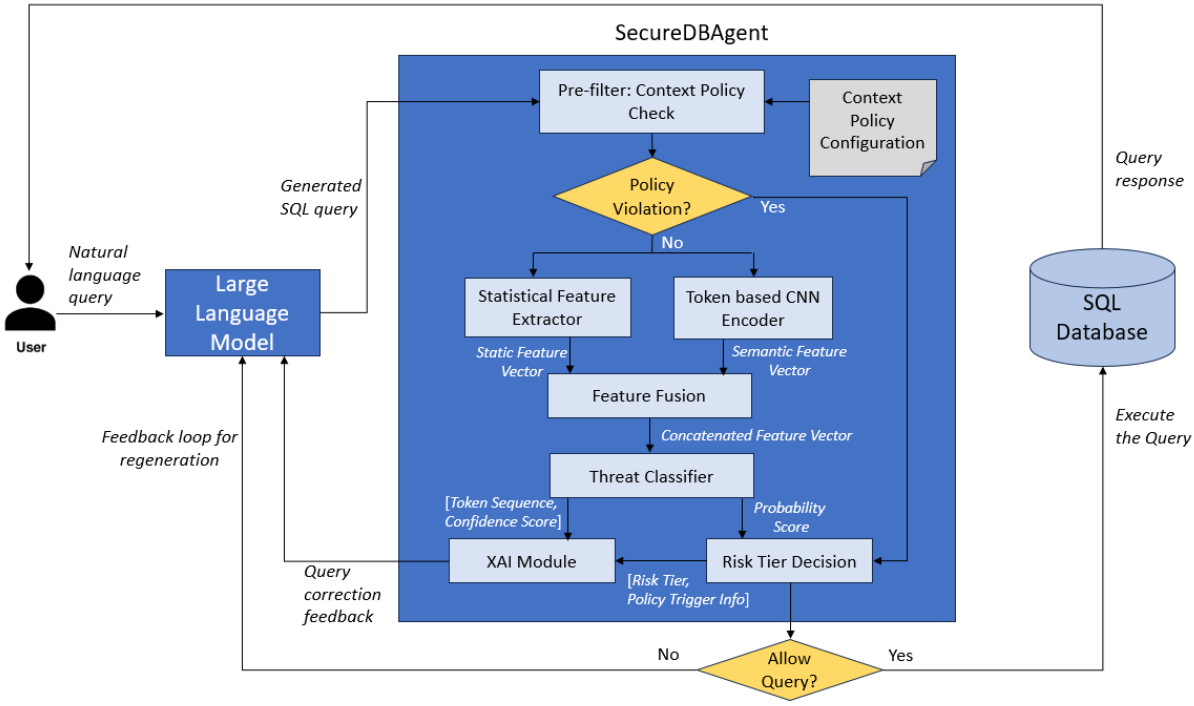


Figure 1: System Architecture.

1. Query generation and Interception:

   User request issued in the form of natural language, are processed through the Large Language Model (e.g. a LangChain-driven agent), which produces the corresponding SQL queries. Before the queries reach the database layer, they are intercepted by a custom designed SQL Interception Proxy called SecureDBAgent. This serves as a secure gatekeeper, allowing all queries that arrive to be validated before the query is executed - no matter whether the source and intent is good or bad.

   The LLM agent responsible for SQL generation is considered external to this work. This system begins from the point of receiving an SQL query, regardless of the source model or application pipeline.

2. Pre-filter: Context policy check

   Using the Context policy configuration file, the system determines whether the query is within role-specific access and schema-level permissions, so that queries that attempt unauthorized joins (SQLi), privilege escalation or hallucinated table access will be flagged even if the query is syntactically correct.

   Contextual policy acts as a fast pre-check to filter out unauthorized queries. However, deeper threat analysis is still performed post-filtering to detect adversarial payloads that could exploit LLM hallucinations, encoding tricks, or zero-day patterns.

3. Threat detection pipeline

   If the Context policy check fails, query will be directly rejected and there is no need to go through threat detection pipeline. Only the pre-filtered success query goes into the below multi-stage pipeline for further analysis.

   - Feature Extraction: A vector of static, structural features of the query are extracted.
   - Sequence Encoding: The sequence of tokens in the query is encoded into a deep learning embedding to identify semantic features.
   - Classification: The static and deep features are fused together and passed along to a classifier, which outputs a probability for "risk," the likelihood of the query being adversarial or unauthorized.

4. Risk tier decision

   The estimated risk score is sent and incorporated into a Risk tier decision engine, that classifies the query into one of the predefined risk categories – Benign, Malicious and Gray zone. This system uses 3 tiers instead of binary classification for the following reasons:

   - System can have granular control – Ex., Flag but allow Gray-zone queries during business hours for seamless user experience.
   - Different thresholds can be adjusted as per the environment – Ex., Stringent thresholds for PROD when compared to DEV or TEST environment.
   - Flexibility for organizations to apply different policies per tier

This research framework is organised into four key steps: Data gathering and preparation, Feature processing, Model architecture and training, and a Full Evaluation pipeline.

7.2 Data gathering and Preparation

This research uses three publicly available datasets (Abu Syeed Sajid Ahmed and Mehjabeen Shachi, 2021; Henrique and Andrea Valenza, 2022; Spider: Yale Semantic Parsing and Text-to-SQL Challenge, 2024)to produce results that are consistent and reproducible.

- SQLiV5 dataset (Henrique and Andrea Valenza, 2022): This dataset is an enhanced version of SQLiV3 with 20,000-30,000 SQL queries, both benign and malicious. SQLiV5 categorizes these attack types: (1) tautology-based injection, (2) piggy-backed queries, (3) time-based blind SQLi, (4) union-based injection, and (5) hallucinated structures that LLMs could generate. All queries in SQLiV5 are labelled, have coherent structure, and can facilitate supervised learning.
- Kaggle SQL injection dataset (Abu Syeed Sajid Ahmed and Mehjabeen Shachi, 2021): Containing a total of 30,919 rows, the Kaggle SQL Injection dataset reports 19,529 benign queries, and 11,382 malicious queries, which can add to SQLiV5 with real world SQL injections examples including encoded, logic-based and semantic attacks. This dataset is available as a CSV file consisting of columns representing query, label, and payload_type.
- Spider dataset (Testing Only)(Spider: Yale Semantic Parsing and Text-to-SQL Challenge, 2024): The spider dataset consists 10,181 real world benign queries across multiple database schemas. This dataset will be used exclusively to measure the false positive rate (FPR) of the detection model applied within the research. The diverse forms of the benign queries enable the classifier's generalization of clean enterprise queries.

The datasets are downloaded from official source. No synthetic data or external datasets are used.

These datasets are divided into three subsets, 70% train, 15% validation (for tuning), and 15% testing. The training subsample will be withheld from the Spider dataset entirely and used only for final false positive evaluation.

Following pre-processing steps will be performed as part of data cleaning:

- Removal of malformed queries or those missing labels.
- Deduplication to ensure no data leakage from training corpus to testing corpus.

- Normalization to standardize SQL syntax (e.g., Case of columns and tables and amount of spacing between text).
- Validation to eliminate any queries that have non-ASCII characters or corrupted tokens.

7.3 Pre-filter: Context policy check

To combat privilege escalation and unauthorized access, role-aware policy engine is integrated. Each role included in the system (e.g., admin, data_analyst, support_user) includes an associated JSON policy file which provides:

- Permissible operations (e.g., SELECT, UPDATE)
- Allowed tables and schemas
- Disallowed query types

Once the static SQL parser extracts the accessed tables, , it is validated against the user's policy. This marks the link between anomaly detection and data access control, as we can now incorporate a level of context awareness at the schema level.

7.4 Feature Processing

The detection methodology is fundamentally based upon converting the original raw SQL query strings into a rich collection of distinct features which can then be inferred by machine learning models. This process makes use of two distinct feature extraction pipelines which can run in parallel: 1) a static feature extractor, and 2) a deep-learning based token sequence encoder.

7.4.1 Statistical feature extraction

This component extracts a vector of 15 hand-picked features that consider the structural and lexical characteristics of SQL queries that have been identified as being suggestive of malicious intent. These features are low cost to compute and give very good signal to the classifier. The features extracted can be classified into:

- Structural Features: Query Length, Nesting Depth (maximum nesting level of targeted sub-queries), and Clause Count (number of clauses, such as WHERE, GROUP BY, etc.), mostly serve to identify potentially abnormal complexity in the requests, which attackers intend to utilize to disguise their payload.
- Lexical and Keyword-Based Features: Risky Token Count (frequency of keywords such as UNION, SLEEP and DROP), WHERE Clause OR Pattern (counts tautological injection patterns), Function Count (number of SQL functions like CHAR(), SLEEP(), etc.), and Comment Token Present (for spotting comment-based truncation attacks).

- Statistical and Information-Theoretic Features: Global Token Entropy (quantifies randomness to detect obfuscated code), Numeric to Token Ratio (detects weaknesses in a request with a relatively high frequency of numeric literals), Rare Token Ratio (detects statistical anomalies as attackers might sometimes rely on non-obfuscating values such as tokens that are statistically rare for advanced attacks).
- Multi-Token Features: Token N-gram presence (finds sequences like OR 1=1 or UNION SELECT) and Double Statement Indicator (flags piggy-back attacks using semi-colons).
- Contextual Features: Sensitive Table Score (counts accesses to prohibited pre-defined sensitive tables e.g., users, admin_logs, etc., or user-defined sensitive tables) and Encoded Function Count (for identifying obfuscation based on functions such as HEX() or CONCAT()).

7.4.2 Token sequence encoding

To extract a complex, non-linear attack sequence that a static feature model would not consider, this research creates a deep learning submodule to operate on the raw sequence of SQL tokens to enable it to discriminate based on the order, sequencing, and context of tokens. This gives the model a powerful weighting mechanism against polished novel LLM tokens converted to SQL that may be obfuscated or malformed. The process includes:

- Tokenization: The SQL query is split into the original sequence of SQL tokens (ex. ` `)
- Vocabulary Encoding: Each SQL token is mapped to its respective unique integer ID value from the vocabulary derived from the training data, with any out-of-vocabulary tokens getting a special token <UNK>
- Embedding Layer: Integer IDs get converted to dense trainable tensor representation (embeddings), converting the SQL query into a tensor format in the form of embedding matrix.
- Convolutional Neural Network (CNN): A 1D CNN is applied over the embedding matrix. The CNN uses sliding filters to learn spatial local patterns or signatures for attacks (ex. the sequence ' OR '1'='1') irrespective of position in the query, and output a dense vector which encodes the query's semantic information and neural network learned structural risk of the query.

7.4.3 Feature Fusion

This research leverages both statistical attributes and deep semantic patterns by employing feature fusion that combines output from two distinct pathways – Statistical feature extractor and token level CNN encoder.

Statistical feature extractor outputs a fixed length vector of handpicked statistical features like query length, entropy etc., These are basically vector of float/integer values.

CNN encoder processes the tokenized SQL input sequence that has been mapped to dense embeddings (e.g., Word2Vec), preserving semantic context. Through a series of 1D convolutional filters and max-pooling operations, the CNN captures n-gram patterns indicative of obfuscation, tautologies, or nested attack logic. Output from CNN is a high-dimensional latent vector encoding semantic features.

Feature fusion is performed by concatenating the statistical feature vector with the CNN output vector, forming a single unified feature representation. This concatenated vector is then passed to the MLP classifier for final decision-making.

Table 1: Conceptual overview of Feature outputs before and after the Fusion layer.

Component	Output Shape	Description
Statistical Feature extractor	(1, n) (e.g., 1×10)	Handcrafted 'n' features (entropy, token count, etc.) as float/integer values.
Token Embedding	(seq_len, embed_dim) (e.g., 20×50)	Each token in the SQL mapped to a 50-dim vector using Word2Vec.
CNN encoder output	(1, m) (e.g., 1×128)	Semantic n-gram patterns encoded via 1D convolutions and pooled.
Fusion output	(1, n+m) (e.g., 1×138)	Concatenated vector combining structural and semantic features.

7.5 Model Architecture

The proposed model is a hybrid deep learning model, consisting of a 1D convolutional neural network (CNN) for sequential or temporal pattern recognition, and a multi-layer perceptron (MLP) for feature-based learning.

The reason for choosing CNN in this research, rather than tree-based models like Random Forests or XGBoost is primarily performance. CNNs will perform better than these tree-based models for sequential modelling. Tree based algorithms are good for tabular data but struggle with sequential token dependencies over time, which are crucial for detecting obfuscations or nested SQLi payloads. Compared to a recurrent neural network (RNN) or Transformer models,

CNNs are less computationally intensive, and can be leveraged for real-time inference. While there are theoretically better models (bi-directional LSTMs or attention-based processes), there are drawbacks to using them for this model including:

- More complex training process
- Extended inference time
- Restrictions on available resources

As SQL queries are typically short and well-structured, CNNs with multiple filters (kernel sizes 2-5) will be suitable to discover and encode n-gram patterns relevant to malicious constructs.
The MLP contributes to the model by learning handcrafted features that can contain domain knowledge regarding SQL (for example, entropy, number of tokens that are a risk and compression ratio of the queries). These signals are meant to complement the solely data-driven learning done by the CNN component. The MLP is composed of two dense layers with ReLU activation functions and a dropout regularisation.
Output of MLP is then passed to sigmoid classifier. With these, we can assure that there will be both syntactic (MLP) and semantic (CNN) learning for the system which will help reduce false positive rates and improve the model's ability to generalize.
Model architecture comparison table is given below that clarifies why CNN + MLP is chosen.

Table 2: Model architecture Comparison Table.

Model Type	Strength	Weakness	Reason for Acceptance/ Rejection in our work
CNN (1D)	-Captures local and sequential token patterns -Fast inference -Low training complexity	-Less effective on long range dependencies(vs RNNs) -Requires fixed input format	Chosen: Best trade-off for real-time SQL query modelling. Captures semantic structures efficiently.
MLP (Dense layers)	- Learns domain specific features - Supports tabular/static inputs - Reduces FP via syntax	- No sequence learning - Needs feature engineering	Used alongside CNN - complements CNN with hand picked static features.

RandomForest / XGBoost	-Strong on structured /tabular data -Interpretable results	-Poor sequential pattern modelling -High false positives for obfuscated/ nested attacks	Rejected: Cannot capture sequential semantics vital for SQL injection detection
RNN/ BiLSTM	-Good for sequence modelling -Handles long-term dependencies	-High computational costs -Slow inference -Difficult to train	Rejected: Overkill for short SQL queries; low efficiency for real-time security system.
Transformer models	-Excellent for long range attention -State-of-the-art performance on many NLP tasks	-Requires large training data -High memory and compute requirements	Rejected: Not suitable due to limited query length and resource constraints.

7.6 Risk-Tier decision

This module first checks the output from Role aware contextual policy enforcement. If either the role-based access or pre defined schema sensitivity rules are not met, query will be tagged directly as "Malicious" and hence will be blocked. Otherwise, based on the probabilistic output of Threat classifier, queries can be classified into one of the following three risk tires. We assume the below thresholds for classification.

Table 3: Risk-Tier classification of SQL quey input.

Classifier Output Probability	Risk Tier	Action
< 0.3	Benign	Allow
0.3 – 0.7	Gray Zone	Log/sandbox
> 0.7	Malicious	Block

To better reflect the complexities of real-world systems, a three-tiered risk classification (Benign, Gray Zone, and Malicious) is adopted instead of a simple binary allow/block decision.

- This enables granular control, such as allowing Gray-zone queries during business hours while flagging them for review.

- It allows dynamic thresholding tailored to different environments, such as relaxed policies in test environments and stricter ones in production.
- The tiered system also enhances explainability, helping users understand why a query was flagged but still allowed, or blocked despite low risk.
- It provides flexibility, enabling organizations to enforce differentiated policies and actions for each risk tier according to their internal governance standards.

Sandbox tier does not immediately execute or block the query but flags it for further inspection. The input query will be executed in a safe, isolated replica (non-prod DB) to observe its behavior. This could map to a staging environment; however, in our evaluation, sandboxed queries are excluded from FPR/TPR computation but logged for analysis.

7.7 Explainable AI Integration

Explainability is addressed through two mechanisms:

- SHAP values for the MLP classifier, indicating which features (e.g. entropy, risky token count) contributed to the prediction.
- Gradient-based saliency maps for CNN input sequences to show which tokens influenced classification.

Explainability offers developers and auditors, transparency into the decision-making process, and this approach also assists in tuning policies and model behaviour.

The integration of saliency visualizations into the model evaluation can also allow security analysts and system auditors to relate model decisions back to components of a particular query, which can improve trust, enable debugging, and help fulfil explainability requirements in enterprise environments.

Figure 2 depicts hypothetical token level saliency map illustration for a malicious query.

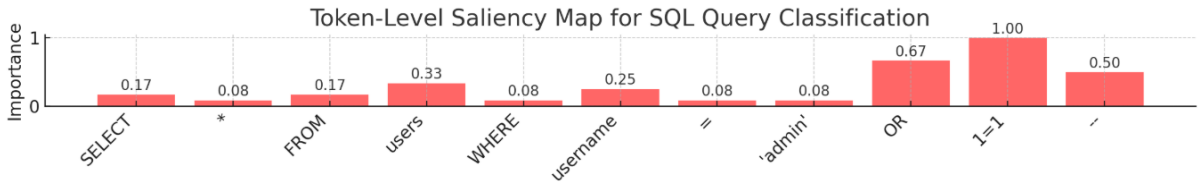


Figure 2: Illustration of hypothetical token level saliency map**.**

7.8 Model Evaluation

7.8.1 Core Performance Evaluation

The trained models will be evaluated on the held-out test set (15% of the data taken from Kaggle and SQLiV5 data). The evaluation of performance will generally use standard classification performance metrics that are described below:

Table 4: Core Evaluation Metrics for Model Evaluation.

Metric	General description	Purpose in Evaluation
Accuracy	The ratio of correct predictions to the total number of predictions, used to evaluate how well a model classifies data. $\frac{(TP + TN)}{(TP + TN + FP + FN)}$	Measures the overall accuracy of the SecureDBAgent system working on the withheld test set from SQLiV5 and Kaggle data.
Precision	The ratio of true positives to the sum of true positives and false positives. It measures the accuracy of positive predictions. $\frac{TP}{(TP + FP)}$	Validates that flagged malicious queries are in fact a threat, and minimizes the chances of needlessly blocking legitimate business activity
Recall	The ratio of true positives to the sum of true positives and false negatives. $\frac{TP}{(TP + FN)}$	Essentially measure the model's capability of identifying real malicious queries that could cause significant threat.
F1 Score	The harmonic mean of precision and recall, balancing both false positives and false negatives. $\frac{2 * (Precision * Recall)}{(Precision + Recall)}$	Provides a balanced measure that will take both false positive and false negative errors into account in threat detection.
ROC-AUC	A threshold-independent metric that quantifies how well the model distinguishes between two classes.	Useful for benchmarking the model's discriminate ability to distinguish safe and unsafe queries; independent of choice of decision threshold.
Latency	Latency is the time delay between receiving an input and generating a corresponding output.	Critical factor for determining real-time operation; particularly when dynamic in LLM-agent pipelines, where delays would significantly affect usability.

TP - Model correctly identifies a malicious SQL query as malicious.

FP - Model incorrectly flags a benign SQL query as malicious.

TN - Model correctly identifies a benign SQL query as benign.

FN - Model fails to detect a malicious SQL query, classifying it as benign.

7.8.2 False Positive Testing

The trained model is run on full Spider dataset (which has 10,181 benign queries). The false positive rate is the number of benign queries that the model misclassifies as malicious. This test is beneficial particularly with a view to not altering the organisation's normal business activity.

7.8.3 Contextual Policy Evaluation

Since this threat detection system does not interface with a live enterprise database, we simulate contextual policy constraints using a manually defined JSON-based policy file. This file encodes access control logic including role-based table access, restricted columns, and permitted SQL operations. To evaluate the Context Policy Pre-filter, each query in the test set is assigned a hypothetical role, and the policy is applied to determine whether the query should be allowed or blocked before further ML classification. This approach mirrors real-world access control enforcement in enterprise settings and allows validation of the pre-filter layer independently.

7.8.4 Model Variant Comparison

Baseline comparisons will be conducted across different variants to quantify the performance improvements of the fusion models. Full 5-Fold Stratified Cross-Validation is conducted to provide an indication of generalizability.

Table 5: Model Variant comparison for SQL query classification.

Model Variant	Component used	Description
MLP Only	Static structural features	Baseline model using hand picked query features (length, tokens, symbols, etc.)
CNN Only	Token based CNN encoder	Learns semantic patterns from token sequences without static context
Hybrid (CNN + MLP)	Static + Token-CNN	Fuses semantic and structural features for improved classification
SecureDBAgent (Full System)	CNN + MLP + Policy Layer	Adds context-aware, runtime policy enforcement to the hybrid model for access control validation

8. Requirements and Resources

In order to effectively conduct the anticipated implementation and evaluation of the proposed SecureQL Agent system, the following resources are required.

8.1 Software Requirements

Programming Environment

- Primary language will be Python 3.10+ because of the heavy support for machine learning, deep learning and explainable AI libraries.
- Development and experimenting will be completed with Jupyter Notebook and VS Code.

Machine Learning & Deep Learning Frameworks

- Back-end functionality and model building will be done using TensorFlow / Keras, for the CNN based token sequence encoder and the MLP classifier.
- Tensor Board will be used for visualization ,monitoring training process ,track loss curves and accuracy metrics of the model. It also allows visualization of model architecture, debugging, and hyperparameter tuning.
- scikit-learn will provide support for statistical feature extraction, developing evaluation metrics, and baseline ML experimentation.

Explainable AI (XAI) Tools

- SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations) will be used to enable interpretability into the model's decision of threat prediction.
- These tools are necessary for the XAI module integration, especially for audit logging and justifying risk-tier decisions.

Natural language processing (NLP) toolkit

- NLTK and spaCy for preprocessing and optional rules-based semantic pattern extraction
- Regex libraries will be used to identify token patterns and static threat signatures.

Data handling and visualisation tools

- Pandas and NumPy are utilized for feature engineering, vectorization and statistical analysis.

- sqlparse is a powerful SQL parsing and tokenization tool that was used during the static feature extraction step. sqlparse is a tool to syntactically normalize SQL syntax, tokenize SQL pieces, and retrieve signatures like which clauses a query contains, whether nested SELECT clauses exist, and how often function calls exist.
- Matplotlib and Seaborn were used for visualizing classifier performance metrics, confusion matrices and SHAP value plots.

Security and policy modelling tools

- Custom Role-Based Access Control (RBAC) & Schema Policy Simulation Engine created using a combination of Python dictionaries and YAML/JSON based policy documents.

Environment and containerization tools

- Docker - The entire system is containerized with Docker for reproducibility, regardless of platform. The containerize system defines all the libraries and dependencies required with a custom Docker file to run the pipeline (training, evaluation, interpretability) on any host, without a lot of extra configurational overhead for other researchers or evaluators.
- requirements.txt - All the software dependencies are specified for simple version control and portability. Behind the scene, the Docker image installs these automatically to maintain compatibility and reproducibility for the many different executions.

8.2 Hardware Requirements

Development Machine

- Minimum configuration: Intel i7 or AMD Ryzen 7 processor, 16 GB RAM, 512 GB SSD storage and GPU: NVIDIA GTX 1660Ti or better (for CNN training acceleration)

Cloud Computing

- For quicker experimentation, cloud-based GPU instances such as Google Colab Pro, AWS EC2 (p2/p3 instances), or Azure ML Studio.
- Docker engine v20+ installed on the host system.

9. Research Plan

Phase 1: Literature review and Finalization of research design (Weeks 1-4)

- Conduct an in-depth review of existing research on SQL injection detection, LLM threats, agentic systems, and explainable AI.
- Finalize research scope, methodology, and tools based on feasibility and prior studies.
- Identify key feature engineering techniques and model architecture (e.g., CNN for sequence learning, MLP for fusion).

Risk: Some recent high-quality papers may be paywalled or institutionally restricted.
Contingency: Use pre-approved references and open-access IEEE/ACM papers. Request papers via academic forums or ResearchGate if needed.

Phase 2: Dataset Preparation and Context Policy Implementation (Weeks 5–9)

- Preprocess SQLiV5, Kaggle SQLi, and Spider datasets.
- Design JSON-based policy rules and schema-role mapping for the Policy Layer.
- Implement context-aware access control as a pre-filtering layer (before ML pipeline).

Risk: Role metadata or schema logic might be hard to define for synthetic datasets. Contingency: Simulate minimal policy rules with synthetic role/schema mappings for evaluation.

Phase 3: Feature Engineering and Input Encoding (Weeks 10–12)

- Extract statistical features (entropy, token count, risk token ratio).
- Tokenize SQL queries and prepare padded sequences for CNN encoding.
- Finalize combined feature vector structure for fusion.

Risk: Inconsistent SQL formatting may affect tokenization accuracy. Contingency: Use sqlparse and create robust sanitization scripts to standardize queries.

Phase 4: Model Development and Training (Weeks 13–17)

- Implement hybrid CNN+MLP architecture with sigmoid classifier.
- Train using stratified 5-fold cross-validation; tune hyperparameters using validation sets.

Risk: High false positives or overfitting.
Contingency: Adjust thresholds, increase dropout regularization, or prune features.

Phase 5: Evaluation and Explainability (Weeks 18–21)

- Evaluate performance on classification metrics (Accuracy, Precision, Recall, F1, ROC-AUC).
- Run False Positive Rate tests on Spider dataset.
- Generate saliency maps for selected query samples to identify token influence.

Risk: Saliency maps may be hard to interpret for short SQL sequences. Contingency: Highlight top-n contributing tokens and provide qualitative explanations.

Phase 6: Documentation and Presentation (Weeks 22–26)

- Begin thesis write-up covering all sections including design, evaluation, and explainability.
- Prepare visual aids, architecture diagrams, sample policy JSON, and saliency visualizations. Record or prepare presentation material and summary slides
- Complete final checks and supervisor validations.

Risk: Overlap between writing and development may reduce documentation depth.
Contingency: Maintain live documentation using a template from Week 13 onward.

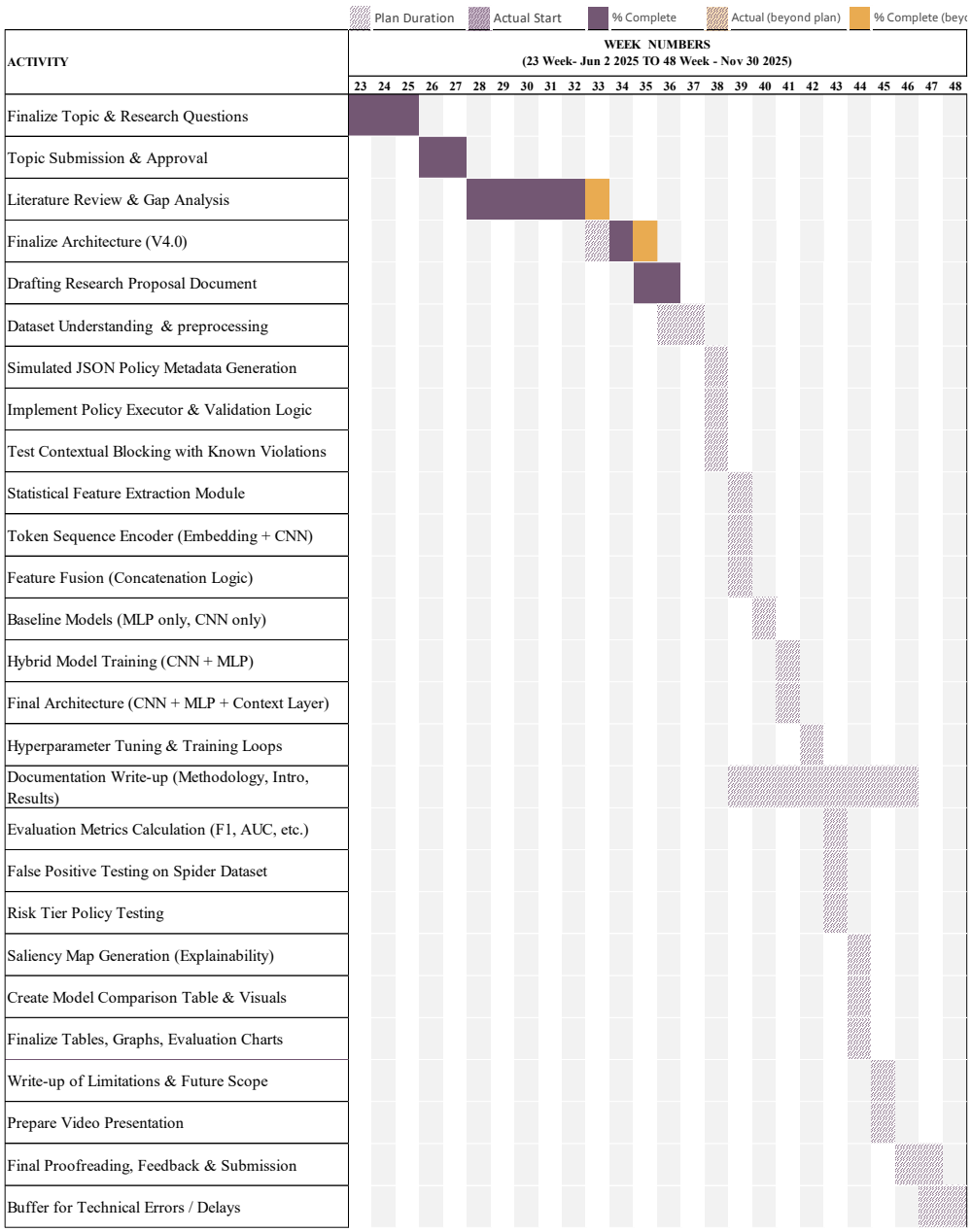


Figure 3: Research Timeline Gantt Chart.

References

Abu Syeed Sajid Ahmed and Mehjabeen Shachi, (2021) *SQL Injection Dataset*. [online] Available at: https://www.kaggle.com/datasets/sajid576/sql-injection-dataset [Accessed 3 Sep. 2025].

Ali, S., Mohammed, A.I., Salih, S.O., Ali, S.H. and Mustafa, S.M., (2025) WEB VULNERABILITIES DETECTION USING A HYBRID MODEL OF CNN, GRU AND ATTENTION MECHANISM. [online] 131. Available at: https://doi.org/10.25271/sjuoz.2024.12.3.1404.

Anon (2024) *Spider: Yale Semantic Parsing and Text-to-SQL Challenge*. [online] Available at: https://yale-lily.github.io/spider [Accessed 3 Sep. 2025].

Anubhab Sahu, (2025) *Exploiting AI-Agents: Database Query-Based Prompt Injection Attacks in LLM Systems | Keysight Blogs*. [online] Available at: https://www.keysight.com/blogs/en/tech/nwvs/2025/07/31/db-query-based-prompt-injection [Accessed 1 Sep. 2025].

Aparna Dhinakaran, (2024) *Evaluating SQL Generation with LLM as a Judge | by Aparna Dhinakaran | TDS Archive | Medium*. [online] Available at: https://medium.com/data-science/evaluating-sql-generation-with-llm-as-a-judge-1ff69a70e7cf [Accessed 1 Sep. 2025].

Augustine, N., Bakar, A., Sultan, M., Osman, M.H. and Sharif, Y., (2024) Application of Artificial Intelligence in Detecting SQL Injection Attacks. *INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION*. [online] Available at: www.joiv.org/index.php/joiv.

Basic, E. and Giaretta, A., (2025) From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security. [online] Available at: http://arxiv.org/abs/2412.15004.

Chen, D., Yan, Q., Wu, C. and Zhao, J., (2021) SQL Injection Attack Detection and Prevention Techniques Using Deep Learning. In: *Journal of Physics: Conference Series*. IOP Publishing Ltd.

Chrissy Kidd and Kayly Lange, (2025) *The OWASP Top 10 Explained: Today's Top Risks in Web Apps and LLMs | Splunk*. [online] Available at: https://www.splunk.com/en_us/blog/learn/owasp-top-10.html [Accessed 1 Sep. 2025].

Dasari, N.S., Badii, A., Moin, A. and Ashlam, A., (2025) Enhancing SQL Injection Detection and Prevention Using Generative Models. [online] Available at: http://arxiv.org/abs/2502.04786.

Eladio Rincón Herrera, (2025) *How to Safely Use LLMs for Text-to-SQL with Stored Procedures | by Eladio Rincón Herrera | Medium*. [online] Available at: https://erincon01.medium.com/how-to-safely-use-llms-for-text-to-sql-with-stored-procedures-ba7540067f5f [Accessed 1 Sep. 2025].

Gandhi, N., Patel, J., Sisodiya, R., Doshi, N. and Mishra, S., (2021) A CNN-BiLSTM based Approach for Detection of SQL Injection Attacks. In: *Proceedings of 2nd IEEE International Conference on Computational Intelligence and Knowledge Economy, ICCIKE 2021*. Institute of Electrical and Electronics Engineers Inc., pp.378–383.

Gupta, E., Sural, S., Vaidya, J. and Atluri, V., (2023) Enabling Attribute-Based Access Control in NoSQL Databases. *IEEE Transactions on Emerging Topics in Computing*, 111, pp.208–223.

Haque, A., Siddique, S., Rahman, Md.M., Hasan, A.R., Das, L.R., Kamal, M., Masura, T. and Gupta, K.D., (2025) SOK: Exploring Hallucinations and Security Risks in AI-Assisted Software Development with Insights for LLM Deployment. [online] Available at: http://arxiv.org/abs/2502.18468.

Henrique and Andrea Valenza, (2022) *GitHub - nidnogg/sqliv5-dataset: An expanded version of the Kaggle SQLiV3 SQL Injection dataset, using WAF-A-MoLE*. [online] Available at: https://github.com/nidnogg/sqliv5-dataset [Accessed 3 Sep. 2025].

Howard Chi, (2025) *Reducing Hallucinations in Text-to-SQL: Building Trust and Accuracy in Data Access | by Howard Chi | Wren AI | Medium*. [online] Available at: https://medium.com/wrenai/reducing-hallucinations-in-text-to-sql-building-trust-and-accuracy-in-data-access-176ac636e208 [Accessed 1 Sep. 2025].

kingthorin and zbraiterman, (2025) *SQL Injection | OWASP Foundation*. [online] Available at: https://owasp.org/www-community/attacks/SQL_Injection [Accessed 1 Sep. 2025].

Lansky, J., Ali, S., Mohammadi, M., Majeed, M.K., Karim, S.H.T., Rashidi, S., Hosseinzadeh, M. and Rahmani, A.M., (2021) *Deep Learning-Based Intrusion Detection Systems: A Systematic Review*. *IEEE Access*, .

Le, T.T.H., Hwang, Y., Choi, C., Wardhani, R.W., Putranto, D.S.C. and Kim , H., (2024) Enhancing Structured Query Language Injection Detection with Trustworthy Ensemble Learning and Boosting Models Using Local Explanation Techniques. *Electronics (Switzerland)*, 1322.

Lin, M., Zhang, H., Lao, J., Li, R., Zhou, Y., Yang, C., Cao, Y. and Tang, M., (2025) ToxicSQL: Migrating SQL Injection Threats into Text-to-SQL Models via Backdoor Attack. [online] Available at: http://arxiv.org/abs/2503.05445.

Nguyen, D.S.W. and Alrubie, D.F., (2025) Designing a Detection Model for SQL Injection Attack. *Journal of Computer and Communications*, [online] 1308, pp.40–79. Available at: https://www.scirp.org/journal/doi.aspx?doi=10.4236/jcc.2025.138003.

Nunes, P., Fonseca, J. and Vieira, M., (2025) Blending Static and Dynamic Analysis for Web Application Vulnerability Detection: Methodology and Case Study. *IEEE Access*, 13, pp.3139–3153.

Singh, B.P., Nigam, P.S. and Tech Scholar, M., (2025) *Detection of SQL Injection Attack Using Machine Learning Techniques: A Review*.

Song, Y., Liu, R., Chen, S., Ren, Q., Zhang, Y. and Yu, Y., (2024) *SecureSQL: Evaluating Data Leakage of Large Language Models as Natural Language Interfaces to Databases*. [online] Available at: https://github.

Sreenu, R., Suryawanshi, S.R. and Ashish, A., (2024) Experimental Study on Self-compacting and Self-healing Concrete with Recycled Coarse Aggregates. *International Journal of Engineering, Transactions A: Basics*, 374, pp.625–634.

Sun, H., Du, Y. and Li, Q., (2023) Deep Learning-Based Detection Technology for SQL Injection Research and Implementation. *Applied Sciences (Switzerland)*, 1316.

Tom Zayats, (2025) *Agentic Semantic Model Improvement: Elevating Text-to-SQL Performance*. [online] Available at: https://www.snowflake.com/en/engineering-blog/agentic-semantic-model-text-to-sql/ [Accessed 1 Sep. 2025].

Yaswanthraj, S., M, A., S, K. and R, J., (2024) SQL Injection and Prevention. *International Journal of Research Publication and Reviews*, 56, pp.1308–1317.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z. and Radev, D.R., (2018) *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. [online] Available at: http://www.databaseanswers.org/.